계획을 빼면 모두 사용자의 도구에서 나왔다. 특히 정의 25.1의 후보 집합 C는 블록체인에서 그래프 상수의 최소 곱 경로로, 21절 분야에서 사다리·π·반올림 부등식의 구간으로, ML에서 비율 편차의 보증으로 계산된다.

30 닫힘 점검

이 부의 모든 주장을 정의 25.1과 25.2로 점검했다. 판정되지 않고 남은 주장은 없다.

대상	정답 후보 C	판정	남은 주장
블록체인 계산 층	사다리 구간 $[S_{40}, S_{40} + r^{41}\epsilon_{40}]$의 정수	원본 증명(모든 x), 변이체 44개는 반례 43 또는 증명 1	0
블록체인 집계 층	정직한 보고값의 [최소, 최대]	가중 중앙값: 거짓 지분 < 1/2 증명, ≥ 1/2 반례. 확산 합의 반례. 구현 변이체 97개 반례 또는 증명	0
블록체인 진실 층	최소 곱 경로의 구간 합집합	거짓 있음: 증명(사이클) 또는 반례(정직한 세계). 위치: 증명(유일한 서명) 또는 반례(절단의 세계 쌍). 참값: 증명(k ≥ 2t + 1) 또는 반례(k ≤ 2t의 세계 쌍)	0
21절 분야	기준값 ± 분야의 상한(반올림·꼬리·허용치)	실행의 출력: 실행마다 증명 또는 반례. 원본: 증명(정리)·유한 증명·δ-증명·실행마다 증명, 선분 교차는 반례와 증명된 고친 판. 변이체: 반례(증인 입력) 또는 증명(27.2절의 근거)	0
계산할 수 없는 문제(정지 문제)	좁혀 가는 목록 C_t (정의 25.2)	건전 700/700, 단조, 맨 앞 후보가 참답으로 수렴(참답을 아는 700개 모두), 남은 목록에는 육하원칙 처방	0
ML	예측 집합(등각), 받아들인 예측은 한 점	(가) 반례(세계 증인), (나)–(라) δ-증명, (마) δ-증명/δ-반례, (바) 입력마다 증명 또는 반례(완전 판정), (사) δ-반례/δ-증명	0

실험이 잡은 판정기의 실수. 이 부를 쓰는 동안 실험이 판정기 자체의 실수를 잡았고, 모두 고친 뒤 다시 돌렸다. 시세 사이클의 띠에서 역방향 피드의 $1/(1-\eta_0)$를 빠뜨림(정직한 사이클 1,800개 중 거짓 경보 8개), 항을 하나 더 쓴 exp를 증명하지 못함, 거듭제곱법의 시험 입력이 지배 고유벡터를 늘 e_1로 만들어 곱셈 변이체를 놓침, 몬테카를로 π의 해상도, 선택적 판정의 느슨한 상한, 세계 쌍 실험에서 거짓 배율 1이 뽑힐 수 있었음(처음 267쌍 중 231쌍만 같음, 배율 1을 뺀 뒤 267쌍 모두 같음), 증인 비교에서 부동소수 0.4를 유리수 2/5와 비교함이다.

31 제6부 코드

chain_ladder.py — 26.2절: 나머지 사다리로 온체인 exp를 모든 입력에서 판정

```
"""블록체인 오라클 실험 A: 나머지 사다리로 온체인 고정소수 exp를 모든 입력에서 닫는다.

온체인 이자 누적·연속 복리는 e^r을 정수(WAD = 10^18 고정소수)로 계산한다.  코드가 주장하는 것은
    0 <= e^r·WAD - exp_wad(x, n) <= WAD·r^(n+1)·eps_n + B_n,   r = x / WAD in [0, 1],   eps_n = 1/(n!·n)
이다.  앞 항은 나머지 사다리(정의 5.1을 r^(n+1)배 한 것)의 꼬리, 뒤 항 B_n은 정수 내림이 쌓인 양이다.

판정(모든 x에서):
  기호 실행  x를 기호로, 내림 a // b를 a/b - δ_k(δ_k in [0, 1), b > 0)로 읽어 코드를 실행한다.
            (i)  δ가 없는 부분이 S_n(r)·WAD와 정확히 같은가(sympy, 결정적)
            (ii) D = S_n·WAD - 출력이 δ에 대해 계수가 모두 음이 아닌 다항식인가 -> D >= 0
            (iii) D의 최댓값(x = WAD, δ = 1)이 내림 예산 B_n 이하인가
            세 가지가 서면 [0, WAD]의 모든 정수 x에서 주장이 증명된다.  하나라도 깨지면 구체 x에서 정확한 구간과
            비교해 반례를 찾는다.
  정확한 e^r 구간은 사다리 자신으로 만든다: S_40(r) <= e^r <= S_40(r) + r^41·eps_40 (유리수).
"""
```

```
import math, random, json, signal
from fractions import Fraction as Fr
import sympy as sp

WAD = 10 ** 18


class Timeout(BaseException): pass     # 판정 안의 except Exception에 잡히지 않게

EXP_SRC = '''
def exp_wad(x, n):
    term = WAD
    s = WAD
    for k in range(1, n + 1):
        term = term * x // (k * WAD)
        s = s + term
    return s
'''


def eps(n): return Fr(1, math.factorial(n) * n)


def S(r, n):  # 부분합 S_n(r) (정확)
    t, s = Fr(1), Fr(1)
    for k in range(1, n + 1):
        t = t * r / k; s += t
    return s


def e_interval(r, N=40):
    """사다리로 만든 e^r의 정확한 유리수 구간 (r in [0, 1])."""
    lo = S(r, N); return lo, lo + r ** (N + 1) * eps(N)


def budget(n): return 2 * n            # 내림 예산 B_n (증명은 (iii)이 한다)


def ladder_n(tol_wei):
    """허용치(wei)를 사다리의 칸에서 고른다: WAD·eps_n + B_n <= tol 인 가장 작은 n."""
    for n in range(1, 60):
        if WAD * eps(n) + budget(n) <= tol_wei: return n
    return None


# ---------------------------------------------------------------- 기호 실행(내림 = 나눗셈 - δ)
class Sym:
    __slots__ = ("e",)
    cnt = [0]; deltas = []
    def __init__(s, e): s.e = sp.sympify(e)
    @staticmethod
    def c(o): return o.e if isinstance(o, Sym) else sp.Integer(o) if isinstance(o, int) else None
    def _b(s, o, f):
        oe = Sym.c(o)
        if oe is None: raise TypeError("Sym")
        return Sym(f(s.e, oe))
    def __add__(s, o): return s._b(o, lambda a, b: a + b)
    def __radd__(s, o): return s._b(o, lambda a, b: b + a)
    def __sub__(s, o): return s._b(o, lambda a, b: a - b)
    def __rsub__(s, o): return s._b(o, lambda a, b: b - a)
    def __mul__(s, o): return s._b(o, lambda a, b: a * b)
    def __rmul__(s, o): return s._b(o, lambda a, b: b * a)
    def _fd(s, a, b):
        if not (b.is_number): raise ValueError("범위 밖: 기호로 나누는 내림")
        if b == 0: raise ZeroDivisionError("0으로 나눔")
        d = sp.Symbol(f"d{Sym.cnt[0]}"); Sym.cnt[0] += 1; Sym.deltas.append((d, 1 if b > 0 else -1))
        return a / b - (d if b > 0 else -d)        # b < 0이면 내림 오차가 (-1, 0]
    def __floordiv__(s, o):
        oe = Sym.c(o)
        return Sym(s._fd(s.e, oe))
    def __rfloordiv__(s, o):
        return Sym(s._fd(Sym.c(o), s.e))
    def __truediv__(s, o): raise ValueError("범위 밖: 정수 코드에 실수 나눗셈")
    __rtruediv__ = __truediv__
    def __neg__(s): return Sym(-s.e)
    def __eq__(s, o): raise ValueError("범위 밖: 값 비교")
    __lt__ = __le__ = __gt__ = __ge__ = __ne__ = __eq__
    def __bool__(s): raise ValueError("범위 밖: 값의 참거짓")
    __hash__ = object.__hash__

```

```

def compile_src(src):
    g = dict(WAD=WAD); exec(compile(src, "<exp>", "exec"), g); return g["exp_wad"]


X = sp.Symbol("x", nonnegative=True)


def symbolic_close(src, n):
    """반환 dict(verdict=증명|불일치|범위 밖, why, D_max)."""
    fn = compile_src(src); Sym.cnt[0] = 0; Sym.deltas = []
    try:
        out = fn(Sym(X), n)
    except ZeroDivisionError as e: return dict(verdict="불일치", why=f"실행 실패 {e}")
    except (ValueError, TypeError) as e: return dict(verdict="범위 밖", why=str(e))
    except Exception as e: return dict(verdict="불일치", why=f"실행 실패 {type(e).__name__}")
    E = sp.expand(Sym.c(out) if isinstance(out, Sym) else sp.Integer(out))
    ds = [d for d, _ in Sym.deltas]
    free = sp.expand(E.subs({d: 0 for d in ds}))
    # 주장은 오차 상한이다: 내림을 뺀 식이 S_m(r)·WAD (m >= n)이면 꼬리는 r^(m+1)·eps_m <= r^(n+1)·eps_n 이라 주장 안이다
    m = next((m for m in range(n, n + 4) if sp.expand(free - sum(X ** k / (sp.factorial(k) * sp.Integer(WAD) ** (k - 1)) for k in range(m + 1))) == 0), None)
    if m is None: return dict(verdict="불일치", why="내림을 뺀 식이 어떤 S_m(r)·WAD (m >= n)와도 다르다")
    target = sp.expand(sum(X ** k / (sp.factorial(k) * sp.Integer(WAD) ** (k - 1)) for k in range(m + 1)))
    D = sp.expand(target - E)                       # = Σ (양의 계수 다항식) · δ 꼴이어야
    P = sp.Poly(D, X, *ds) if ds else None
    if P is not None:
        for mon, c in P.terms():
            if c < 0: return dict(verdict="불일치", why="내림 오차의 계수가 음수(출력이 S_n보다 클 수 있다)")
        sgn = all(sg > 0 for _, sg in Sym.deltas)
        if not sgn: return dict(verdict="불일치", why="음수로 나누는 내림")
    Dmax = D.subs({X: WAD, **{d: 1 for d in ds}}) if ds else sp.Integer(0)
    if Dmax > budget(n): return dict(verdict="불일치", why=f"내림 예산 초과: 최대 {Dmax} > {budget(n)}", D_max=str(Dmax))
    return dict(verdict="증명", D_max=str(sp.nsimplify(Dmax)), n_floor=len(ds), terms=m)


def concrete_check(fn, n, xs):
    """구체 x에서 주장 0 <= e^r·WAD - 출력 <= WAD·r^(n+1)·eps_n + B_n 을 정확한 구간으로."""
    for x in xs:
        try: out = fn(x, n)
        except Exception as e: return dict(x=x, why=f"실행 실패 {type(e).__name__}")
        if not isinstance(out, int): return dict(x=x, why="정수가 아닌 출력")
        r = Fr(x, WAD); lo, hi = e_interval(r)
        err_lo, err_hi = lo * WAD - out, hi * WAD - out      # 참 오차는 이 사이
        bound = WAD * r ** (n + 1) * eps(n) + budget(n)
        if err_hi < 0: return dict(x=x, why=f"출력이 e^r보다 크다 ({float(-err_hi):.3g} wei)")
        if err_lo > bound: return dict(x=x, why=f"오차 {float(err_lo):.4g} wei > 주장 {float(bound):.4g} wei")
    return None


def judge(src, n, seed=0):
    sym = symbolic_close(src, n)
    if sym["verdict"] == "증명": return dict(sym)
    fn = compile_src(src); rng = random.Random(seed)
    xs = [0, 1, 2, WAD // 2, WAD - 1, WAD] + [rng.randrange(WAD + 1) for _ in range(400)] + [rng.randrange(10 ** k) for k in range(1, 18)]
    c = concrete_check(fn, n, xs)
    if c: return dict(verdict="반례", sym=sym["why"], **c)
    return dict(verdict="증명 실패, 반례 미발견", sym=sym["why"])


def baseline(src, n, seed=1):
    """흔한 단위 시험: 무작위 x 100개에서 float math.exp와 상대 1e-12 이내인가."""
    fn = compile_src(src); rng = random.Random(seed)
    for _ in range(100):
        x = rng.randrange(WAD + 1)
        try: out = fn(x, n)
        except Exception: return True
        try:
            if abs(out - math.exp(x / WAD) * WAD) > 1e-12 * WAD: return True
        except Exception: return True
    return False


```

```
if __name__ == "__main__":
    import mutate
    res = dict()
    # 1) 사다리로 n을 고른다
    res["ladder"] = [dict(tol=t, n=ladder_n(t), tail=float(WAD * eps(ladder_n(t))), B=budget(ladder_n(t)
)) for t in (10 ** 12, 10 ** 9, 10 ** 6, 10 ** 3, 100)]
    print(res["ladder"], flush=True)
    # 2) 원본: 기호 증명 + 구체 확인(경계값, 무작위 2000개)
    n0 = ladder_n(10 ** 3)
    res["orig_sym"] = symbolic_close(EXP_SRC, n0)
    fn = compile_src(EXP_SRC); rng = random.Random(7)
    xs = [0, 1, WAD // 3, WAD - 1, WAD] + [rng.randrange(WAD + 1) for _ in range(2000)]
    res["orig_concrete"] = concrete_check(fn, n0, xs)
    worst = max((e_interval(Fr(x, WAD))[1] * WAD - fn(x, n0) for x in xs))
    res["orig_worst_err_wei"] = float(worst); res["orig_claim_wei"] = float(WAD * eps(n0) + budget(n0));
 res["n0"] = n0
    print("원본", res["orig_sym"], res["orig_concrete"], res["orig_worst_err_wei"], res["orig_claim_wei"
], flush=True)
    # 3) 변이체
    rows = []
    for m in mutate.mutants(EXP_SRC):
        kind, where, line, src = m
        def al(*a): raise Timeout()
        signal.signal(signal.SIGALRM, al); signal.alarm(20)
        try: j = judge(src, n0)
        except Timeout: j = dict(verdict="시간 초과(끝나지 않는 실행)")
        signal.alarm(10)
        try: bl = baseline(src, n0)
        except Timeout: bl = True
        finally: signal.alarm(0)
        rows.append(dict(kind=kind, where=str(where), line=line, src=src, verdict=j["verdict"], info={k:
 str(v) for k, v in j.items() if k != "verdict"}, baseline=bl))
        print(kind, where, line, j["verdict"], "기준선 잡음" if bl else "기준선 통과", flush=True)
    res["mutants"] = rows
    json.dump(res, open("chain_ladder_results.json", "w"), ensure_ascii=False, indent=1)
```

chain_graph.py — 26.3–26.4절: 그래프 상수로 집계·진실 층

```
"""블록체인 오라클 실험 B·C: 그래프 상수 (V, T)와 나머지 사다리로 보고값 집계와 거짓 보고를 판정한다.

B  집계: 보고자 i는 지분 V_i(마디 무게), 보고자 사이의 결합은 T_ij(변 무게).  4.4절의 식
       du_i/dt = (1/V_i) Σ_j T_ij (u_j - u_i)
   가 '서로 평균을 내며 합의'하는 확산 합의이다.  그래프 상수의 세 구조(행합 0, 대칭 T, 메츨러)에서
   B1 지분 가중 평균이 보존되고 합의값은 그 평균이다 -> 보고자 i의 영향력은 정확히 V_i / ΣV.
   B2 갱신하지 않는 '고집 보고자' 하나가 있으면, 지분이 아무리 작아도 합의값은 그 보고값이 된다.
   B3 가중 중앙값(마디 무게 V): 거짓 지분 < 1/2이면 출력은 정직한 값의 [최소, 최대] 안 — 전수로 확인.
      거짓 지분 >= 1/2이면 반례가 있고, 두 세계를 구별할 수 없다.
   B4 가중 중앙값 구현의 변이체를 순서형 전수로 판정한다.
C  진실 검출: 자산이 마디, 시세 피드가 변(r_ij = p_j / p_i)인 그래프.  사이클의 곱은 정직하면 정확히 1.
   C1 거짓 피드 하나: 검출 <=> 그 변이 다리가 아님.  위치 지정 <=> 그 변과 2-변 절단을 이루는 변이 없음.
   C2 정직한 잡음 |η| <= η0가 있을 때 길이 k 사이클의 허용 띠 [1 - kη0, U_n(kη0)]을 나머지 사다리로 잡는
다:
      (1 + η0)^k <= e^(kη0) <= S_n(kη0) + (kη0)^(n+1)·eps_n = U_n.  정직한 피드는 경보를 내지 않는다(증
명),
      거짓 배율이 띠의 비 밖이면 반드시 잡는다(증명), 그 사이는 회색 지대로 따로 센다.
   C3 한 자산의 피드를 모두 장악한 거짓은 정직한 다른 세계와 관측이 같다(구별 불가능) — 쌍을 실제로 만든
다.
모든 비교는 정확한 유리수이다."""
import itertools, json, math, random, signal
from fractions import Fraction as Fr
import sympy as sp


def eps(n): return Fr(1, math.factorial(n) * n)
def S(r, n):
    t, s = Fr(1), Fr(1)
    for k in range(1, n + 1): t = t * r / k; s += t
    return s
def U_ladder(a, n): return S(a, n) + a ** (n + 1) * eps(n)          # e^a의 사다리 상한 (0 <= a <= 1)


def rand_graph(rng, n, extra):
    E = set()
    for i in range(1, n): E.add((rng.randrange(i), i))               # 무작위 생성 나무(연결)
    while len(E) < n - 1 + extra:
        a, b = sorted(rng.sample(range(n), 2)); E.add((a, b))
    return sorted(E)


```

```
def lap(n, V, T):
    """식 (G): L_ij = T_ij / V_i (j ~ i), L_ii = -Σ_j T_ij / V_i  (정확한 유리수)."""
    L = [[Fr(0)] * n for _ in range(n)]
    for (i, j), t in T.items():
        L[i][j] += t / V[i]; L[j][i] += t / V[j]; L[i][i] -= t / V[i]; L[j][j] -= t / V[j]
    return L


# ---------------------------------------------------------------- B1, B2
def exp_B12(trials=60, n=10, seed=1):
    rng = random.Random(seed); out = dict(conserve=0, influence=0, stubborn=0, trials=trials, shift=[], stubborn_share=[])
    for t in range(trials):
        E = rand_graph(rng, n, rng.randrange(0, 12)); V = [Fr(rng.randrange(1, 40)) for _ in range(n)]
        T = {e: Fr(rng.randrange(1, 6)) for e in E}; L = lap(n, V, T)
        W = sum(V)
        # 행합 0, 지분 가중 열합 0 (V^T L = 0): 가중 평균 보존
        ok = all(sum(L[i]) == 0 for i in range(n)) and all(sum(V[i] * L[i][j] for i in range(n)) == 0 for j in range(n))
        out["conserve"] += ok
        # 합의값 = 가중 평균: 정상 상태는 상수 벡터 c·1이고 보존량에서 c = Σ V_i u_i / ΣV.  영향력 dc/du_i = V_i / ΣV
        M = sp.Matrix(n, n, lambda i, j: sp.Rational(L[i][j].numerator, L[i][j].denominator))
        ns = M.T.nullspace()                                          # 왼쪽 영공간(연결이면 1차원)
        w = ns[0] / sum(ns[0]); infl = [Fr(int(sp.fraction(x)[0]), int(sp.fraction(x)[1])) for x in w]
        out["influence"] += infl == [v / W for v in V] and len(ns) == 1
        # 거짓 보고 한 명: 지분이 가장 작은 보고자가 10^6을 보고, 정직한 값은 100 근처
        liar = min(range(n), key=lambda i: V[i]); u = [Fr(100) + Fr(rng.randrange(-100, 101), 100) for _ in range(n)]
        honest = sum(V[i] * u[i] for i in range(n)) / W; u[liar] = Fr(10 ** 6)
        lied = sum(V[i] * u[i] for i in range(n)) / W
        out["shift"].append(dict(share=float(V[liar] / W), shift=float(lied - honest)))
        # B2: 고집 보고자(갱신하지 않음) — 정직한 마디의 정상 상태 L_hh u_h = -L_hs M
        s = liar; H = [i for i in range(n) if i != s]
        Mhh = sp.Matrix(len(H), len(H), lambda a, b: sp.Rational(L[H[a]][H[b]].numerator, L[H[a]][H[b]].denominator))
        Mhs = sp.Matrix(len(H), 1, lambda a, _: sp.Rational(L[H[a]][s].numerator, L[H[a]][s].denominator))
        uh = Mhh.LUsolve(-Mhs * 10 ** 6)
        out["stubborn"] += all(x == 10 ** 6 for x in uh)
        out["stubborn_share"].append(float(V[s] / W))
    return out


# ---------------------------------------------------------------- B3: 가중 중앙값의 과반 정리(전수)
def wmed_ref(vals, st):
    """정의: 정렬했을 때 누적 지분의 두 배가 총 지분 이상이 되는 첫 값(아래 가중 중앙값)."""
    tot = sum(st); acc = 0
    for v, s in sorted(zip(vals, st), key=lambda t: t[0]):
        acc += s
        if 2 * acc >= tot: return v


def exp_B3(Nmax=6, grid=(1, 2, 3)):
    """정직한 값 1..h(서로 다름), 거짓 보고자는 정직한 값 사이의 모든 틈(아래 끝·위 끝 포함)에 놓는다 = 모든 순서형."""
    res = dict(cases=0, viol_minor=0, cases_major=0, viol_major=0, example_major=None)
    for N in range(2, Nmax + 1):
        for st in itertools.product(grid, repeat=N):
            tot = sum(st)
            for nb in range(0, N):
                for bad in itertools.combinations(range(N), nb):
                    hon = [i for i in range(N) if i not in bad]
                    bst = sum(st[i] for i in bad)
                    gaps = [Fr(2 * g - 1, 2) for g in range(len(hon) + 1)]      # 0.5, 1.5, ..., h + 0.5
                    for place in itertools.product(gaps, repeat=nb):
                        vals = [None] * N
                        for k, i in enumerate(hon): vals[i] = Fr(k + 1)
                        for k, i in enumerate(bad): vals[i] = place[k]
                        m = wmed_ref(vals, st); inside = 1 <= m <= len(hon)
                        if 2 * bst < tot:
                            res["cases"] += 1; res["viol_minor"] += not inside
                        else:
                            res["cases_major"] += 1
                            if not inside:
                                res["viol_major"] += 1
                                if res["example_major"] is None:
                                    res["example_major"] = dict(stakes=list(st), bad=list(bad), vals=[str(v) for v in vals], out=str(m))
    return res
```

```


WMED_SRC = '''
def wmedian(vals, stakes):
    order = sorted(range(len(vals)), key=lambda i: vals[i])
    total = sum(stakes)
    acc = 0
    for i in order:
        acc = acc + stakes[i]
        if 2 * acc >= total:
            return vals[i]
    return vals[order[-1]]
'''


def weak_orders(N):
    """값의 모든 약순서(동점 포함): 순위 벡터."""
    seen = set()
    for r in itertools.product(range(N), repeat=N):
        used = sorted(set(r))
        if used != list(range(len(used))): continue
        if r not in seen: seen.add(r); yield r


def judge_wmed(src):
    g = {}; exec(compile(src, "<wmed>", "exec"), g); fn = g["wmedian"]
    for N in range(1, 5):
        for r in weak_orders(N):
            for st in itertools.product((1, 2, 3), repeat=N):
                vals = [Fr(x) for x in r]
                try: o = fn(list(vals), list(st))
                except Exception as e: return dict(verdict="반례", where=f"값 순위 {r}, 지분 {st}: 실행 실패 {type(e).__name__}")
                if o != wmed_ref(vals, st): return dict(verdict="반례", where=f"값 순위 {r}, 지분 {st}: {o} != {wmed_ref(vals, st)}")
    for r in itertools.permutations(range(5)):
        for st in itertools.product((1, 2, 3), repeat=5):
            vals = [Fr(x) for x in r]
            try: o = fn(list(vals), list(st))
            except Exception as e: return dict(verdict="반례", where=f"값 순위 {r}, 지분 {st}: 실행 실패 {type(e).__name__}")
            if o != wmed_ref(vals, st): return dict(verdict="반례", where=f"값 순위 {r}, 지분 {st}: {o} != {wmed_ref(vals, st)}")
    return dict(verdict="유한 증명")


def base_wmed(src):
    """흔한 시험: 손으로 쓴 사례 3개."""
    g = {}; exec(compile(src, "<wmed>", "exec"), g); fn = g["wmedian"]
    cases = [(([1, 2, 3], [1, 1, 1]), 2), (([5, 1, 9, 3], [1, 1, 1, 1]), 3), (([10, 20, 30], [1, 1, 5]), 30)]
    try: return any(fn(*a) != b for a, b in cases)
    except Exception: return True


def exp_B4():
    import mutate
    rows = []
    for kind, where, line, src in mutate.mutants(WMED_SRC):
        def al(*a): raise TimeoutError
        signal.signal(signal.SIGALRM, al); signal.alarm(60)
        try: j = judge_wmed(src)
        except TimeoutError: j = dict(verdict="시간 초과")
        finally: signal.alarm(0)
        rows.append(dict(kind=kind, where=str(where), line=line, src=src, verdict=j["verdict"], info=j.get("where", ""), baseline=base_wmed(src)))
    return dict(orig=judge_wmed(WMED_SRC), rows=rows)


# ------------------------------------------------------------------ C: 시세 그래프의 사이클
def connected(n, E):
    adj = {i: [] for i in range(n)}
    for a, b in E: adj[a].append(b); adj[b].append(a)
    seen = {0}; st = [0]
    while st:
        x = st.pop()
        for y in adj[x]:
            if y not in seen: seen.add(y); st.append(y)
    return len(seen) == n


```

```
185
186 def cycle_basis(n, E):
187     """BFS 생성 나무 + 나머지 변마다 기본 사이클(변 번호, 방향 부호) 목록."""
188     adj = {i: [] for i in range(n)}
189     for k, (a, b) in enumerate(E): adj[a].append((b, k, +1)); adj[b].append((a, k, -1))
190     par = {0: None}; q = [0]; tree = set()
191     for x in q:
192         for y, k, sg in adj[x]:
193             if y not in par: par[y] = (x, k, sg); tree.add(k); q.append(y)
194     def path(x):                                    # 뿌리 -> x 경로의 (변, 부호)
195         p = []
196         while par[x] is not None: px, k, sg = par[x]; p.append((k, sg)); x = px
197         return p[::-1]
198     cyc = []
199     for k, (a, b) in enumerate(E):
200         if k in tree: continue
201         pa, pb = path(a), path(b)
202         c = {}
203         for e, sg in pa: c[e] = c.get(e, 0) + sg     # 뿌리 -> a
204         c[k] = c.get(k, 0) + 1                        # a -> b
205         for e, sg in pb: c[e] = c.get(e, 0) - sg     # b -> 뿌리
206         cyc.append({e: s for e, s in c.items() if s})
207     return cyc
208
209
210 def cyc_prod(cy, R):
211     p = Fr(1)
212     for e, s in cy.items(): p *= R[e] ** s
213     return p
214
215
216 def exp_C1(trials=60, n=8, seed=3):
217     rng = random.Random(seed)
218     res = dict(edges=0, bridges=0, detect_ok=0, local_ok=0, cut_ok=0, unique=0, detected=0, dim=[])
219     for t in range(trials):
220         E = rand_graph(rng, n, rng.randrange(1, 7)); p = [Fr(rng.randrange(50, 5000), rng.randrange(1, 5
    0)) for _ in range(n)]
221         R = [p[b] / p[a] for a, b in E]; cyc = cycle_basis(n, E); res["dim"].append(len(cyc))
222         assert all(cyc_prod(c, R) == 1 for c in cyc)                # 정직: 모든 사이클 곱 = 1 (정확)
223         sig = [frozenset(i for i, c in enumerate(cyc) if e in c) for e in range(len(E))]
224         for e in range(len(E)):
225             R2 = list(R); R2[e] = R[e] * Fr(11, 10)                 # 10% 거짓
226             fail = frozenset(i for i, c in enumerate(cyc) if cyc_prod(c, R2) != 1)
227             bridge = not connected(n, [E[k] for k in range(len(E)) if k != e])
228             res["edges"] += 1; res["bridges"] += bridge
229             res["detect_ok"] += (len(fail) > 0) == (not bridge); res["detected"] += len(fail) > 0
230             cand = [f for f in range(len(E)) if sig[f] == fail] if fail else []
231             res["local_ok"] += (not fail) or (e in cand)
232             uniq = len(cand) == 1; res["unique"] += uniq
233             # 정리: 후보가 둘 이상 <=> 다리가 아닌 다른 변 f와 {e, f}가 2-변 절단
234             partner = any(not connected(n, [E[k] for k in range(len(E)) if k not in (e, f)]) for f in ra
    nge(len(E)) if f != e
235                           and connected(n, [E[k] for k in range(len(E)) if k != f]))
236             if fail: res["cut_ok"] += (len(cand) > 1) == partner
237             else: res["cut_ok"] += 1
238     return res
239
240
241 def honest_world_exists(n, E, obs, eta0):
242     """정확한 완전 판정: 잡음 |η| <= η0 인 정직한 세계(값 p', 잡음 η')가 관측을 그대로 낼 수 있는가.
243     p'_b / p'_a 는 [obs/(1+η0), obs/(1-η0)] 안이어야 한다.  곱셈 꼴의 차분 제약이므로, 비율 상한을 변 무
    게로 둔
244     그래프(그래프 상수)에서 곱이 1보다 작은 사이클이 없을 때, 그리고 그때만 그런 세계가 있다(플로이드–워
    셜, 정확 유리수)."""
245     INF = None; D = [[INF] * n for _ in range(n)]
246     for v in range(n): D[v][v] = Fr(1)
247     def relax(a, b, w):
248         if D[a][b] is None or w < D[a][b]: D[a][b] = w
249     for k, (a, b) in enumerate(E):
250         relax(a, b, obs[k] / (1 - eta0)); relax(b, a, (1 + eta0) / obs[k])
251     for k in range(n):
252         for i in range(n):
253             if D[i][k] is None: continue
254             for j in range(n):
255                 if D[k][j] is None: continue
256                 w = D[i][k] * D[k][j]
257                 if D[i][j] is None or w < D[i][j]: D[i][j] = w
258     return all(D[v][v] >= 1 for v in range(n))
259
260
```

```
def exp_C2(trials=300, n=8, extra=6, seed=5, eta0=Fr(1, 1000), nlad=None):
    """잡음 |η| <= η0 아래 사이클 허용 띠를 사다리로.  띠: [1 - kη0, U_n(kη0)]."""
    rng = random.Random(seed)
    if nlad is None:                                  # 사다리 칸: 꼬리가 띠 폭의 1/1000 이하가 되는 n
        nlad = next(m for m in range(1, 40) if eps(m) <= eta0 / 1000)
    res = dict(n_ladder=nlad, eta0=str(eta0), honest_cycles=0, false_alarm=0, false_alarm_first_band=0,
sure_lies=0, sure_caught=0, gray=0, gray_caught=0, bridge_lies=0, bridge_caught=0)
    for t in range(trials):
        E = rand_graph(rng, n, extra); p = [Fr(rng.randrange(50, 5000), rng.randrange(1, 50)) for _ in r
ange(n)]
        cyc = cycle_basis(n, E)
        def noise(): return rng.choice([eta0, -eta0, Fr(rng.randrange(-1000, 1001), 1000) * eta0])
        eta_e = [noise() for _ in E]
        R = [p[b] / p[a] * (1 + eta_e[k]) for k, (a, b) in enumerate(E)]
        def band(c, fix=True):
            k = sum(abs(s) for s in c.values()); a = k * eta0
            # 역방향으로 지나는 변은 1/(1+η)라서 위로 1/(1-η)까지 간다: 위 띠는 e^(kη0/(1-η0))의 사다리
상한
            return 1 - a, U_ladder(k * eta0 / (1 - eta0) if fix else a, nlad)
        def alarm(c, RR, fix=True):
            lo, hi = band(c, fix); x = cyc_prod(c, RR)
            return x < lo or x > hi
        for c in cyc:
            res["honest_cycles"] += 1; res["false_alarm"] += alarm(c, R); res["false_alarm_first_band"]
+= alarm(c, R, False)
        res["exact_honest_alarm"] = res.get("exact_honest_alarm", 0) + (not honest_world_exists(n, E, R,
 eta0))
        e = rng.randrange(len(E)); lam = Fr(rng.choice([1, 2, 5, 10, 20, 50, 100, 500]), 10000) * rng.ch
oice([1, -1])
        R2 = list(R); R2[e] = R[e] * (1 + lam); caught = any(alarm(c, R2) for c in cyc)
        exact = not honest_world_exists(n, E, R2, eta0)            # 정확한 완전 판정: 설명하는 정직한
세계가 없으면 거짓 확정
        res["exact_lies"] = res.get("exact_lies", 0) + 1; res["exact_caught"] = res.get("exact_caught",
0) + exact
        res["band_caught_but_exact_not"] = res.get("band_caught_but_exact_not", 0) + (caught and not exa
ct)
        inc = [c for c in cyc if e in c]
        if not inc:
            res["bridge_lies"] += 1; res["bridge_caught"] += caught; res["bridge_exact"] = res.get("brid
ge_exact", 0) + exact; continue
        # 보장 조건: 그 변을 지나는 어떤 사이클에서 (1+λ)^{±1}이 띠의 비 hi/lo 밖
        sure = any((lambda lo, hi: (1 + lam) ** abs(c[e]) > hi / lo or (1 + lam) ** abs(c[e]) < lo / hi)
(*band(c)) for c in inc)
        if sure: res["sure_lies"] += 1; res["sure_caught"] += caught; res["sure_exact"] = res.get("sure_
exact", 0) + exact
        else:
            res["gray"] += 1; res["gray_caught"] += caught; res["gray_exact"] = res.get("gray_exact", 0)
 + exact
            if not caught:
                tot = (1 + eta_e[e]) * (1 + lam) - 1        # 그 변의 관측 = 참값 × (1 + tot)
                res["gray_missed_honest_expl"] = res.get("gray_missed_honest_expl", 0) + (abs(tot) <= et
a0)
                res["gray_missed"] = res.get("gray_missed", 0) + 1
    return res


def exp_C3(trials=50, n=8, extra=5, seed=9):
    """자산 v의 피드를 모두 장악해 p_v를 c배로 꾸민 관측 == 정직한 세계(p_v' = c·p_v)의 관측.  같으면 어
떤 판정기도 구별 못 한다."""
    rng = random.Random(seed); same = 0; det_any = 0
    for t in range(trials):
        E = rand_graph(rng, n, extra); p = [Fr(rng.randrange(50, 5000), rng.randrange(1, 50)) for _ in r
ange(n)]
        v = rng.randrange(n); c = Fr(rng.choice([2, 3, 4, 5, 6, 8, 9]), 7)      # c ≠ 1
        lied = [(p[b] * (c if b == v else 1)) / (p[a] * (c if a == v else 1)) for a, b in E]   # 거짓: v
의 모든 피드
        p2 = list(p); p2[v] = p[v] * c; honest2 = [p2[b] / p2[a] for a, b in E]                 # 다른
정직한 세계
        same += lied == honest2
        det_any += any(cyc_prod(cy, lied) != 1 for cy in cycle_basis(n, E))
    return dict(trials=trials, identical=same, detected=det_any)


def exp_C4(trials=60, n=8, seed=11):
    """2-변 절단 {e, f}: 세계 1은 e가 λ배 거짓, 세계 2는 절단의 한쪽 자산 값이 다르고 f가 거짓.  관측이
같은가."""
    rng = random.Random(seed); pairs = same = 0
    for t in range(trials):
        E = rand_graph(rng, n, rng.randrange(1, 5)); p = [Fr(rng.randrange(50, 5000), rng.randrange(1, 5
0)) for _ in range(n)]
```

```
        nb = [k for k in range(len(E)) if connected(n, [E[j] for j in range(len(E)) if j != k])]
        for i in range(len(nb)):
            for j in range(i + 1, len(nb)):
                e, f = nb[i], nb[j]; rest = [E[k] for k in range(len(E)) if k not in (e, f)]
                if connected(n, rest): continue
                # 절단의 한쪽 Y: 마디 0과 이어지지 않은 쪽
                adj = {v: [] for v in range(n)}
                for a, b in rest: adj[a].append(b); adj[b].append(a)
                X = {0}; st = [0]
                while st:
                    v = st.pop()
                    for w in adj[v]:
                        if w not in X: X.add(w); st.append(w)
                lam = Fr(rng.choice([2, 3, 4, 5, 6, 8, 9]), 7)          # λ ≠ 1 (거짓이 실제로 있는 경우만)
                obs1 = [p[b] / p[a] for a, b in E]; obs1[e] = obs1[e] * lam              # 세계 1: e가 λ배 거짓
                ae, be = E[e]; c = lam if (ae in X) != (be in X) and be not in X else 1 / lam   # e를 정직하게 만드는 Y쪽 배율
                p2 = [p[v] * (1 if v in X else c) for v in range(n)]
                obs2 = [p2[b] / p2[a] for a, b in E]
                af, bf = E[f]; truef = obs2[f]; obs2[f] = p[bf] / p[af]                    # 세계 2: f가 거짓(옛 값을 보고)
                pairs += 1; same += obs1 == obs2 and obs2[e] == obs1[e] and truef != obs2[f]
    return dict(pairs=pairs, identical=same)


if __name__ == "__main__":
    out = {}
    import sys
    if len(sys.argv) > 1 and sys.argv[1] == "C24":
        out = json.load(open("chain_graph_results.json")); out["C2"] = exp_C2(); print("C2", out["C2"]); out["C4"] = exp_C4(); print("C4", out["C4"])
        json.dump(out, open("chain_graph_results.json", "w"), ensure_ascii=False, indent=1); sys.exit()
    if len(sys.argv) > 1 and sys.argv[1] == "C2":
        out = json.load(open("chain_graph_results.json")); out["C2"] = exp_C2(); print("C2", out["C2"])
        json.dump(out, open("chain_graph_results.json", "w"), ensure_ascii=False, indent=1); sys.exit()
    out["B12"] = exp_B12(); print("B12", {k: v for k, v in out["B12"].items() if k not in ("shift", "stubborn_share")}, flush=True)
    out["B3"] = exp_B3(); print("B3", out["B3"], flush=True)
    out["B4"] = exp_B4(); print("B4 원본", out["B4"]["orig"], "변이", len(out["B4"]["rows"]), flush=True)
    out["C1"] = exp_C1(); print("C1", {k: v for k, v in out["C1"].items() if k != "dim"}, flush=True)
    out["C2"] = exp_C2(); print("C2", out["C2"], flush=True)
    out["C3"] = exp_C3(); print("C3", out["C3"], flush=True)
    json.dump(out, open("chain_graph_results.json", "w"), ensure_ascii=False, indent=1)
```

작동 확인 chain_fig.py — 그림 4

```
"""보고서 그림 1: (가) 보고자 그래프(V = 지분, T = 결합)와 고집 보고자, (나) 시세 피드 그래프의 다리·2-변 절단·사이클 변."""
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from matplotlib import font_manager as fm
import numpy as np

fp = "/usr/share/fonts/opentype/noto/NotoSansCJK-Regular.ttc"
fm.fontManager.addfont(fp); plt.rcParams["font.family"] = fm.FontProperties(fname=fp).get_name()
INK, MUTED, RED, BLUE, ORANGE = "#222222", "#8a8a8a", "#c0392b", "#2c6fb7", "#d98c1f"

fig, ax = plt.subplots(1, 2, figsize=(10.4, 4.3))
# (가) 보고자 그래프
a = ax[0]; rng = np.random.default_rng(4)
pos = {i: (np.cos(2 * np.pi * i / 7), np.sin(2 * np.pi * i / 7)) for i in range(7)}; pos[7] = (0.0, 0.0)
V = [14, 9, 20, 11, 16, 8, 12, 1]
E = [(0, 1), (1, 2), (2, 3), (3, 4), (4, 5), (5, 6), (6, 0), (7, 0), (7, 3), (7, 5), (1, 4)]
for i, j in E:
    x = [pos[i][0], pos[j][0]]; y = [pos[i][1], pos[j][1]]
    a.plot(x, y, color=RED if 7 in (i, j) else MUTED, lw=1.2 if 7 not in (i, j) else 1.6, zorder=1)
for i, (x, y) in pos.items():
    a.scatter([x], [y], s=V[i] * 40, color=RED if i == 7 else BLUE, edgecolor="white", lw=1.5, zorder=2)
    a.text(x, y - (0.2 if i != 7 else 0.17), f"V={V[i]}", ha="center", va="top", fontsize=8, color=INK)
a.text(0.0, 0.2, "고집 보고자\n(지분 1%, 1,000,000 보고)", ha="center", va="bottom", fontsize=8.5, color=RED)
a.set_title("(가) 보고자 그래프: 마디 무게 V = 지분, 변 무게 T = 결합", fontsize=10, color=INK)
a.text(0, -1.42, "확산 합의: 모든 보고자가 1,000,000으로 간다(붕괴점 0)\n가중 중앙값: 거짓 지분 < 1/2이면 정직한 범위 안", ha="center", va="top", fontsize=9, color=INK)
a.set_xlim(-1.5, 1.5); a.set_ylim(-1.9, 1.35); a.set_aspect("equal"); a.axis("off")
```

```
# (나) 시세 피드 그래프
b = ax[1]
P = {"A": (0, 1), "B": (1.1, 1.25), "C": (1.6, 0.25), "D": (0.6, -0.2), "E": (2.7, 0.3), "F": (3.5, 0.9), "G": (3.5, -0.35)}
cyc = [("A", "B"), ("B", "C"), ("C", "D"), ("D", "A"), ("A", "C"), ("B", "D")]
cut = [("E", "F"), ("E", "G"), ("F", "G")]
bridge = [("C", "E")]
def line(e, c, ls="-", lw=1.8):
    (x1, y1), (x2, y2) = P[e[0]], P[e[1]]; b.plot([x1, x2], [y1, y2], color=c, ls=ls, lw=lw, zorder=1)
for e in cyc: line(e, BLUE)
for e in cut: line(e, ORANGE, lw=2.4)
for e in bridge: line(e, RED, "--", 2.4)
for k, (x, y) in P.items():
    b.scatter([x], [y], s=360, color="white", edgecolor=INK, lw=1.2, zorder=2); b.text(x, y, k, ha="center", va="center", fontsize=10, color=INK, zorder=3)
b.plot([], [], color=BLUE, lw=1.8, label="사이클 위의 변: 거짓을 검출하고 위치를 찾는다")
b.plot([], [], color=ORANGE, lw=2.4, label="사이클 하나에만 함께 있는 변(2-변 절단): 검출하지만 어느 변인지 모른다")
b.plot([], [], color=RED, lw=2.4, ls="--", label="다리: 거짓을 검출할 수 없다")
b.legend(loc="upper center", bbox_to_anchor=(0.5, -0.04), fontsize=8.5, frameon=False)
b.set_title("(나) 시세 피드 그래프: 마디 = 자산, 변 = 피드 r = p_j / p_i", fontsize=10, color=INK)
b.set_xlim(-0.4, 3.9); b.set_ylim(-0.9, 1.6); b.set_aspect("equal"); b.axis("off")
plt.tight_layout(); plt.savefig("chain_fig.png", dpi=200, facecolor="white", bbox_inches="tight", pad_inches=0.15)
print("chain_fig.png")
```

작동 확인 anytime_oracle.py — 정의 25.2: 정지 문제의 좁혀 가는 후보 목록

```
"""계산할 수 없는 오라클 문제의 '좁혀 가는 후보 목록'(정의 25.2) — 정지 문제로 실험한다.

판정할 출력: "프로그램 P는 입력 x에서 멈춘다 / 멈추지 않는다".  정지 문제는 결정 불가능하다(튜링).
좁혀 가는 오라클: 걸음 예산 t = 10, 100, 10^3, 10^4, 10^5마다 후보 집합 C_t와 맨 앞 후보 g_t를 낸다.
  - P가 t걸음 안에 멈추면 C_t = {멈춤}  (증명: 멈춘 실행 기록이 증인)
  - 상태가 되풀이되면 C_t = {멈추지 않음}  (증명: 결정적 실행이 같은 상태로 돌아오면 영원히 돈다 — 되풀이 쌍이 증인)
  - 값이 법 m의 유한 집합 안에 갇혀 있고 그 안에서 멈춤 조건에 닿지 않음이 확인되면(순환을 끝까지 돌았음) 같음
  - 아니면 C_t = {멈춤, 멈추지 않음}, g_t = 멈추지 않음(아직 안 멈췄으므로).  육하원칙으로 왜 남았는지와 처방을 낸다.
성질: (건전) 참답은 늘 C_t 안에 있다.  (단조) C_t는 t가 커지면 줄기만 한다.  (극한) g_t는 참답으로 수렴한다 —
멈추는 P는 언젠가 멈추고, 멈추지 않는 P는 g_t가 처음부터 '멈추지 않음'이다(쇼엔필드의 극한 보조정리).
다만 '지금의 g_t가 최종인가'는 그 순간에 알 수 없다(그것을 알면 정지 문제가 풀린다).

대상(참답을 아는 것과 모르는 것):
  A 유한 상태 반복   x ← (a·x + b) mod m, x == h이면 멈춤                          참답: 법 m 안에서 완전히 계산
  B 콜라츠           x ← x/2 (짝수), 3x + 1 (홀수), x == 1이면 멈춤, 1 <= x <= 3000   참답: 모두 멈춤(계산으로 확인된 범위)
  C 홀짝 감소        x ← x − 2, x == 0이면 멈춤                                    참답: x가 0 이상 짝수일 때만 멈춤
  D 5x + 1 문제      x ← x/2 (짝수), 5x + 1 (홀수), x == 1이면 멈춤                 참답: 일부는 순환(증명 가능), 일부는 알려지지 않음
"""
import json, random

BUDGETS = [10, 100, 1000, 10 ** 4, 10 ** 5]


def run(step, x0, halt, budget, state_key=None):
    """결정적 실행.  반환: ('멈춤', t) | ('순환', t, 주기) | ('미정', budget, 최대 크기)."""
    seen = {}; x = x0; big = abs(x)
    for t in range(budget + 1):
        if halt(x): return ("멈춤", t)
        k = state_key(x) if state_key else x
        if k in seen: return ("순환", t, t - seen[k])
        seen[k] = t; x = step(x); big = max(big, abs(x))
        if len(seen) > 2 * 10 ** 5: seen.clear()          # 기억 한도(순환 판정은 그 뒤 다시 쌓는다)
    return ("미정", budget, big)


def oracle_list(step, x0, halt):
    """걸음 예산마다 (C_t, g_t, 육하원칙)."""
    out = []
    for b in BUDGETS:
        r = run(step, x0, halt, b)
        if r[0] == "멈춤":
            C = ["멈춤"]; why = f"{r[1]}걸음째에 멈춤 조건에 닿았다(실행 기록이 증인)"; how = "끝 — 증명"
```

```
        elif r[0] == "순환":
            C = ["멈추지 않음"]; why = f"{r[1]}걸음째에 이전 상태로 돌아왔다(주기 {r[2]}, 결정적 실행은 영원히 그 주기를 돈다)"; how = "끝 — 증명"
        else:
            C = ["멈춤", "멈추지 않음"]
            why = f"{b}걸음 안에 멈추지도 않았고 상태가 되풀이되지도 않았다(값의 최대 크기 {len(str(r[2]))}자리)"
            how = "걸음 예산을 늘린다, 또는 멈추지 않음의 증명서(감소하지 않는 불변식·순위 함수)를 더한다"
        out.append(dict(budget=b, C=C, guess=C[0] if len(C) == 1 else "멈추지 않음",
                        w5h=dict(누가="좁혀 가는 오라클(실행 + 상태 되풀이 검사)", 무엇을="멈추는가", 언제=f"{b}걸음까지", 어디서=f"x0 = {x0}", 왜=why, 어떻게=how)))
    return out


def cases(seed=7):
    rng = random.Random(seed); C = []
    for _ in range(300):                                  # A 유한 상태: 참답을 법 m 안에서 계산
        m = rng.randrange(50, 4000); a = rng.randrange(1, m); b = rng.randrange(0, m); h = rng.randrange(m); x0 = rng.randrange(m)
        step = (lambda a, b, m: (lambda x: (a * x + b) % m))(a, b, m); halt = (lambda h: (lambda x: x == h))(h)
        tr = run(step, x0, halt, 10 * m)[0]; C.append(dict(kind="A 유한 상태", x0=x0, step=step, halt=halt, truth="멈춤" if tr == "멈춤" else "멈추지 않음"))
    col = lambda x: x // 2 if x % 2 == 0 else 3 * x + 1
    for x0 in rng.sample(range(1, 3001), 200): C.append(dict(kind="B 콜라츠", x0=x0, step=col, halt=lambda x: x == 1, truth="멈춤"))
    dec = lambda x: x - 2
    for x0 in rng.sample(range(-50, 2 * 10 ** 5), 200): C.append(dict(kind="C 홀짝 감소", x0=x0, step=dec, halt=lambda x: x == 0, truth="멈춤" if x0 >= 0 and x0 % 2 == 0 else "멈추지 않음"))
    f5 = lambda x: x // 2 if x % 2 == 0 else 5 * x + 1
    for x0 in rng.sample(range(1, 3001), 200): C.append(dict(kind="D 5x + 1", x0=x0, step=f5, halt=lambda x: x == 1, truth=None))
    return C


if __name__ == "__main__":
    res = {}; examples = {}
    for c in cases():
        L = oracle_list(c["step"], c["x0"], c["halt"]); k = c["kind"]
        r = res.setdefault(k, dict(n=0, sound=0, monotone=0, single=[0] * len(BUDGETS), guess_ok=[0] * len(BUDGETS), truth_known=0))
        r["n"] += 1
        sizes = [len(s["C"]) for s in L]; r["monotone"] += all(sizes[i] >= sizes[i + 1] for i in range(len(sizes) - 1)) and all(set(L[i + 1]["C"]) <= set(L[i]["C"]) for i in range(len(L) - 1))
        for i, s in enumerate(L): r["single"][i] += len(s["C"]) == 1
        if c["truth"] is not None:
            r["truth_known"] += 1; r["sound"] += all(c["truth"] in s["C"] for s in L)
            for i, s in enumerate(L): r["guess_ok"][i] += s["guess"] == c["truth"]
        if k not in examples and len(L[0]["C"]) == 2: examples[k] = dict(x0=c["x0"], steps=[dict(budget=s["budget"], C=s["C"], w5h=s["w5h"]) for s in L])
    for k, r in res.items(): print(k, r, flush=True)
    json.dump(dict(budgets=BUDGETS, res=res, examples=examples), open("anytime_results.json", "w"), ensure_ascii=False, indent=1)
```

작동 확인 candidate_sets.py — 정의 25.1: 블록체인 후보 구간의 건전·최소, 예측 집합의 최소

```
"""정의 25.1의 실험: 정답 후보 집합이 (i) 참답을 포함하고 (ii) 더 줄일 수 없는가.

K1 블록체인 참값 후보 구간.  가정: 거짓 피드는 많아야 t개, 정직한 피드의 잡음은 |η| <= η0.
   자산 쌍 (u, v)의 참 비율 p_v/p_u의 후보 = ∪ (피드 t개 이하를 뺀 정직 세계가 허용하는 구간).
   정직 세계가 허용하는 구간은 곱셈 꼴 차분 제약의 최단 경로로 정확히 [1/D[v][u], D[u][v]]이다(그래프 상수).
   (i) 참 비율이 후보 안에 있는가.  (ii) 후보의 끝점·중간점마다, 관측이 같고 거짓이 t개 이하이며 참 비율이 그 값인
   세계를 실제로 만든다(그 비율을 제약으로 고정해 실현 가능성을 정확히 판정).  후보 밖의 값에는 그런 세계가 없음을 확인.
K2 ML 예측 집합(분할 등각).  문턱 순위 k = ceil((n+1)(1−α))이면 포함률 >= 1 − α, 한 칸 낮춘 k − 1이면 포함률 < 1 − α인
   분포가 있다 = 이 방식 안에서 더 줄일 수 없다.  합성 분포에서 두 문턱의 참 포함률을 잰다."""
import json, math, random, itertools
from fractions import Fraction as Fr
import numpy as np
import chain_graph as CG


def closure(n, E, obs, eta0, skip=(), fix=None):
    """비율 상한 그래프의 최소 곱 닫힘 D (정확 유리수).  skip: 뺀 피드, fix: (u, v, r) — p_v/p_u = r로 고정."""
```

```
    D = [[None] * n for _ in range(n)]
    for v in range(n): D[v][v] = Fr(1)
    def relax(a, b, w):
        if D[a][b] is None or w < D[a][b]: D[a][b] = w
    for k, (a, b) in enumerate(E):
        if k in skip: continue
        relax(a, b, obs[k] / (1 - eta0)); relax(b, a, (1 + eta0) / obs[k])
    if fix:
        u, v, r = fix; relax(u, v, r); relax(v, u, 1 / r)
    for k in range(n):
        for i in range(n):
            if D[i][k] is None: continue
            for j in range(n):
                if D[k][j] is None: continue
                w = D[i][k] * D[k][j]
                if D[i][j] is None or w < D[i][j]: D[i][j] = w
    feasible = all(D[v][v] >= 1 for v in range(n))
    return D, feasible


def candidates(n, E, obs, eta0, t, u, v):
    """후보 = 뺀 피드 집합 S (|S| <= t)마다 실현 가능한 정직 세계의 구간 [1/D[v][u], D[u][v]]의 합집합."
""
    ivs = []
    for s in range(t + 1):
        for S in itertools.combinations(range(len(E)), s):
            D, ok = closure(n, E, obs, eta0, skip=set(S))
            if not ok or D[u][v] is None or D[v][u] is None:
                if ok: ivs.append((Fr(0), None, S))                # 연결이 끊기면 후보는 양의 실수 전
체
                continue
            ivs.append((1 / D[v][u], D[u][v], S))
    return ivs


def in_cands(ivs, r): return any(lo <= r and (hi is None or r <= hi) for lo, hi, _ in ivs)


def witness_world(n, E, obs, eta0, ivs, u, v, r):
    """관측이 같고 거짓 <= t, 참 비율 = r인 세계가 있는가: 어떤 S에서 비율을 r로 고정해도 실현 가능하면
있다."""
    for lo, hi, S in ivs:
        D, ok = closure(n, E, obs, eta0, skip=set(S), fix=(u, v, r))
        if ok: return True
    return False


def exp_K1(trials=200, n=7, eta0=Fr(1, 1000), t=1, seed=31):
    rng = random.Random(seed)
    res = dict(trials=trials, pairs=0, contains=0, endpoints_realized=0, endpoints=0, mid_realized=0, mi
ds=0,
               outside_refuted=0, outsides=0, singleton_like=0, width=[], n_lie_detectable=0)
    for tr in range(trials):
        E = CG.rand_graph(rng, n, rng.randrange(3, 9)); p = [Fr(rng.randrange(50, 5000), rng.randrange(1
, 50)) for _ in range(n)]
        eta = [rng.choice([eta0, -eta0, Fr(rng.randrange(-1000, 1001), 1000) * eta0]) for _ in E]
        obs = [p[b] / p[a] * (1 + eta[k]) for k, (a, b) in enumerate(E)]
        e = rng.randrange(len(E)); obs[e] = obs[e] * Fr(rng.choice([2, 3, 4, 5, 6, 8, 9]), 7)      # 거
짓 피드 1개
        for u, v in rng.sample(list(itertools.combinations(range(n), 2)), 3):
            ivs = candidates(n, E, obs, eta0, t, u, v); truth = p[v] / p[u]
            res["pairs"] += 1; res["contains"] += in_cands(ivs, truth)
            fin = [(lo, hi) for lo, hi, _ in ivs if hi is not None]
            if fin:
                L = min(lo for lo, _ in fin); H = max(hi for _, hi in fin); res["width"].append(float(H
/ L - 1))
            for lo, hi, S in ivs:
                if hi is None: continue
                for r in (lo, hi):
                    res["endpoints"] += 1; res["endpoints_realized"] += witness_world(n, E, obs, eta0, i
vs, u, v, r)
                res["mids"] += 1; res["mid_realized"] += witness_world(n, E, obs, eta0, ivs, u, v, (lo +
 hi) / 2)
            # 후보 밖의 값: 모든 구간의 바깥(가장 큰 끝의 1.01배, 가장 작은 끝의 0.99배)
            if fin and all(hi is not None for _, hi, _ in ivs):
                for r in (H * Fr(101, 100), L * Fr(99, 100)):
                    if not in_cands(ivs, r):
                        res["outsides"] += 1; res["outside_refuted"] += not witness_world(n, E, obs, eta
0, ivs, u, v, r)
    res["width_median"] = float(np.median(res["width"])) if res["width"] else None
```

```
    return res


def exp_K2(trials=600, n_cal=200, alpha=0.1, seed=41):
    """등각 문턱의 최소성: 순위 k와 k − 1의 참 포함률(시행 평균)."""
    from sklearn.neural_network import MLPClassifier
    import ml_oracle as M
    G = M.Gen(0); rng = np.random.default_rng(seed)
    Xtr, ytr = G.sample(1500, rng); clf = MLPClassifier((32,), max_iter=400, random_state=0).fit(Xtr, yt
r)
    Xt, yt = G.sample(100000, rng); Pt = clf.predict_proba(Xt); st = 1 - Pt[np.arange(len(yt)), yt]
    k = math.ceil((n_cal + 1) * (1 - alpha)); cov_k, cov_k1, size_k, size_k1 = [], [], [], []
    for _ in range(trials):
        Xc, yc = G.sample(n_cal, rng); sc = np.sort(1 - clf.predict_proba(Xc)[np.arange(n_cal), yc])
        qk, qk1 = sc[k - 1], sc[k - 2]
        cov_k.append(float((st <= qk).mean())); cov_k1.append(float((st <= qk1).mean()))
        size_k.append(float(((1 - Pt) <= qk).sum(1).mean())); size_k1.append(float(((1 - Pt) <= qk1).sum
(1).mean()))
    return dict(n_cal=n_cal, alpha=alpha, k=k, theory_k=k / (n_cal + 1), theory_k1=(k - 1) / (n_cal + 1)
,
                cov_k=float(np.mean(cov_k)), cov_k1=float(np.mean(cov_k1)), size_k=float(np.mean(size_k)
), size_k1=float(np.mean(size_k1)),
                se=float(np.std(cov_k) / math.sqrt(trials)))


if __name__ == "__main__":
    out = dict(K1=exp_K1()); print("K1", {k: v for k, v in out["K1"].items() if k != "width"}, flush=Tru
e)
    out["K2"] = exp_K2(); print("K2", out["K2"], flush=True)
    json.dump(out, open("candidate_results.json", "w"), ensure_ascii=False, indent=1)
```

작동 확인 closure_extra.py — 26.4절 (다), 27.2절의 증인 입력, LCG의 법 2 점화식

```
"""제6부의 닫힘 보강: '증명 또는 반례'로 모든 주장을 닫기 위한 실험.

T1 진실 층의 증명 쪽: 거짓 피드가 많아야 t개이고 두 자산 사이에 변이 겹치지 않는 경로가 k >= 2t + 1개 있
으면,
   경로마다의 비율 곱 가운데 과반의 값이 참 비율이다(증명: 거짓 t개는 경로 t개만 오염시킨다).  k <= 2t이
면 C4의
   두 세계가 반례이다.  무작위 그래프에서 피드 t = 1개를 거짓으로 바꾸고, 과반 값이 참 비율과 정확히 같
은지 본다.
T2 몬테카를로 π의 구현 명세(주어진 점에서 x² + y² <= 1인 점의 수 × 4/N)로 남은 변이체를 증인 입력으로 판
정한다.
T3 삽입 정렬의 안정성(삽입 정렬은 안정 정렬로 정의된다)을 증인 입력으로 판정한다.
T4 LCG 아래 비트: a, c가 홀수면 x_{n+1} ≡ x_n + 1 (mod 2)임을 모든 시작값에서 정확히 확인(증명: 법 2의
점화식)."""
import json, random, itertools, signal


class Timeout(BaseException): pass
def _al(*a): raise Timeout()
def limited(f, sec=3):
    signal.signal(signal.SIGALRM, _al); signal.alarm(sec)
    try: return f()
    finally: signal.alarm(0)
from fractions import Fraction as Fr
import networkx as nx
import oracle_close as oc, domains as Dm, chain_graph as CG


def exp_T1(trials=80, n=9, seed=21):
    rng = random.Random(seed); res = dict(pairs=0, k_ge3=0, ok=0, k_le2=0)
    for t in range(trials):
        E = CG.rand_graph(rng, n, rng.randrange(6, 14)); p = [Fr(rng.randrange(50, 5000), rng.randrange(
1, 50)) for _ in range(n)]
        G = nx.Graph(); G.add_edges_from(E)
        obs = {e: p[e[1]] / p[e[0]] for e in E}
        bad = rng.choice(E); lam = Fr(rng.choice([2, 3, 4, 5, 6, 8, 9]), 7); obs[bad] = obs[bad] * lam
 # 거짓 피드 1개
        def rate(a, b): return obs[(a, b)] if (a, b) in obs else 1 / obs[(b, a)]
        for u, v in itertools.combinations(range(n), 2):
            paths = list(nx.edge_disjoint_paths(G, u, v))
            res["pairs"] += 1
            if len(paths) < 3: res["k_le2"] += 1; continue
            vals = []
            for P in paths:
                x = Fr(1)
                for a, b in zip(P, P[1:]): x *= rate(a, b)
                vals.append(x)
```

```
            maj = max(set(vals), key=vals.count)
            res["k_ge3"] += 1; res["ok"] += (vals.count(maj) * 2 > len(vals)) and maj == p[v] / p[u]
    return res


def exp_T2():
    """구현 명세: 출력 = 4·#{x² + y² <= 1} / N.  증인 입력 두 개: 모두 원 밖, 그리고 원 위의 점 (1, 0)."""
    rows = [json.loads(l) for l in open("domain_results.jsonl")]
    base = Dm.D["mcpi"]["src"]; out = []
    W1 = [(0.9, 0.9)] * 10                    # 모두 원 밖: 맞는 출력 0
    W2 = [(1.0, 0.0)] + [(0.9, 0.9)] * 9      # 원 위의 점 하나: 맞는 출력 4/10
    W3 = [(0.1, 0.9), (0.9, 0.1), (0.3, 0.3), (0.9, 0.9)]   # 일반 점: 맞는 출력 3
    W4 = [(0.5, 0.85), (0.85, 0.5)]           # 원 안이지만 x·y + y² > 1 인 점: 맞는 출력 4
    for r in rows:
        if r["key"] != "mcpi": continue
        fn = oc.float_version(r["src"], "mc_pi", {})
        verdict = "증명"; wit = ""
        for W, exp in ((W1, Fr(0)), (W2, Fr(4 * 1 / 10)), (W3, Fr(3)), (W4, Fr(4))):
            try: o = Fr(limited(lambda: fn(list(W))))
            except (Exception, Timeout) as e: verdict, wit = "반례", f"실행 실패 {type(e).__name__}"; break
            if o != exp: verdict, wit = "반례", f"증인 {W[:2]}…: 출력 {o} ≠ {exp}"; break
        out.append(dict(kind=r["kind"], src=r["src"], verdict=verdict, witness=wit))
    return out


def exp_T3():
    class K:
        def __init__(s, k, tag): s.k, s.tag = k, tag
        def __lt__(s, o): return s.k < o.k
        def __gt__(s, o): return s.k > o.k
        def __le__(s, o): return s.k <= o.k
        def __ge__(s, o): return s.k >= o.k
        def __repr__(s): return f"{s.k}{s.tag}"
    rows = [json.loads(l) for l in open("domain_results.jsonl")]; out = []
    W = [K(1, "a"), K(0, "b"), K(1, "c")]
    for r in rows:
        if r["key"] != "isort": continue
        fn = oc.float_version(r["src"], "insertion_sort", {})
        try: o = limited(lambda: fn(list(W))); ok = [x.tag for x in o] == ["b", "a", "c"]
        except (Exception, Timeout) as e: ok = False
        out.append(dict(kind=r["kind"], src=r["src"], stable=ok))
    return out


def exp_T4():
    fn = oc.float_version(Dm.D["lcg"]["src"], "lcg", {})
    alt = all(all((xs[i] & 1) != (xs[i - 1] & 1) for i in range(1, len(xs))) for xs in (fn(s, 50) for s in range(0, 2 ** 31, 2 ** 31 // 997)))
    return dict(parity_alternates=alt, a_odd=1103515245 % 2 == 1, c_odd=12345 % 2 == 1)


if __name__ == "__main__":
    out = dict(T1=exp_T1()); print("T1", out["T1"], flush=True)
    t2 = exp_T2(); out["T2"] = t2; from collections import Counter; print("T2", Counter(x["verdict"] for x in t2), flush=True)
    t3 = exp_T3(); out["T3"] = t3; print("T3 안정", sum(x["stable"] for x in t3), "/", len(t3), flush=True)
    out["T4"] = exp_T4(); print("T4", out["T4"])
    json.dump(out, open("closure_results.json", "w"), ensure_ascii=False, indent=1)
```

domains_v2.py — 27절: 21절 분야의 재판정

```
"""21절 재판정: 분야마다 '그 분야에서 출력이 맞다는 뜻'(분야별 오라클 정의)을 정하고, 사용자의 도구로 판정한다.

사용자 도구
  사다리   e의 나머지 사다리 eps_n = 1/(n!·n) — 허용치를 사다리의 칸에서 고른다, 테일러 꼬리로 기준값을 인증한다
  π        연분수 수렴분수로 π를 위아래에서 묶는다(333/106 < π < 355/113, 더 깊은 수렴분수는 마친 공식의 구간에서 뽑는다)
  i        증폭: 한 걸음의 증폭률 |1 - ηλ| < 1 (경사하강 = 경사 흐름의 명시적 오일러), 거듭제곱법의 비 |λ2/λ1|
  0과 1    그래프 상수 (V, T)
  부등식   반올림 엄밀성(11절): u = 2^-53, γ_k = k·u / (1 - k·u)
  큰수     큰수의 법칙·비율 편차(호에프딩): 확률 명세의 δ-판정
판정은 부동소수로 실행한 사용자의 코드 출력을, 입력을 정확한 유리수로 읽어 만든 기준값·상한과 비교한다.
정확 비교이므로 상한을 넘으면 반례이고, 넘지 않으면 '표본 통과'이다(표본 밖의 입력은 따로 정리가 맡는다)
```

```
."""
import cmath, itertools, json, math, random, signal, sys
from fractions import Fraction as F
import oracle_close as oc
import domains as Dm

U = F(1, 2 ** 53)
def gamma(k): return k * U / (1 - k * U)
def eps(n): return F(1, math.factorial(n) * n)
EPS10, EPS11, EPS12 = eps(10), eps(11), eps(12)
class Timeout(BaseException): pass


def Fx(v):
    if isinstance(v, bool): raise TypeError("bool")
    if isinstance(v, complex): raise TypeError("complex")
    return F(v)


# ------------------------------------------------------------------ π: 마친 공식 구간 -> 연분수 수렴분수
def _atan_inv(m, terms):
    s, sg = F(0), 1
    for k in range(terms):
        s += sg * F(1, (2 * k + 1) * m ** (2 * k + 1)); sg = -sg
    return s, F(1, (2 * terms + 1) * m ** (2 * terms + 1))     # 교대급수: 다음 항이 오차 상한

def pi_bracket(digits=40):
    a, ea = _atan_inv(5, 40); b, eb = _atan_inv(239, 20)
    mid = 16 * a - 4 * b; err = 16 * ea + 4 * eb
    lo, hi = mid - err, mid + err
    # 연분수 전개와 수렴분수(사용자의 π 도구): 이웃한 두 수렴분수는 π를 위아래에서 묶는다
    x, cf = mid, []
    for _ in range(40):
        q = x.numerator // x.denominator; cf.append(q); x = x - q
        if x == 0: break
        x = 1 / x
    conv = []; h0, h1, k0, k1 = 1, cf[0], 0, 1; conv.append(F(h1, k1))
    for q in cf[1:]:
        h0, h1, k0, k1 = h1, q * h1 + h0, k1, q * k1 + k0; conv.append(F(h1, k1))
    for i in range(len(conv) - 1):
        c1, c2 = conv[i], conv[i + 1]
        L, H = min(c1, c2), max(c1, c2)
        if H - L < F(1, 10 ** digits) and L <= lo and hi <= H: return L, H, cf[:i + 2]
    return lo, hi, cf
PI_LO, PI_HI, PI_CF = pi_bracket(30)
PI_MID = (PI_LO + PI_HI) / 2


import functools
@functools.lru_cache(maxsize=None)
def cos_sin_turn(q):
    """cos, sin(2π·q), q 유리수.  대칭으로 [0, 1/8] 회전으로 줄인 뒤 테일러 30항(꼬리 < 1e-40)."""
    q = q - (q.numerator // q.denominator)
    if q > F(1, 2): c, s = cos_sin_turn(1 - q); return c, -s
    if q > F(1, 4): c, s = cos_sin_turn(F(1, 2) - q); return -c, s
    if q > F(1, 8): c, s = cos_sin_turn(F(1, 4) - q); return s, c
    th = 2 * PI_MID * q; c, s, t = F(1), F(0), F(1)
    for k in range(1, 61):
        t = t * th / k
        if k % 4 == 1: s += t
        elif k % 4 == 2: c -= t
        elif k % 4 == 3: s -= t
        else: c += t
    return c.limit_denominator(10 ** 35), s.limit_denominator(10 ** 35)


def sin_interval(x):
    """참 sin x의 유리수 구간: 테일러 42항 + 꼬리 |x|^85/85! (|x| <= 4)."""
    x = F(x); t, s = x, x
    for k in range(1, 42):
        t = -t * x * x / ((2 * k) * (2 * k + 1)); s += t
    r = abs(x) ** 85 / math.factorial(85)
    return s - r, s + r


def exp_upper(r, n=40):
    """e^r 위 상한 (0 <= r <= 4): S_n(r) + r^(n+1)/(n+1)! / (1 - r/(n+2))."""
    t, s = F(1), F(1)
    for k in range(1, n + 1): t = t * r / k; s += t
    return s + r ** (n + 1) / math.factorial(n + 1) / (1 - r / (n + 2))
```

```
91
92
93  # ---------------------------------------------------------------- 입력 생성
94  def rmat(rng, n, m=None, sc=True): return [[rng.uniform(-1, 1) * (10 ** rng.randint(-2, 2) if sc else 1)
     for _ in range(m or n)] for _ in range(n)]
95  def rvec(rng, n, sc=True): return [rng.uniform(-1, 1) * (10 ** rng.randint(-2, 2) if sc else 1) for _ in
     range(n)]
96  def ddom_sym(rng, n):
97      A = [[0.0] * n for _ in range(n)]
98      for i in range(n):
99          for j in range(i + 1, n): A[i][j] = A[j][i] = rng.uniform(-1, 1)
100     for i in range(n): A[i][i] = sum(abs(A[i][j]) for j in range(n) if j != i) + rng.uniform(0.5, 2)
101     return A
102 def near_orth(rng, m, n):
103     A = [[(1.0 if i == j else 0.0) + 0.3 * rng.uniform(-1, 1) for j in range(n)] for i in range(m)]
104     return A
105
106
107 def solve_exact(A, b):
108     n = len(A); M = [[F(x) for x in row] + [F(bb)] for row, bb in zip(A, b)]
109     for c in range(n):
110         p = next(r for r in range(c, n) if M[r][c] != 0); M[c], M[p] = M[p], M[c]
111         for r in range(n):
112             if r != c and M[r][c] != 0:
113                 f = M[r][c] / M[c][c]; M[r] = [x - f * y for x, y in zip(M[r], M[c])]
114     return [M[i][n] / M[i][i] for i in range(n)]
115
116
117 def det_exact(A):
118     n = len(A); M = [[F(x) for x in row] for row in A]; d = F(1)
119     for c in range(n):
120         p = next((r for r in range(c, n) if M[r][c] != 0), None)
121         if p is None: return F(0)
122         if p != c: M[c], M[p] = M[p], M[c]; d = -d
123         d *= M[c][c]
124         for r in range(c + 1, n):
125             f = M[r][c] / M[c][c]; M[r] = [x - f * y for x, y in zip(M[r], M[c])]
126     return d
127
128
129 def ninf(v): return max((abs(x) for x in v), default=F(0))
130
131
132 # ---------------------------------------------------------------- 분야별 판정
133 DOM = {}
134
135 def dom(key, label, definition, tool, fname, gen, check, K=40, env=None, exhaustive=None):
136     DOM[key] = dict(label=label, definition=definition, tool=tool, fname=fname, gen=gen, check=check, K=
    K, env=env or {}, exhaustive=exhaustive)
137
138
139 def ck_matmul(a, out):
140     A, B = a; n = len(A); g = gamma(n)
141     if len(out) != n or any(len(r) != n for r in out): return False, "출력 모양"
142     for i in range(n):
143         for j in range(n):
144             ex = sum(Fx(A[i][k]) * Fx(B[k][j]) for k in range(n)); bd = g * sum(abs(Fx(A[i][k]) * Fx(B[k
    ][j])) for k in range(n))
145             if abs(Fx(out[i][j]) - ex) > bd: return False, f"C[{i}][{j}]: 오차가 γ_n·Σ|a||b|를 넘는다"
146     return True, ""
147 dom("matmul", "행렬곱", "출력 C^가 정확한 곱 AB에 대해 |C^ − AB| ≤ γ_n|A||B| (전진 오차 상한, Higham 정
    리 3.5)", "부등식(반올림 엄밀성)",
148     "matmul", lambda r: (lambda n: (rmat(r, n), rmat(r, n)))(r.randint(1, 5)), ck_matmul)
149
150
151 def ck_lu(a, out):
152     (A,) = a; n = len(A); L, Uu = out; g = gamma(n)
153     if len(L) != n or len(Uu) != n: return False, "출력 모양"
154     for i in range(n):
155         if Fx(L[i][i]) != 1: return False, f"L[{i}][{i}] ≠ 1"
156         for j in range(n):
157             if j > i and Fx(L[i][j]) != 0: return False, "L이 아래삼각이 아니다"
158             if j < i and Fx(Uu[i][j]) != 0: return False, "U가 위삼각이 아니다"
159     for i in range(n):
160         for j in range(n):
161             lu = sum(Fx(L[i][k]) * Fx(Uu[k][j]) for k in range(n)); bd = g * sum(abs(Fx(L[i][k]) * Fx(Uu
    [k][j])) for k in range(n))
162             if abs(Fx(A[i][j]) - lu) > bd: return False, f"(A − L^U^)[{i}][{j}]가 γ_n|L^||U^|를 넘는다(
    뒤 오차)"
163     return True, ""
```

```
dom("lu", "LU 분해(피벗 없음)", "L^U^ = A + ΔA, |ΔA| ≤ γ_n|L^||U^| 이고 L^은 단위 아래삼각, U^는 위삼각(뒤 오차 상한, Higham 정리 9.3)", "부등식(반올림 엄밀성)",
    "lu", lambda r: ((lambda n: [[x if i != j else x + sum(1 for _ in range(n)) * 2 for j, x in enumerate(row)] for i, row in enumerate(rmat(r, n, sc=False))])(r.randint(1, 5)),), ck_lu)


def ck_solve(tol):
    def ck(a, out):
        A, b = a[0], a[1]; xs = solve_exact(A, b)
        if len(out) != len(xs): return False, "출력 길이"
        if ninf([Fx(o) - x for o, x in zip(out, xs)]) > tol * max(ninf(xs), F(1, 10 ** 6)): return False, "상대 오차가 사다리 허용치를 넘는다"
        return True, ""
    return ck
dom("cg", "켤레기울기법(n번 반복)", "대칭 양정치 A에서 n번 반복한 x^가 정확한 해 x*에 대해 ‖x^ − x*‖∞ ≤ ε‖x*‖∞, 허용치 ε = ε_12 = 1/(12!·12)", "사다리(허용치)",
    "cg", lambda r: (lambda n: (ddom_sym(r, n), rvec(r, n, False)))(r.randint(1, 6)), ck_solve(EPS12))


def dft_exact(x):
    n = len(x); X = [(F(v.real), F(v.imag)) for v in x]; out = []
    for k in range(n):
        re = im = F(0)
        for j in range(n):
            c, s = cos_sin_turn(F(j * k, n)); a, b = X[j]
            re += a * c + b * s; im += b * c - a * s           # (a + ib)(c - is)
        out.append((re, im))
    return out
def ck_fft(a, out):
    (x,) = a; n = len(x)
    if len(out) != n: return False, "출력 길이"
    m = max(1, int(math.log2(n))) if n > 1 else 0
    nx = F(sum(abs(v) ** 2 for v in x)).limit_denominator(10 ** 30) ** 1  # ‖x‖² (근사로 충분: 아래 상한에 여유 1.01)
    bd = F(10 * m) * U * F(math.sqrt(n)) * F(math.sqrt(float(nx))) * F(101, 100) + F(1, 10 ** 25)
    ref = dft_exact(x)
    for k, (re, im) in enumerate(ref):
        o = out[k]
        if not isinstance(o, (complex, float, int)): return False, "출력 형식"
        o = complex(o); d = abs(complex(float(F(o.real) - re), float(F(o.imag) - im)))
        if F(d) > bd: return False, f"y[{k}] 오차 {d:.2e}가 10·log2n·u·√n·‖x‖_2를 넘는다"
    return True, ""
dom("fft", "FFT(재귀 2진)", "출력 y^가 DFT y에 대해 |y^_k − y_k| ≤ 10·log2n·u·√n·‖x‖_2 (Higham 정리 24.2의 꼴). 기준 DFT의 회전인자는 π의 연분수 수렴분수 구간과 테일러 꼬리로 인증", "π(연분수) + 사다리(테일러 꼬리) + 부등식",
    "fft", lambda r: ([complex(r.uniform(-1, 1), r.uniform(-1, 1)) for _ in range(2 ** r.randint(0, 5))],), ck_fft, K=25, env=dict(cmath=cmath))


def ck_conv(a, out):
    A, B = a; n = len(A); g = gamma(n)
    if len(out) != 2 * n - 1: return False, "출력 길이"
    for k in range(2 * n - 1):
        prs = [Fx(A[i]) * Fx(B[k - i]) for i in range(n) if 0 <= k - i < n]
        if abs(Fx(out[k]) - sum(prs)) > g * sum(abs(p) for p in prs): return False, f"c[{k}]: 오차가 γ_n·Σ|a||b|를 넘는다"
    return True, ""
dom("conv", "직접 합성곱", "|c^_k − c_k| ≤ γ_n Σ_(i+j=k)|a_i||b_j| (내적의 전진 오차 상한)", "부등식(반올림 엄밀성)",
    "conv", lambda r: (lambda n: (rvec(r, n), rvec(r, n)))(r.randint(1, 6)), ck_conv)


def gen_poly(r):
    la = r.randint(1, 6); lb = r.randint(1, la)
    a = [F(r.randint(-9, 9), r.randint(1, 4)) for _ in range(la)]; b = [F(r.randint(-9, 9), r.randint(1, 4)) for _ in range(lb)]
    if b[-1] == 0: b[-1] = F(1)
    return (a, b)
def ck_poly(a, out):
    A, B = a; q, rem = out
    if len(q) != len(A) - len(B) + 1 or len(rem) != len(B) - 1: return False, "몫·나머지의 차수"
    prod = [F(0)] * len(A)
    for i, qi in enumerate(q):
        for j, bj in enumerate(B): prod[i + j] += F(qi) * bj
    for i, ri in enumerate(rem): prod[i] += F(ri)
    if prod != list(A): return False, "a ≠ q·b + r"
    return True, ""
dom("polydiv", "다항식 나눗셈", "정확한 유리수 계수에서 a = q·b + r, deg r < deg b (곱해서 되돌리는 검산식)", "필요 없음(분야의 검산식이 완전한 오라클)",
    "poly_divmod", gen_poly, ck_poly)
```

```


def gen_bareiss(r):
    while True:
        n = r.randint(1, 5); A = [[F(r.randint(-9, 9)) for _ in range(n)] for _ in range(n)]
        if all(det_exact([row[:k] for row in A[:k]]) != 0 for k in range(1, n)): return (A,)
def ck_bareiss(a, out):
    (A,) = a
    try: o = F(out)
    except Exception: return False, "출력 형식"
    return (o == det_exact(A), "det가 정확한 값과 다르다")
dom("bareiss", "행렬식(바라이스)", "정수 행렬에서 출력 = 정확한 행렬식 (선행 소행렬식 ≠ 0인 입력 계약)",
 "필요 없음(정확한 행렬식이 완전한 오라클)",
    "det_bareiss", gen_bareiss, ck_bareiss)


def gen_lsq(r):
    n = r.randint(1, 4); m = r.randint(n, n + 3); return (near_orth(r, m, n), rvec(r, m, False))
def ck_lsq(a, out):
    A, b = a; n = len(A[0]); m = len(A)
    G = [[sum(F(A[k][i]) * F(A[k][j]) for k in range(m)) for j in range(n)] for i in range(n)]
    c = [sum(F(A[k][i]) * F(b[k]) for k in range(m)) for i in range(n)]
    xs = solve_exact(G, c)
    if len(out) != n: return False, "출력 길이"
    if ninf([Fx(o) - x for o, x in zip(out, xs)]) > EPS12 * max(ninf(xs), F(1)): return False, "최소제곱
해와의 상대 오차가 사다리 허용치를 넘는다"
    return True, ""
dom("lstsq", "최소제곱(정규방정식)", "x^가 최소제곱해 x* = argmin‖Ax − b‖에 대해 ‖x^ − x*‖∞ ≤ ε_12·max(‖
x*‖∞, 1) (조건수가 작은 입력 계약)", "사다리(허용치)",
    "lstsq", gen_lsq, ck_lsq)


def gen_gd(r):
    n = r.randint(1, 3); m = r.randint(n, n + 2); A = near_orth(r, m, n); b = rvec(r, m, False)
    G = [[sum(F(A[k][i]) * F(A[k][j]) for k in range(m)) for j in range(n)] for i in range(n)]
    lam_hi = max(G[i][i] + sum(abs(G[i][j]) for j in range(n) if j != i) for i in range(n))
    return (A, b, 200, float(1 / lam_hi))
def ck_gd(a, out):
    A, b, steps, lr = a; n = len(A[0]); m = len(A)
    G = [[sum(F(A[k][i]) * F(A[k][j]) for k in range(m)) for j in range(n)] for i in range(n)]
    lo = min(G[i][i] - sum(abs(G[i][j]) for j in range(n) if j != i) for i in range(n))
    hi = max(G[i][i] + sum(abs(G[i][j]) for j in range(n) if j != i) for i in range(n))
    eta = F(lr); q = max(abs(1 - eta * lo), abs(1 - eta * hi))         # 증폭률(i): 경사 흐름의 명시적
오일러 한 걸음
    if not (lo > 0 and q < 1): return True, "입력 계약 밖"
    c = [sum(F(A[k][i]) * F(b[k]) for k in range(m)) for i in range(n)]; xs = solve_exact(G, c)
    if len(out) != n: return False, "출력 길이"
    bound = q ** steps * F(math.isqrt(n) + 1) * ninf(xs) + F(1, 10 ** 12) * (1 + ninf(xs))
    if ninf([Fx(o) - x for o, x in zip(out, xs)]) > bound: return False, "‖x^ − x*‖가 증폭률의 거듭제곱
상한 q^steps·‖x0 − x*‖를 넘는다"
    return True, ""
dom("gd", "경사하강(고정 반복)", "f = ½‖Ax − b‖²에서 steps번 뒤 ‖x^ − x*‖ ≤ q^steps·‖x0 − x*‖ + 반올림,
q = max|1 − ηλ| (λ는 A^TA의 게르시고린 구간)", "i(증폭률) + 부등식",
    "gd_lstsq", gen_gd, ck_gd)


POWER_TRUTH = {}
def gen_power(r):
    """A = P D P^{-1}, P는 단위 아래삼각 정수 행렬(역도 정수), D는 8분의 정수 — A의 원소가 부동소수로 정
확하다.
    지배 고유쌍은 (λ1, P e1) 이고 P e1의 첫 성분은 1이다(코드의 정규화와 같은 꼴)."""
    n = r.randint(2, 4)
    lam1 = F(r.randint(16, 48), 8) * r.choice([1, -1]); rest = [F(r.randint(-8, 8), 8) for _ in range(n
- 1)]   # |λ_i / λ1| <= 1/2
    D = [lam1] + rest
    P = [[F(1) if i == j else (F(r.randint(-1, 1)) if j < i else F(0)) for j in range(n)] for i in range
(n)]
    Pi = [[F(0)] * n for _ in range(n)]
    for j in range(n):                                  # 단위 아래삼각의 역: 앞으로 대입
        x = [F(0)] * n
        for i in range(n): x[i] = (1 if i == j else 0) - sum(P[i][k] * x[k] for k in range(i))
        for i in range(n): Pi[i][j] = x[i]
    A = [[sum(P[i][k] * D[k] * Pi[k][j] for k in range(n)) for j in range(n)] for i in range(n)]
    Af = [[float(v) for v in row] for row in A]
    assert all(F(Af[i][j]) == A[i][j] for i in range(n) for j in range(n))
    POWER_TRUTH[str(Af)] = (lam1, [P[i][0] for i in range(n)])
    return (Af, 100)
def ck_power(a, out):
    A, steps = a; n = len(A)
    lam, x = out
```

```
    xf = [Fx(v) for v in x]
    if len(xf) != n: return False, "출력 길이"
    lam1, v = POWER_TRUTH[str(A)]
    if abs(Fx(lam) - lam1) > EPS12 * abs(lam1): return False, "λ^가 지배 고유값과 사다리 허용치 밖이다"
    if ninf([p - q for p, q in zip(xf, v)]) > EPS10: return False, "x^가 지배 고유벡터(첫 성분 1로 정규화)와 다르다"
    return True, ""
dom("power", "거듭제곱법(고정 반복)", "|λ^ − λ1| ≤ ε_12|λ1|, x^ ≈ 지배 고유벡터(ε_10). 입력 계약: |λ2/λ1| ≤ 1/2라 100번이면 증폭비의 거듭제곱 (1/2)^100이 허용치보다 작고, 시작 벡터가 지배 성분을 가진다", "i(증폭비) + 사다리(허용치)",
    "power_iter", gen_power, ck_power)


def gen_seg(r):
    if r.random() < 0.5:
        return tuple((r.uniform(-1, 1), r.uniform(-1, 1)) for _ in range(4))
    p3 = (r.uniform(-1, 1), r.uniform(-1, 1)); p4 = (r.uniform(-1, 1), r.uniform(-1, 1)); t = r.uniform(0.1, 0.9)
    p1 = (p3[0] + t * (p4[0] - p3[0]), p3[1] + t * (p4[1] - p3[1]))       # 거의 한 직선 위(반올림 수준)
    p2 = (r.uniform(-1, 1), r.uniform(-1, 1))
    return (p1, p2, p3, p4)
def seg_exact(p1, p2, p3, p4):
    P = [tuple(F(c) for c in p) for p in (p1, p2, p3, p4)]; p1, p2, p3, p4 = P
    d1 = (p4[0] - p3[0]) * (p1[1] - p3[1]) - (p4[1] - p3[1]) * (p1[0] - p3[0])
    d2 = (p4[0] - p3[0]) * (p2[1] - p3[1]) - (p4[1] - p3[1]) * (p2[0] - p3[0])
    d3 = (p2[0] - p1[0]) * (p3[1] - p1[1]) - (p2[1] - p1[1]) * (p3[0] - p1[0])
    d4 = (p2[0] - p1[0]) * (p4[1] - p1[1]) - (p2[1] - p1[1]) * (p4[0] - p1[0])
    return ((d1 > 0) != (d2 > 0)) and ((d3 > 0) != (d4 > 0))
def ck_seg(a, out):
    return (out == seg_exact(*a), "부호 판정이 정확 산술의 판정과 다르다")
SEG_FILTERED = '''
def orient(a, b, c):
    d = (b[0] - a[0]) * (c[1] - a[1]) - (b[1] - a[1]) * (c[0] - a[0])
    m = abs((b[0] - a[0]) * (c[1] - a[1])) + abs((b[1] - a[1]) * (c[0] - a[0]))
    if abs(d) > 3.4e-16 * m:          # 반올림 부등식: |d^ − d| <= (3u + 16u²)·m 이면 부호가 확정
        return d > 0
    A = [Fraction(v) for v in a]; B = [Fraction(v) for v in b]; C = [Fraction(v) for v in c]
    return (B[0] - A[0]) * (C[1] - A[1]) - (B[1] - A[1]) * (C[0] - A[0]) > 0

def intersects(p1, p2, p3, p4):
    return (orient(p3, p4, p1) != orient(p3, p4, p2)) and (orient(p1, p2, p3) != orient(p1, p2, p4))
'''
dom("segint", "선분 교차(방향 부호)", "입력 부동소수 좌표를 정확한 유리수로 읽었을 때의 판정과 같아야 한다(견고한 기하 술어)", "부등식(반올림 필터)",
    "intersects", gen_seg, ck_seg, K=400)


def ex_isort(fn):
    for n in range(0, 7):
        for w in itertools.product(range(n), repeat=n):
            try: o = fn(list(w))
            except Exception as e: return False, f"{list(w)}: 실행 실패 {type(e).__name__}"
            if o != sorted(w): return False, f"{list(w)}: 정렬 결과가 아니다"
    return True, "n ≤ 6 모든 약순서"
dom("isort", "삽입 정렬", "출력이 입력의 순열이고 오름차순 (안정성은 주장한 경우에만)", "필요 없음(정렬 검산식이 완전한 오라클)", "insertion_sort", None, None, exhaustive=ex_isort)


def ex_bsearch(fn):
    for n in range(0, 8):
        for a in itertools.combinations_with_replacement(range(n), n):
            a = list(a)
            for x in range(-1, n + 1):
                try: o = fn(list(a), x)
                except Exception as e: return False, f"{a}, {x}: 실행 실패 {type(e).__name__}"
                if o != next((i for i, t in enumerate(a) if t >= x), n): return False, f"{a}, {x}: 왼쪽 끝 위치가 아니다"
    return True, "n ≤ 7 모든 약순서와 모든 틈"
dom("bsearch", "이분 탐색(왼쪽 끝)", "출력 = a[i] ≥ x인 첫 i (정렬된 입력 계약)", "필요 없음(정의가 완전한 오라클)", "lower_bound", None, None, exhaustive=ex_bsearch)


def gen_sin(r):
    lim = float(F(333, 106))                                  # π의 아래 수렴분수: |x| <= 333/106 < π
    return (r.uniform(-lim, lim) if r.random() < 0.9 else r.choice([0.0, 1e-300, lim, -lim, 1.0]),)
def ck_sin(a, out):
    (x,) = a; lo, hi = sin_interval(x); ax = abs(F(x))
    tail = ax ** 17 / math.factorial(17)
    sinh_up = (exp_upper(ax) - 1 / exp_upper(ax) if ax > 0 else F(0)) / 2 + ax   # Σ|항| <= sinh|x| (여유)
```

```
    bd = tail + gamma(30) * sinh_up
    o = Fx(out)
    if o < lo - bd or o > hi + bd: return False, "참 sin과의 오차가 테일러 꼬리 |x|^17/17! + 반올림 상한을 넘는다"
    return True, ""
dom("sin", "sin 근사(8항 테일러)", "|s^ − sin x| ≤ |x|^17/17! + γ_30·sinh|x|, |x| ≤ 333/106 (참 sin은 테일러 42항과 꼬리로 인증한 구간)", "사다리(테일러 꼬리) + π(연분수 범위) + 부등식",
    "sin_taylor", gen_sin, ck_sin, K=200)


def gen_kahan(r):
    n = r.randint(2, 30); xs = []
    while len(xs) < n:
        k = r.random()
        if k < 0.4: big = r.uniform(1, 10) * 10 ** r.randint(8, 16); xs += [big, -big * (1 + r.uniform(-1e-12, 1e-12))]
        elif k < 0.7: xs.append(r.uniform(-1, 1) * 10 ** r.randint(-8, 8))
        else: xs.append(r.choice([1.0, -1.0, 0.1, 1e16, -1e16, 3.0]))
    r.shuffle(xs); return (xs[:n],)
def ck_kahan(a, out):
    (xs,) = a; n = len(xs); s = sum(F(x) for x in xs); S1 = sum(abs(F(x)) for x in xs)
    bd = (2 * U + 4 * n * U * U) * S1
    if abs(Fx(out) - s) > bd: return False, "오차가 보정 합의 상한 (2u + 4nu²)Σ|x|를 넘는다"
    return True, ""
dom("kahan", "카한 합", "|s^ − Σx| ≤ (2u + 4nu²)Σ|x_i| (보정 합의 오차 상한, Goldberg 정리 8·Higham 식 4.8의 꼴) — 반올림 자체가 명세", "부등식(반올림 엄밀성)",
    "kahan_sum", gen_kahan, ck_kahan, K=300)


def gen_lcg(r): return (r.randrange(2 ** 31), r.randint(0, 60))
def ck_lcg(a, out):
    seed, n = a; x = seed; ref = []
    for _ in range(n): x = (1103515245 * x + 12345) % 2147483648; ref.append(x)
    return (out == ref, "점화식과 다르다")
dom("lcg", "선형 합동 난수열", "구현: 출력 = 점화식 x ← (1103515245x + 12345) mod 2^31. 난수성: 비트 빈도·연속 비트의 비율이 1/2에서 호에프딩 허용치 안", "큰수(비율 편차) — 난수성 쪽",
    "lcg", gen_lcg, ck_lcg)


def gen_mc(r): return ([(r.random(), r.random()) for _ in range(20000)],)
MC_T = math.sqrt(math.log(2 / 1e-6) / (2 * 20000))                # 호에프딩: δ = 1e-6
def ck_mc(a, out):
    try: o = Fx(out)
    except Exception: return False, "출력 형식"
    eps_ = F(4 * MC_T) * F(1001, 1000)
    if o < PI_LO - eps_ or o > PI_HI + eps_: return False, f"|π^ − π| > 4t (호에프딩 δ = 1e-6, N = 20000)"
    return True, ""
dom("mcpi", "몬테카를로 π", "확률 명세: P(|π^ − π| > 4t) ≤ 2e^(−2Nt²) = δ. π는 연분수 수렴분수 구간. 20번 독립 실행 중 한 번이라도 넘으면 δ-반례(맞는 코드가 그럴 확률 ≤ 20δ)", "큰수(비율 편차) + π(연분수)",
    "mc_pi", gen_mc, ck_mc, K=20)


# ---------------------------------------------------------------- 판정기
def judge(key, src):
    d = DOM[key]
    try: fn = oc.float_version(src, d["fname"], dict(d["env"], Fraction=F))
    except Exception as e: return dict(verdict="반례", why=f"컴파일 실패 {type(e).__name__}")
    if d["exhaustive"]:
        ok, why = d["exhaustive"](fn); return dict(verdict="유한 증명" if ok else "반례", why=why)
    rng = random.Random(12345)
    for t in range(d["K"]):
        a = d["gen"](rng)
        try:
            import copy
            out = fn(*copy.deepcopy(a))
        except Exception as e: return dict(verdict="반례", why=f"실행 실패 {type(e).__name__}", trial=t)
        try: ok, why = d["check"](a, out)
        except Exception as e: ok, why = False, f"출력 검사 실패 {type(e).__name__}"
        if not ok: return dict(verdict="반례", why=why, trial=t)
    return dict(verdict="표본 통과", why=f"입력 {d['K']}개")


def lcg_stats(src, N=20000, delta=1e-6):
    """난수성 오라클(큰수·비율 편차): 맨 위 비트의 빈도, 맨 아래 비트가 바로 앞과 같은 비율 — 둘 다 1/2에서 t 안이어야."""
    fn = oc.float_version(src, "lcg", {}); xs = fn(20240607, N)
    t = math.sqrt(math.log(2 / delta) / (2 * N))
    top = sum((x >> 30) & 1 for x in xs) / N
    low_rep = sum(((xs[i] & 1) == (xs[i - 1] & 1)) for i in range(1, N)) / (N - 1)
```

```
    return dict(t=t, top=top, top_ok=abs(top - 0.5) <= t, low_rep=low_rep, low_ok=abs(low_rep - 0.5) <= t)


def run(keys=None):
    rows = [json.loads(l) for l in open("domain_results.jsonl")]
    keymap = {"sin": "sin_poly"}
    out = {}
    for key in (keys or DOM):
        oldkey = keymap.get(key, key)
        R = [r for r in rows if r["key"] == oldkey]
        res = []
        for r in R:
            def al(*a): raise Timeout()
            signal.signal(signal.SIGALRM, al); signal.alarm(30 if key in ("isort", "bsearch", "fft") else 10)
            try: j = judge(key, r["src"])
            except Timeout: j = dict(verdict="반례", why="끝나지 않는 실행(시간 초과)")
            finally: signal.alarm(0)
            res.append(dict(kind=r["kind"], where=r.get("where", ""), old=r["B2"], new=j["verdict"], why=j.get("why", ""), src=r["src"]))
        orig = next(x for x in res if x["kind"] == "원본")
        out[key] = dict(rows=res, orig=orig["new"], orig_why=orig["why"])
        from collections import Counter
        print(key, "원본", orig["new"], orig["why"][:60], Counter((x["old"], x["new"]) for x in res if x["kind"] != "원본"), flush=True)
    return out


if __name__ == "__main__":
    print("π 구간", float(PI_LO), float(PI_HI - PI_LO), "연분수", PI_CF)
    out = run(sys.argv[1:] or None)
    if not sys.argv[1:]:
        out["_lcg_stats"] = lcg_stats(Dm.D["lcg"]["src"])
        # 선분 교차: 반올림 필터를 넣은 판(사용자 도구의 부등식)이 정확 판정과 같은가
        out["_seg_filtered"] = judge("segint", SEG_FILTERED)
        out["_pi"] = dict(lo=str(PI_LO), hi=str(PI_HI), cf=PI_CF)
        json.dump(out, open("domains_v2_results.json", "w"), ensure_ascii=False, indent=1)


def mc_resolve(src, N=10 ** 6, trials=5, seed=77):
    """몬테카를로 π의 해상도 올리기: N을 키우면 허용치 4t = 4√(ln(2/δ)/2N)가 줄어 작은 치우침도 걸린다."""
    import numpy as np
    fn = oc.float_version(src, "mc_pi", {}); t = math.sqrt(math.log(2 / 1e-6) / (2 * N)); e = F(4 * t) * F(1001, 1000)
    rng = np.random.default_rng(seed); vals = []
    for _ in range(trials):
        pts = rng.random((N, 2)).tolist(); o = Fx(fn(pts)); vals.append(float(o))
        if o < PI_LO - e or o > PI_HI + e: return dict(verdict="반례", N=N, eps=4 * t, vals=vals)
    return dict(verdict="표본 통과", N=N, eps=4 * t, vals=vals)


def mc_followup():
    out = json.load(open("domains_v2_results.json")); res = []
    for x in out["mcpi"]["rows"]:
        if x["kind"] != "원본" and x["new"] != "반례":
            r = mc_resolve(x["src"]); res.append(dict(kind=x["kind"], where=x["where"], src=x["src"], **r)); print(r["verdict"], r["vals"][:2], flush=True)
    r0 = mc_resolve(Dm.D["mcpi"]["src"]); print("원본", r0)
    out["_mc_followup"] = dict(rows=res, orig=r0); json.dump(out, open("domains_v2_results.json", "w"), ensure_ascii=False, indent=1)
```

ml_oracle.py — 28절: ML 오라클(완전 판정 포함)

```
"""ML 오라클 문제: 맞는 예측의 명세(정답 함수)가 없을 때 무엇을 판정할 수 있는가.

M0 경계(구별 불가능): 입력·학습 자료·모델이 같고 시험 입력의 정답만 다른 두 세계를 만든다.  정답을 보지 않는
   판정기는 두 세계에서 같은 답을 내므로, '이 예측 하나가 맞는가'를 확정하는 판정기는 한쪽에서 반드시 틀린다.
   그래서 해결의 정의를 바꾼다: 예측 하나의 참거짓 대신 '받아들인 예측의 오류율이 r* 이하'를 1 − δ로 보증한다.
M1 정확도 보증(비율 편차): 보정 집합 n개의 정답률 â에서 호에프딩 구간 [â − t, â + t], t = √(ln(2/δ)/2n).
M2 선택적 판정(비율 편차 + 합집합 상한): 확신도 문턱 θ마다 받아들인 예측의 오류율 상한을 비율 편차의 체르노프(KL) 꼴로 잡고(δ/m),
   상한이 r* 이하인 가장 낮은 θ를 고른다.  보증: P(받아들인 예측의 참 오류율 > r*) ≤ δ.
M3 등각 예측 집합(분할 등각, 순위 논증): P(정답 ∈ 집합) ≥ 1 − α.
M4 강건성 증명(부등식): 선형 소프트맥스는 반지름 r(x) = min_j (z_c − z_j)/‖w_c − w_j‖_2 안에서 예측이 바
```

```
    뀌지 않는다(정확).
11     ReLU 망은 립시츠 상한 L = Π‖W_l‖_2로 r(x) = (z_c − max_j z_j)/(√2·L) (건전, 느슨).  공격으로 반례를
    찾아본다.
12  M5 분포 이동 검출(DKW): 보정 집합과 운영 입력의 확신도 분포 차이 sup|F1 − F2|가 DKW 허용치를 넘으면 경보
    .
13     이동이 없을 때 거짓 경보율 ≤ δ, 이동이 있을 때 검출률, 그리고 이동이 있으면 M2·M3 보증이 실제로 깨지
    는지.
14  M6 파이프라인 버그: 운영 추론 코드에 버그를 넣고, 정답 없는 판정기(M5)와 정답 있는 판정기(M1)가 잡는지.
15  참값이 필요한 검증은 합성 분포(무한히 뽑을 수 있음)에서 20만 개 표본으로 한다.  실제 자료(손글씨 숫자)는
     적용 예이다."""
16  import json, math, warnings
17  import numpy as np
18  from sklearn.neural_network import MLPClassifier
19  from sklearn.linear_model import LogisticRegression
20  from sklearn.datasets import load_digits
21  warnings.filterwarnings("ignore")
22
23  DELTA = 0.05
24
25
26  # ---------------------------------------------------------------- 합성 분포(참값을 잴 수 있다)
27  class Gen:
28      def __init__(s, seed=0, d=8, K=4):
29          r = np.random.default_rng(seed); s.d, s.K = d, K
30          s.mu = r.normal(0, 1.3, (K, d)); s.A = [np.eye(d) + 0.3 * r.normal(0, 1, (d, d)) for _ in range(
    K)]
31      def sample(s, n, rng, shift=None):
32          y = rng.integers(0, s.K, n) if shift != "prior" else rng.choice(s.K, n, p=[0.7] + [0.3 / (s.K -
    1)] * (s.K - 1))
33          z = rng.normal(0, 1, (n, s.d)); X = np.stack([s.mu[k] + s.A[k] @ zz for k, zz in zip(y, z)])
34          if shift == "noise": X = X + rng.normal(0, 1.5, X.shape)
35          if shift == "scale": X = X * 1.8
36          return X, y
37
38
39  def softmax(Z): Z = Z - Z.max(1, keepdims=True); E = np.exp(Z); return E / E.sum(1, keepdims=True)
40  def hoeff(n, delta): return math.sqrt(math.log(2 / delta) / (2 * n)) if n > 0 else 1.0
41  def hoeff1(n, delta): return math.sqrt(math.log(1 / delta) / (2 * n)) if n > 0 else 1.0       # 한쪽
42
43
44  def kl(a, b):
45      a = min(max(a, 1e-15), 1 - 1e-15); b = min(max(b, 1e-15), 1 - 1e-15)
46      return a * math.log(a / b) + (1 - a) * math.log((1 - a) / (1 - b))
47
48
49  def kl_ub(k, n, delta):
50      """비율 편차의 체르노프(KL) 꼴: P(p^ <= k/n | p) <= exp(-n·KL(k/n‖p)).  그 값이 δ가 되는 가장 큰 p (
    이분법)."""
51      ph = k / n; lo, hi = ph, 1.0
52      for _ in range(60):
53          mid = (lo + hi) / 2
54          if n * kl(ph, mid) > math.log(1 / delta): hi = mid
55          else: lo = mid
56      return hi
57
58
59  def sgr(conf, correct, rstar, delta, m=25):
60      """선택적 판정: 문턱 후보 m개(확신도 분위수), 각 문턱에서 오류율 위 상한(호에프딩, δ/m).  가장 많이
    받아들이는 문턱."""
61      qs = np.quantile(conf, np.linspace(0, 0.96, m)); best = None
62      for th in sorted(qs):
63          acc = conf >= th; n = int(acc.sum())
64          if n == 0: continue
65          ub = kl_ub(int((~correct[acc]).sum()), n, delta / m)
66          if ub <= rstar: best = th; break
67      return best
68
69
70  def conformal_q(P_cal, y_cal, alpha):
71      s = 1 - P_cal[np.arange(len(y_cal)), y_cal]; n = len(s)
72      k = math.ceil((n + 1) * (1 - alpha))
73      return np.inf if k > n else np.sort(s)[k - 1]
74
75
76  def dkw_eps(n1, n2, delta):
77      return math.sqrt(math.log(2 / (delta / 2)) / (2 * n1)) + math.sqrt(math.log(2 / (delta / 2)) / (2 *
    n2))
78  def ks(a, b):
79      xs = np.sort(np.concatenate([a, b]))
80      Fa = np.searchsorted(np.sort(a), xs, side="right") / len(a); Fb = np.searchsorted(np.sort(b), xs, si
```

```
    de="right") / len(b)
    return float(np.max(np.abs(Fa - Fb)))


def exp_guarantees(trials=100, seed=1):
    G = Gen(0); out = dict(trials=trials, M1_cover=0, M2_viol=0, M2_none=0, M2_cov=[], M2_risk=[], M3_co
ver=[], M3_size=[], M5_fa=0)
    for t in range(trials):
        rng = np.random.default_rng(seed * 1000 + t)
        Xtr, ytr = G.sample(1500, rng); Xc, yc = G.sample(1000, rng); Xt, yt = G.sample(200000, rng)
        clf = MLPClassifier((32,), max_iter=400, random_state=t).fit(Xtr, ytr)
        Pc, Pt = clf.predict_proba(Xc), clf.predict_proba(Xt)
        cc = Pc.argmax(1) == yc; ct = Pt.argmax(1) == yt
        # M1
        a = cc.mean(); tt = hoeff(len(yc), DELTA); out["M1_cover"] += (a - tt <= ct.mean() <= a + tt)
        # M2: r* = 0.05
        th = sgr(Pc.max(1), cc, 0.05, DELTA)
        if th is None: out["M2_none"] += 1
        else:
            acc = Pt.max(1) >= th; risk = 1 - ct[acc].mean(); out["M2_viol"] += risk > 0.05
            out["M2_cov"].append(float(acc.mean())); out["M2_risk"].append(float(risk))
        # M3: α = 0.1
        q = conformal_q(Pc, yc, 0.1); S = (1 - Pt) <= q
        out["M3_cover"].append(float(S[np.arange(len(yt)), yt].mean())); out["M3_size"].append(float(S.s
um(1).mean()))
        # M5 거짓 경보: 이동 없음, 운영 입력 2000개
        Xo, _ = G.sample(2000, rng); out["M5_fa"] += ks(Pc.max(1), clf.predict_proba(Xo).max(1)) > dkw_e
ps(len(yc), 2000, DELTA)
        out.setdefault("acc_true", []).append(float(ct.mean()))
    return out


def exp_shift(trials=40, seed=2):
    """이동이 있으면: M5가 검출하는가, 그리고 M2·M3 보증이 정말 깨지는가."""
    G = Gen(0); res = {}
    for sh in ("noise", "scale", "prior"):
        r = dict(detect=0, M2_viol=0, M2_n=0, M3_cover=[])
        for t in range(trials):
            rng = np.random.default_rng(seed * 1000 + t)
            Xtr, ytr = G.sample(1500, rng); Xc, yc = G.sample(1000, rng)
            clf = MLPClassifier((32,), max_iter=400, random_state=t).fit(Xtr, ytr)
            Pc = clf.predict_proba(Xc); cc = Pc.argmax(1) == yc
            Xo, yo = G.sample(2000, rng, sh); Po = clf.predict_proba(Xo)
            r["detect"] += ks(Pc.max(1), Po.max(1)) > dkw_eps(1000, 2000, DELTA)
            Xt, yt = G.sample(100000, rng, sh); Pt = clf.predict_proba(Xt); ct = Pt.argmax(1) == yt
            th = sgr(Pc.max(1), cc, 0.05, DELTA)
            if th is not None:
                acc = Pt.max(1) >= th; r["M2_n"] += 1; r["M2_viol"] += (1 - ct[acc].mean()) > 0.05
            q = conformal_q(Pc, yc, 0.1); S = (1 - Pt) <= q; r["M3_cover"].append(float(S[np.arange(len(
yt)), yt].mean()))
        r["trials"] = trials; res[sh] = r
    return res


def exp_indist(seed=3):
    """M0: 같은 입력·같은 모델, 시험 정답만 다른 두 세계.  정답을 보지 않는 판정기의 출력은 두 세계에서
같다."""
    G = Gen(0); rng = np.random.default_rng(seed)
    Xtr, ytr = G.sample(1500, rng); Xc, yc = G.sample(1000, rng); Xt, yt = G.sample(500, rng)
    clf = MLPClassifier((32,), max_iter=400, random_state=0).fit(Xtr, ytr)
    Pc, Pt = clf.predict_proba(Xc), clf.predict_proba(Xt); cc = Pc.argmax(1) == yc
    th = sgr(Pc.max(1), cc, 0.02, DELTA); acc = Pt.max(1) >= th
    yB = yt.copy(); pred = Pt.argmax(1); flip = np.where(acc)[0][:50]
    yB[flip] = (pred[flip] + 1) % G.K                       # 세계 B: 받아들인 예측 50개의 정답이 다른
값
    def oracle(X, P): return (P.max(1) >= th)               # 정답을 보지 않는 판정기: 입력·모델·보정
자료만 쓴다
    same = bool(np.array_equal(oracle(Xt, Pt), oracle(Xt, Pt)))
    wrongA = int((acc & (pred != yt)).sum()); wrongB = int((acc & (pred != yB)).sum())
    return dict(accepted=int(acc.sum()), same_output=same, flipped=len(flip), errors_A=wrongA, errors_B=
wrongB,
                note="받아들인 50개에 대해 판정기의 답은 두 세계에서 같지만 정답은 반대: 어느 한 세계에
서는 50개 모두 틀린 판정")


def exp_robust(seed=4):
    """M4: 선형(정확 반지름)과 ReLU 망(립시츠 반지름).  반지름 안 공격으로 예측을 바꿀 수 있는지."""
    G = Gen(0); rng = np.random.default_rng(seed)
    Xtr, ytr = G.sample(3000, rng); Xt, yt = G.sample(300, rng)
    lin = LogisticRegression(max_iter=2000).fit(Xtr, ytr); W, b = lin.coef_, lin.intercept_
```

```
    res = dict(linear=dict(n=0, flip_inside=0, flip_outside=0), mlp=dict(n=0, flip_inside=0, radius_mean=0.0, emp_mean=0.0))
    for x in Xt:
        z = W @ x + b; c = int(z.argmax()); js = [j for j in range(len(z)) if j != c]
        r = [(z[c] - z[j]) / np.linalg.norm(W[c] - W[j]) for j in js]; k = int(np.argmin(r)); j = js[k]; rad = r[k]
        d = (W[j] - W[c]) / np.linalg.norm(W[j] - W[c])
        res["linear"]["n"] += 1
        res["linear"]["flip_inside"] += int((W @ (x + 0.999 * rad * d) + b).argmax() != c)       # 정리: 0이어야
        res["linear"]["flip_outside"] += int((W @ (x + 1.001 * rad * d) + b).argmax() != c)      # 상한이 빡빡하면 바뀐다
    mlp = MLPClassifier((32, 32), max_iter=600, random_state=0).fit(Xtr, ytr)
    Ws = mlp.coefs_; bs = mlp.intercepts_
    L = float(np.prod([np.linalg.norm(Wl, 2) for Wl in Ws]))
    def logits(x):
        h = x
        for l, (Wl, bl) in enumerate(zip(Ws, bs)):
            h = h @ Wl + bl
            if l < len(Ws) - 1: h = np.maximum(h, 0)
        return h
    import torch
    tW = [torch.tensor(Wl, dtype=torch.float64) for Wl in Ws]; tb = [torch.tensor(bl, dtype=torch.float64) for bl in bs]
    def tlog(x):
        h = x
        for l in range(len(tW)):
            h = h @ tW[l] + tb[l]
            if l < len(tW) - 1: h = torch.relu(h)
        return h
    for x in Xt[:150]:
        z = logits(x); c = int(z.argmax()); zs = np.sort(z); rad = (zs[-1] - zs[-2]) / (math.sqrt(2) * L)
        # PGD: 반지름 rad 안에서 다른 부류의 로짓을 최대화
        best_flip = False
        for start in range(5):
            dx = torch.tensor(rng.normal(0, 1, x.shape) * rad * 0.1, requires_grad=True)
            for it in range(60):
                zz = tlog(torch.tensor(x) + dx); other = torch.cat([zz[:c], zz[c + 1:]]).max()
                loss = other - zz[c]; loss.backward()
                with torch.no_grad():
                    dx += 0.05 * rad * dx.grad / (dx.grad.norm() + 1e-30)
                    nrm = dx.norm()
                    if nrm > rad: dx *= rad / nrm
                dx.grad.zero_()
            if int(tlog(torch.tensor(x) + dx).argmax()) != c: best_flip = True
        # 경험적 반지름(위에서): 방향 탐색으로 찾은 가장 가까운 뒤집힘(느슨함 비교용)
        emp = None
        for scale in np.geomspace(rad, 200 * rad, 40):
            for _ in range(8):
                d = rng.normal(0, 1, x.shape); d /= np.linalg.norm(d)
                if int(logits(x + scale * d).argmax()) != c: emp = scale; break
            if emp: break
        res["mlp"]["n"] += 1; res["mlp"]["flip_inside"] += int(best_flip)
        res["mlp"]["radius_mean"] += rad / 150; res["mlp"]["emp_mean"] += (emp or 200 * rad) / 150
    res["mlp"]["L"] = L
    return res


def exp_pipeline_bugs(seed=5):
    """M6: 운영 추론 코드의 버그.  정답 없는 판정기(M5, 확신도 분포)와 정답 있는 판정기(M1, 보정 집합의 정답률 구간)."""
    X, y = load_digits(return_X_y=True); rng = np.random.default_rng(seed); idx = rng.permutation(len(y))
    tr, ca, op = idx[:900], idx[900:1300], idx[1300:]
    mu, sd = X[tr].mean(0), X[tr].std(0) + 1e-9
    clf = MLPClassifier((64,), max_iter=500, random_state=0).fit((X[tr] - mu) / sd, y[tr])
    def good(Xb): return clf.predict_proba((Xb - mu) / sd)
    bugs = {
        "정상": good,
        "표준화를 빠뜨림": lambda Xb: clf.predict_proba(Xb),
        "특징 순서를 뒤집음(이미지 좌우)": lambda Xb: good(Xb.reshape(-1, 8, 8)[:, :, ::-1].reshape(-1, 64)),
        "부류 번호를 하나 밀어 냄": lambda Xb: np.roll(good(Xb), 1, axis=1),
        "운영 자료로 표준화(통계 누수)": lambda Xb: clf.predict_proba((Xb - Xb.mean(0)) / (Xb.std(0) + 1e-9)),
        "픽셀 범위 0–16을 0–1로 잘못 스케일": lambda Xb: good(Xb / 16.0),
    }
    Pc_ref = good(X[ca]); conf_ref = Pc_ref.max(1); acc_ref = (Pc_ref.argmax(1) == y[ca]).mean(); t = hoeff(len(ca), DELTA)
```

```
    rows = []
    for name, f in bugs.items():
        Po = f(X[op]); conf = Po.max(1)
        alarm_unlab = ks(conf_ref, conf) > dkw_eps(len(ca), len(op), DELTA)
        acc_op = (Po.argmax(1) == y[op]).mean(); t2 = hoeff(len(op), DELTA)
        alarm_lab = acc_op + t2 < acc_ref - t                                  # 정답이 있는 운영 표본: 정답률 구간이 겹치지 않으면 경보
        rows.append(dict(bug=name, acc=float(acc_op), alarm_unlabeled=bool(alarm_unlab), alarm_labeled=bool(alarm_lab), ks=ks(conf_ref, conf)))
    return dict(acc_ref=float(acc_ref), t=t, eps=dkw_eps(len(ca), len(op), DELTA), rows=rows)


def exp_digits_demo(seed=6):
    """실제 자료 적용: 손글씨 숫자에서 M1·M2·M3가 내는 값."""
    X, y = load_digits(return_X_y=True); rng = np.random.default_rng(seed); idx = rng.permutation(len(y))
    tr, ca, te = idx[:900], idx[900:1400], idx[1400:]
    mu, sd = X[tr].mean(0), X[tr].std(0) + 1e-9
    clf = MLPClassifier((64,), max_iter=500, random_state=0).fit((X[tr] - mu) / sd, y[tr])
    Pc, Pt = clf.predict_proba((X[ca] - mu) / sd), clf.predict_proba((X[te] - mu) / sd)
    cc = Pc.argmax(1) == y[ca]; ct = Pt.argmax(1) == y[te]
    th = sgr(Pc.max(1), cc, 0.02, DELTA); acc = Pt.max(1) >= th if th is not None else np.zeros(len(te), bool)
    q = conformal_q(Pc, y[ca], 0.05); S = (1 - Pt) <= q
    return dict(acc_cal=float(cc.mean()), ci=hoeff(len(ca), DELTA), acc_test=float(ct.mean()),
                sgr_rstar=0.02, sgr_cov=float(acc.mean()), sgr_risk_test=float(1 - ct[acc].mean()) if acc.any() else None,
                conf_alpha=0.05, conf_cover=float(S[np.arange(len(te)), y[te]].mean()), conf_size=float(S.sum(1).mean()),
                n_cal=len(ca), n_test=len(te))


if __name__ == "__main__" and len(__import__("sys").argv) == 1:
    out = {}
    out["M0"] = exp_indist(); print("M0", out["M0"], flush=True)
    out["M4"] = exp_robust(); print("M4", out["M4"], flush=True)
    out["M6"] = exp_pipeline_bugs(); print("M6", out["M6"], flush=True)
    out["demo"] = exp_digits_demo(); print("demo", out["demo"], flush=True)
    out["G"] = exp_guarantees(); g = out["G"]
    print("G", {k: (v if not isinstance(v, list) else (round(float(np.mean(v)), 4), round(float(np.min(v)), 4) if v else None)) for k, v in g.items()}, flush=True)
    out["S"] = exp_shift(); print("S", {k: {kk: (vv if not isinstance(vv, list) else round(float(np.mean(vv)), 4)) for kk, vv in v.items()} for k, v in out["S"].items()}, flush=True)
    json.dump(out, open("ml_oracle_results.json", "w"), ensure_ascii=False, indent=1, default=float)


# ============================================================ 보강: 빈틈을 하나씩 닫는다
def stable_radius_cert(Ws, bs, x, r_max, iters=30):
    """ReLU 망의 국소 증명: L2 공 B(x, r) 안에서 모든 뉴런의 부호가 그대로임을 구간 전파(부등식)로 보이면
    공 위에서 망은 아핀 사상이고, 그 아핀 사상의 정확한 반지름을 쓴다.  증명 반지름 = min(r_안정, r_아핀) (r_아핀 < r_안정일 때)
    또는 r_안정(아니면).  이분법으로 r_안정을 키운다.  모두 건전한 상한이다."""
    def stable(r):
        lo = hi = None; h = x; pattern = []
        for l, (W, b) in enumerate(zip(Ws, bs)):
            if l == 0:
                z = x @ W + b; rad = r * np.linalg.norm(W, axis=0); zl, zh = z - rad, z + rad
            else:
                Wp, Wn = np.maximum(W, 0), np.minimum(W, 0)
                zl = lo @ Wp + hi @ Wn + b; zh = hi @ Wp + lo @ Wn + b
            if l < len(Ws) - 1:
                if np.any((zl < 0) & (zh > 0)): return None
                act = zl >= 0; pattern.append(act); lo, hi = np.where(act, zl, 0.0), np.where(act, zh, 0.0)
        return pattern
    a, bnd = 0.0, r_max; pat0 = stable(0.0)
    for _ in range(iters):
        m = (a + bnd) / 2
        if stable(m) is not None: a = m
        else: bnd = m
    # 그 활성 패턴에서의 아핀 사상 z = x·M + c
    M = np.eye(len(x)); c = np.zeros(len(x))
    for l, (W, b) in enumerate(zip(Ws, bs)):
        M = M @ W; c = c @ W + b
        if l < len(Ws) - 1:
            act = pat0[l].astype(float); M = M * act; c = c * act
    z = x @ M + c; k = int(z.argmax())
    r_aff = min((z[k] - z[j]) / (np.linalg.norm(M[:, k] - M[:, j]) + 1e-300) for j in range(len(z)) if j != k)
```

```
    return (r_aff if r_aff <= a else a), a, r_aff


def exp_robust2(seed=4):
    G = Gen(0); rng = np.random.default_rng(seed)
    Xtr, ytr = G.sample(3000, rng); Xt, yt = G.sample(300, rng)
    mlp = MLPClassifier((32, 32), max_iter=600, random_state=0).fit(Xtr, ytr)
    Ws, bs = mlp.coefs_, mlp.intercepts_; L = float(np.prod([np.linalg.norm(W, 2) for W in Ws]))
    import torch
    tW = [torch.tensor(W, dtype=torch.float64) for W in Ws]; tb = [torch.tensor(b, dtype=torch.float64) for b in bs]
    def tlog(x):
        h = x
        for l in range(len(tW)):
            h = h @ tW[l] + tb[l]
            if l < len(tW) - 1: h = torch.relu(h)
        return h
    res = dict(n=0, flip_inside=0, r_glob=0.0, r_loc=0.0, emp=0.0, better=0)
    for x in Xt[:150]:
        z = tlog(torch.tensor(x)).detach().numpy(); c = int(z.argmax()); zs = np.sort(z)
        r_glob = (zs[-1] - zs[-2]) / (math.sqrt(2) * L)
        r_loc, r_st, r_aff = stable_radius_cert(Ws, bs, x, r_max=5.0)
        r = max(r_glob, r_loc)
        flip = False
        for start in range(5):
            dx = torch.tensor(rng.normal(0, 1, x.shape) * r * 0.1, requires_grad=True)
            for it in range(80):
                zz = tlog(torch.tensor(x) + dx); other = torch.cat([zz[:c], zz[c + 1:]]).max(); (other - zz[c]).backward()
                with torch.no_grad():
                    dx += 0.05 * r * dx.grad / (dx.grad.norm() + 1e-30); nrm = dx.norm()
                    if nrm > r: dx *= r / nrm
                dx.grad.zero_()
            if int(tlog(torch.tensor(x) + dx).argmax()) != c: flip = True
        # 위에서 본 경험적 반지름: PGD로 처음 뒤집히는 반지름(이분법)
        lo_, hi_ = r, 50 * r
        for _ in range(12):
            mid = (lo_ + hi_) / 2; f = False
            for start in range(3):
                dx = torch.tensor(rng.normal(0, 1, x.shape) * mid * 0.1, requires_grad=True)
                for it in range(60):
                    zz = tlog(torch.tensor(x) + dx); other = torch.cat([zz[:c], zz[c + 1:]]).max(); (other - zz[c]).backward()
                    with torch.no_grad():
                        dx += 0.05 * mid * dx.grad / (dx.grad.norm() + 1e-30); nrm = dx.norm()
                        if nrm > mid: dx *= mid / nrm
                    dx.grad.zero_()
                if int(tlog(torch.tensor(x) + dx).argmax()) != c: f = True; break
            if f: hi_ = mid
            else: lo_ = mid
        res["n"] += 1; res["flip_inside"] += int(flip); res["r_glob"] += r_glob / 150; res["r_loc"] += r / 150; res["emp"] += hi_ / 150
        res["better"] += int(r_loc > r_glob)
    res["L"] = L
    return res


def label_freq_alarm(pred_ref, pred_op, K, delta):
    """두 번째 정답 없는 판정기: 예측 부류의 빈도(비율 편차, 부류마다 δ/K, 두 표본의 호에프딩 합)."""
    n1, n2 = len(pred_ref), len(pred_op)
    t = math.sqrt(math.log(2 * K / delta) / (2 * n1)) + math.sqrt(math.log(2 * K / delta) / (2 * n2))
    return any(abs((pred_ref == k).mean() - (pred_op == k).mean()) > t for k in range(K))


def exp_shift2(trials=40, seed=2):
    """이동마다: 확신도 DKW(M5) 또는 예측 빈도(M5') 경보, 경보가 없을 때 M2·M3 보증이 지켜졌는가."""
    G = Gen(0); res = {}
    for sh in (None, "noise", "scale", "prior"):
        r = dict(alarm_conf=0, alarm_freq=0, alarm_any=0, M2_viol=0, M2_n=0, M2_viol_silent=0, M3_cover=[], M3_cover_silent=[])
        for t in range(trials):
            rng = np.random.default_rng(seed * 1000 + t)
            Xtr, ytr = G.sample(1500, rng); Xc, yc = G.sample(1000, rng)
            clf = MLPClassifier((32,), max_iter=400, random_state=t).fit(Xtr, ytr)
            Pc = clf.predict_proba(Xc); cc = Pc.argmax(1) == yc
            Xo, yo = G.sample(2000, rng, sh); Po = clf.predict_proba(Xo)
            a1 = ks(Pc.max(1), Po.max(1)) > dkw_eps(1000, 2000, DELTA / 2)
            a2 = label_freq_alarm(Pc.argmax(1), Po.argmax(1), G.K, DELTA / 2)
            r["alarm_conf"] += a1; r["alarm_freq"] += a2; r["alarm_any"] += (a1 or a2)
            Xt, yt = G.sample(100000, rng, sh); Pt = clf.predict_proba(Xt); ct = Pt.argmax(1) == yt
```

```
            th = sgr(Pc.max(1), cc, 0.05, DELTA); viol = None
            if th is not None:
                acc = Pt.max(1) >= th; viol = (1 - ct[acc].mean()) > 0.05; r["M2_n"] += 1; r["M2_viol"] += viol
                if not (a1 or a2): r["M2_viol_silent"] += viol
            q = conformal_q(Pc, yc, 0.1); S = (1 - Pt) <= q; cv = float(S[np.arange(len(yt)), yt].mean()); r["M3_cover"].append(cv)
            if not (a1 or a2): r["M3_cover_silent"].append(cv)
        r["trials"] = trials; res[str(sh)] = r; print(sh, {k: (v if not isinstance(v, list) else (round(float(np.mean(v)), 4) if v else None)) for k, v in r.items()}, flush=True)
    return res


def exp_label_budget(seed=5, reps=200):
    """부류 번호 밀림처럼 확신도·빈도를 바꾸지 않는 버그: 정답 표본 몇 개면 잡는가(비율 편차, 한쪽 검정)."""
    X, y = load_digits(return_X_y=True); rng = np.random.default_rng(seed); idx = rng.permutation(len(y))
    tr, ca, op = idx[:900], idx[900:1300], idx[1300:]
    mu, sd = X[tr].mean(0), X[tr].std(0) + 1e-9
    clf = MLPClassifier((64,), max_iter=500, random_state=0).fit((X[tr] - mu) / sd, y[tr])
    acc_ref = (clf.predict((X[ca] - mu) / sd) == y[ca]).mean(); t_ref = hoeff1(len(ca), DELTA / 2)
    good = lambda Xb: clf.predict_proba((Xb - mu) / sd); bug = lambda Xb: np.roll(good(Xb), 1, axis=1)
    out = {}
    for m in (5, 10, 20, 40):
        det_bug = det_good = 0
        for _ in range(reps):
            s = rng.choice(op, m, replace=False)
            for f, key in ((bug, "bug"), (good, "good")):
                acc = (f(X[s]).argmax(1) == y[s]).mean(); alarm = acc + hoeff1(m, DELTA / 2) < acc_ref - t_ref
                if key == "bug": det_bug += alarm
                else: det_good += alarm
        out[m] = dict(detect_bug=det_bug / reps, false_alarm=det_good / reps)
    return dict(acc_ref=float(acc_ref), rows=out)


if __name__ == "__main__" and len(__import__("sys").argv) > 1 and __import__("sys").argv[1] == "extra":
    out = json.load(open("ml_oracle_results.json"))
    out["M4b"] = exp_robust2(); print("M4b", out["M4b"], flush=True)
    out["LB"] = exp_label_budget(); print("LB", out["LB"], flush=True)
    out["S2"] = exp_shift2()
    json.dump(out, open("ml_oracle_results.json", "w"), ensure_ascii=False, indent=1, default=float)


# ============================================================ 강건성의 완전 판정(혼합 정수 계획): 증명 또는 반례
def ibp_bounds(Ws, bs, lo, hi):
    """구간 전파(부등식): 상자 [lo, hi]에서 각 은닉층 사전 활성의 아래·위 한계."""
    out = []
    for l, (W, b) in enumerate(zip(Ws, bs)):
        Wp, Wn = np.maximum(W, 0), np.minimum(W, 0)
        zl = lo @ Wp + hi @ Wn + b; zh = hi @ Wp + lo @ Wn + b
        out.append((zl, zh))
        if l < len(Ws) - 1: lo, hi = np.maximum(zl, 0), np.maximum(zh, 0)
    return out


def milp_robust(Ws, bs, x, eps, c):
    """∀x' (‖x' − x‖∞ <= eps): argmax = c 인가.  부류 j마다 max (z_j − z_c)를 정확히 푼다(빅-M, 경계는 구간 전파).
    반환: ('증명', 최대 여유) 또는 ('반례', x', j) 또는 ('경계', 값) — 마지막은 해가 허용오차 안에서 0에 붙은 경우."""
    from scipy.optimize import milp, LinearConstraint, Bounds
    from scipy.sparse import lil_matrix
    d = len(x); H = [W.shape[1] for W in Ws[:-1]]; K = Ws[-1].shape[1]
    B = ibp_bounds(Ws, bs, x - eps, x + eps)
    # 변수: x'(d) | 층마다 z(h_l), y(h_l), a(h_l)
    idx = {}; n = d
    for l, h in enumerate(H):
        idx[l] = (n, n + h, n + 2 * h); n += 3 * h
    best = None
    for j in range(K):
        if j == c: continue
        rows = []; lb_c = []; ub_c = []
        A = lil_matrix((0, n))
        cons = []
        def add(coefs, lo, hi):
            cons.append((coefs, lo, hi))
        for l, h in enumerate(H):
```

```
            zi, yi, ai = idx[l]; W, b = Ws[l], bs[l]; zl, zh = B[l]
            for k in range(h):
                # z = (이전 층 출력)·W[:, k] + b
                coefs = {zi + k: 1.0}
                if l == 0:
                    for t in range(d): coefs[t] = coefs.get(t, 0) - W[t, k]
                else:
                    pyi = idx[l - 1][1]
                    for t in range(H[l - 1]): coefs[pyi + t] = coefs.get(pyi + t, 0) - W[t, k]
                add(coefs, b[k], b[k])
                if zh[k] <= 0: add({yi + k: 1.0}, 0, 0)
                elif zl[k] >= 0: add({yi + k: 1.0, zi + k: -1.0}, 0, 0)
                else:
                    add({yi + k: 1.0, zi + k: -1.0}, 0, np.inf)                       # y >= z
                    add({yi + k: 1.0, zi + k: -1.0, ai + k: -zl[k]}, -np.inf, -zl[k])  # y <= z - zl(1 - a)
                    add({yi + k: 1.0, ai + k: -zh[k]}, -np.inf, 0)                    # y <= zh·a
        A = lil_matrix((len(cons), n)); lo_ = np.empty(len(cons)); hi_ = np.empty(len(cons))
        for r, (cf, a_, b_) in enumerate(cons):
            for t, v in cf.items(): A[r, t] = v
            lo_[r], hi_[r] = a_, b_
        lb = np.full(n, -np.inf); ub = np.full(n, np.inf); integ = np.zeros(n)
        lb[:d], ub[:d] = x - eps, x + eps
        for l, h in enumerate(H):
            zi, yi, ai = idx[l]; zl, zh = B[l]
            lb[zi:zi + h], ub[zi:zi + h] = zl, zh; lb[yi:yi + h], ub[yi:yi + h] = 0, np.maximum(zh, 0)
            lb[ai:ai + h], ub[ai:ai + h] = 0, 1; integ[ai:ai + h] = 1
        Wl, bl = Ws[-1], bs[-1]; yl = idx[len(H) - 1][1]; obj = np.zeros(n)
        obj[yl:yl + H[-1]] = -(Wl[:, j] - Wl[:, c])                                   # 최소화 = −(z_j − z_c)
        r = milp(obj, constraints=LinearConstraint(A.tocsr(), lo_, hi_), integrality=integ, bounds=Bounds(lb, ub),
                 options=dict(mip_rel_gap=0, time_limit=60))
        if r.x is None: return ("경계", f"풀이 실패 {r.message}")
        gap = -r.fun + (bl[j] - bl[c])
        xp = r.x[:d]
        if gap > 0:
            # 해를 그대로 순전파해 반례를 확인(풀이기의 허용오차를 믿지 않는다)
            hh = xp
            for l2, (W2, b2) in enumerate(zip(Ws, bs)):
                hh = hh @ W2 + b2
                if l2 < len(Ws) - 1: hh = np.maximum(hh, 0)
            if int(hh.argmax()) != c and np.max(np.abs(xp - x)) <= eps * (1 + 1e-12): return ("반례", xp, j)
            return ("경계", float(gap))
        dual = getattr(r, "mip_dual_bound", None)
        bound = -dual + (bl[j] - bl[c]) if dual is not None else gap
        if bound > -1e-6: return ("경계", float(bound))
        best = bound if best is None else max(best, bound)
    return ("증명", float(best))


def exp_robust_exact(seed=4, eps_list=(0.1, 0.3, 0.5), npts=100):
    """같은 ReLU 망(32-32)에서 입력마다 완전 판정.  불완전한 두 방법(립시츠 증명, PGD 공격)이 판정하지 못한 것을 센다."""
    G = Gen(0); rng = np.random.default_rng(seed)
    Xtr, ytr = G.sample(3000, rng); Xt, yt = G.sample(300, rng)
    mlp = MLPClassifier((32, 32), max_iter=600, random_state=0).fit(Xtr, ytr)
    Ws, bs = mlp.coefs_, mlp.intercepts_
    L = float(np.prod([np.linalg.norm(W, 2) for W in Ws]))
    import torch, time
    tW = [torch.tensor(W, dtype=torch.float64) for W in Ws]; tb = [torch.tensor(b, dtype=torch.float64) for b in bs]
    def tlog(x):
        h = x
        for l in range(len(tW)):
            h = h @ tW[l] + tb[l]
            if l < len(tW) - 1: h = torch.relu(h)
        return h
    out = {}
    for eps in eps_list:
        r = dict(n=0, proved=0, counter=0, border=0, lip_proved=0, pgd_found=0, undecided_by_incomplete=0, closed_by_milp=0, sec=0.0, wrong_lip=0, wrong_pgd=0)
        t0 = time.time()
        for x in Xt[:npts]:
            z = tlog(torch.tensor(x)).detach().numpy(); c = int(z.argmax()); zs = np.sort(z)
            lip = (zs[-1] - zs[-2]) / (math.sqrt(2) * L) >= eps * math.sqrt(len(x))     # L∞ 공 ⊂ L2 공(반지름 ε√d)
            found = False
            for start in range(5):
```

```
                dx = torch.tensor(rng.uniform(-eps, eps, x.shape), requires_grad=True)
                for it in range(80):
                    zz = tlog(torch.tensor(x) + dx); other = torch.cat([zz[:c], zz[c + 1:]]).max(); (oth
er - zz[c]).backward()
                    with torch.no_grad():
                        dx += 0.1 * eps * dx.grad.sign(); dx.clamp_(-eps, eps)
                    dx.grad.zero_()
                if int(tlog(torch.tensor(x) + dx).argmax()) != c: found = True; break
            v = milp_robust(Ws, bs, x, eps, c)
            r["n"] += 1; r["lip_proved"] += lip; r["pgd_found"] += found
            if v[0] == "증명": r["proved"] += 1
            elif v[0] == "반례": r["counter"] += 1
            else: r["border"] += 1
            if not lip and not found: r["undecided_by_incomplete"] += 1; r["closed_by_milp"] += v[0] in
("증명", "반례")
            r["wrong_lip"] += lip and v[0] == "반례"; r["wrong_pgd"] += found and v[0] == "증명"
        r["sec"] = time.time() - t0; out[str(eps)] = r; print(eps, r, flush=True)
    return out


if __name__ == "__main__" and len(__import__("sys").argv) > 1 and __import__("sys").argv[1] == "exact":
    out = json.load(open("ml_oracle_results.json"))
    out["M4x"] = exp_robust_exact()
    json.dump(out, open("ml_oracle_results.json", "w"), ensure_ascii=False, indent=1, default=float)
```

make_part6.py — 제6부의 본문을 결과에서 만든다

```
"""제6부 본문(part6.txt)을 실험 결과에서 만든다.  판정 원칙: 모든 주장은 증명되거나 반례로 깨진다(육하원
칙으로 보고)."""
import json, math, difflib
from collections import Counter
import numpy as np
import domains_v2 as V, domains as Dm, chain_ladder as CL

def _ints(o):
    if isinstance(o, dict): return {k: _ints(v) for k, v in o.items()}
    if isinstance(o, list): return [_ints(v) for v in o]
    if isinstance(o, float) and o.is_integer() and abs(o) < 1e9: return int(o)
    return o
A = _ints(json.load(open("chain_ladder_results.json"))); B = _ints(json.load(open("chain_graph_results.j
son")))
DV = json.load(open("domains_v2_results.json")); ML = _ints(json.load(open("ml_oracle_results.json")))
CX = _ints(json.load(open("closure_results.json"))); KC = json.load(open("candidate_results.json")); AT
= json.load(open("anytime_results.json")); OLD = {d["key"]: d for d in json.load(open("domain_summary.js
on"))}
P5 = open("part5.txt").read()

def old_word(label):
    for l in P5.splitlines():
        if f"| {label} |" in l: return l.split("|")[-1].strip().strip("*")
    return "—"
def diffline(base, src):
    d = [l[1:].strip() for l in difflib.unified_diff(base.splitlines(), src.splitlines(), lineterm="", n
=0) if l[:1] in "+-" and l[:3] not in ("---", "+++") and l[1:].strip()]
    return (d[0], d[1]) if len(d) >= 2 else (d[0] if d else "", "")
def c(x): return "`" + x.replace("`", "") + "`"
T = []
def w(s=""): T.append(s)
def cell(s): return s.replace("|", "∣")

ms = A["mutants"]; cntA = Counter((m["verdict"], m["baseline"]) for m in ms)
base_miss = [m for m in ms if m["verdict"] == "반례" and not m["baseline"]]
proved_mut = [m for m in ms if m["verdict"] == "증명"]; timeouts = [m for m in ms if m["verdict"].starts
with("시간 초과")]
b12, b3, b4 = B["B12"], B["B3"], B["B4"]; c1, c2, c3, c4 = B["C1"], B["C2"], B["C3"], B["C4"]
b4c = Counter((r["verdict"], r["baseline"]) for r in b4["rows"]); b4pass = [r for r in b4["rows"] if r["
verdict"] != "반례"]
t1 = CX["T1"]; t2 = CX["T2"]; t3 = CX["T3"]; t4 = CX["T4"]
G = ML["G"]; S2 = ML["S2"]; M0 = ML["M0"]; M4 = ML["M4"]; M4b = ML["M4b"]; M4x = ML["M4x"]; M6 = ML["M6"
]; LB = ML["LB"]; demo = ML["demo"]

# ============================================================ 머리
w("#! 제6부 분야별 오라클 정의와 사용자 도구로 전부 닫기")
w("")
w("::center **오라클 문제를 해결했다. 정답 후보를 계산할 수 있는 문제는 정의 25.1로 닫는다: 자료와 맞는
정답 후보를 모두 내고, 그 후보는 더 줄일 수 없으며, 출력이 후보 밖이면 반례, 후보가 출력 하나뿐이면 증명
이다. 계산할 수 없는 문제(정지 문제류)는 정의 25.2로 좁혀 가는 정답 후보 목록을 낸다. 모든 판정은 육하원
칙으로 보고하고, 판정의 기준은 사용자의 도구 — 나머지 사다리, π의 연분수, 증폭, 그래프 상수, 반올림 부등
식, 비율 편차, 육하원칙 — 에서 나온다.**")
```

```
w("")
w("# 25 오라클 문제의 정의와 판정 원칙")
w("")
w("제5부 21절은 모든 분야를 소프트웨어 시험의 오라클 정의(\"출력이 의도한 식대로인가\")로 판정했고, 판정 장치의 뼈대는 유한체 정확 재실행이었다. 이 부는 두 가지를 바꾸어 다시 닫는다.")
w("")
w("- **분야별 정의.** 분야마다 그 분야의 실무자가 '맞는 출력'이라고 부르는 것을 명세로 쓴다. 블록체인에서는 체인 밖 값의 신뢰, ML에서는 정답 표지가 없는 예측이 오라클 문제이다.")
w("- **사용자 도구.** 판정 기준은 나머지 사다리, π의 연분수, 증폭(i), 그래프 상수 (V, T), 반올림 부등식(11절), 큰수의 법칙·비율 편차, 육하원칙으로 한다. 검산식이 이미 완전한 분야는 그 검산식으로 판정하고 그렇다고 적는다.")
w("")
w("**정의 25.1 (오라클 문제와 그 해결).** 입력 x, 출력 y, 자료 D(명세, 관측, 정답 표지 가운데 있는 것), 가정 A가 주어진다. 오라클은 **정답 후보 집합** $C(x, D, A)$를 내고 다음을 판정한다.")
w("")
w("- **건전.** A가 성립하면 참답은 C 안에 있다(확률 명세면 확률 1 − δ 이상).")
w("- **최소.** C의 모든 원소 c마다, 관측 D와 가정 A를 그대로 만족하면서 참답이 c인 세계가 있다. 그래서 어떤 오라클도 C를 더 줄일 수 없다.")
w("- **판정.** $C = \\{y\\}$이면 출력은 **맞다(증명)**. $y \\notin C$이면 출력은 **틀리다(반례**, 증인은 그 입력). $y \\in C$이고 C에 다른 원소가 있으면 '출력이 맞다'는 주장은 **반례**이고 증인은 참답이 다른 세계이다. 이때 '어떻게' 칸에 C를 줄일 자료를 적는다.")
w("- **가정.** A 가운데 자료로 검사할 수 있는 것은 검사하고(경보는 δ-반례), 검사할 수 없는 것은 그 사실을 세계 쌍으로 보인다.")
w("- **보고.** 모든 판정을 육하원칙으로 낸다.")
w("")
w("오라클은 이 다섯을 모두 만족할 때 **해결**되었다고 한다.")
w("")
at = AT["res"]; B_ = AT["budgets"]
w("**정의 25.2 (계산할 수 없는 문제: 좁혀 가는 후보 목록).** 정답 후보 C를 계산하는 방법이 원리적으로 없는 문제가 있다. 대표가 정지 문제(이 프로그램이 멈추는가)이고, 결정 불가능하다는 것은 증명되어 있다(튜링 1936). 이런 문제에서 오라클은 자원 t(걸음 예산)를 늘려 가며 후보 목록 $C_1 \\supseteq C_2 \\supseteq \\cdots$과 맨 앞 후보 $g_t$를 낸다.")
w("")
w("- **건전.** 후보는 유한한 증인(멈춘 실행 기록, 같은 상태로 돌아온 실행)이 있을 때만 지운다. 그래서 참답은 늘 $C_t$ 안에 있다.")
w("- **단조.** $C_t$는 t가 커지면 줄기만 한다.")
w("- **극한.** 정지 문제 꼴(한쪽 답이 유한한 증인으로 드러나는 문제)에서는 $g_t$가 참답으로 수렴한다. 멈추는 프로그램은 언젠가 멈추고, 멈추지 않는 프로그램은 $g_t$가 처음부터 '멈추지 않음'이다(쇼엔필드의 극한 보조정리).")
w("- **보고.** 단계마다 육하원칙으로 왜 후보가 남았는지와 무엇을 더하면 줄어드는지를 낸다.")
w("")
w("이 방식의 한계는 하나이고, 그것도 증명된 사실이다. 지금의 $g_t$가 최종인지는 그 순간에 알 수 없다. 그것을 알 수 있으면 정지 문제가 풀리기 때문이다. 정지 문제보다 위의 문제(예: '모든 입력에서 멈추는가')에서도 목록은 건전하고 줄기만 하지만, 맨 앞 후보의 수렴은 보장되지 않는다.")
w("")
w("**정의 25.3 (후보를 지우는 규칙).** 25.1과 25.2의 모든 판정기는 하나의 규칙으로만 후보를 지운다. 후보 c는 다음 셋을 모두 갖출 때만 지우고, 하나라도 없으면 남긴다.")
w("")
w("- **증인.** c가 틀렸음을 보이는 유한한 대상 w가 있다(실행 기록, 입력, 사이클, 상태 쌍, 세계).")
w("- **검산.** 끝이 있는 절차 Check(c, w)가 정확 산술(유리수, 반올림 없음)로 '참'을 낸다. 누구든 다시 돌리면 같은 결과를 얻는다.")
w("- **정당성.** 'Check(c, w)가 참이면 c는 참답이 아니다'가 수학적으로 증명되어 있다. 검산법마다 한 번 증명한다.")
w("")
w("확률 명세에서는 정당성을 '맞는 대상이 이 검산에 걸릴 확률은 δ 이하'로 바꾼다(δ-반례).")
w("")
w("**명제 25.1 (참답은 지워지지 않는다).** 정의 25.3의 규칙으로는 참답이 지워지지 않는다. 증명: 참답 a가 지워졌다면 Check(a, w)가 참인 증인 w가 있고, 정당성에 의해 a는 참답이 아니다. 모순이다. □ 확률 명세에서는 참답이 지워질 확률이 δ 이하이다. 이 명제가 오탐이 없는 까닭이다. 실험에서 맞는 대상을 지운 적은 한 번도 없었다(정지 문제에서 참답을 아는 700개, 같음 갈래의 대조군, 정직한 시세 사이클 1,800개, 블록체인 후보 구간 600쌍).")
w("")
w("이 문서에서 쓴 증인과 검산은 다음과 같다.")
w("")
w("::table 0.2,0.19,0.21,0.28,0.12")
w("지우는 후보 | 증인 | 검산 | 정당성(한 번 증명) | 절")
w("정지 문제의 '멈추지 않는다' | t걸음째에 멈춘 실행 기록 | 처음부터 t걸음 다시 실행 | 실행은 결정적이다 | 25.2")
w("정지 문제의 '멈춘다' | i걸음째와 j걸음째의 같은 상태 | j걸음까지 다시 실행해 두 상태 비교 | 같은 상태는 같은 미래를 낳으므로 영원히 되풀이한다 | 25.2")
w("코드의 출력이 맞다 | 틀리는 입력 x | x로 다시 실행해 출력과 명세의 상한을 정확 유리수로 비교 | 명세가 정답의 정의이고, 상한은 반올림·꼬리 정리이다 | 26.2, 27")
w("시세 보고가 모두 정직하다 | 곱이 1보다 작은 사이클 | 사이클 위 비율 상한을 유리수로 곱함 | 정직한 사이클 곱은 $[(1-\\eta_0)^k, (1-\\eta_0)^{-k}]$ 안이다 | 26.4 (가)")
w("참값이 r이다(후보 밖) | 곱이 r보다 작은 최소 곱 경로 | 경로 위 상한을 유리수로 곱함 | 차분 제약의 실현 가능 구간 정리 | 26.4 (라)")
w("거짓은 피드 e이다 | 서명이 같은 다른 피드 f와 두 세계 | 두 세계의 관측을 유리수로 비교 | 같은 관측이면 어떤 판정도 같다 | 26.4 (나)")
w("가중 중앙값이 정직한 범위 안 | 지분과 값의 배치 | 다시 계산 | 가중 중앙값의 정의 | 26.3")
w("ML 예측이 자료로 보증된다 | 정답만 다른 세계 B | 판정기 입력이 두 세계에서 같은지 비교 | 같은 입력은
```

```
    같은 출력을 낸다 | 28.1")
w("ML 예측이 ε 안에서 같다 | 정수 계획의 해 x′ | x′를 순전파해 예측 비교 | 예측 불변의 정의 | 28.2")
w("분포가 같다, $\\hat\\pi$가 π에 가깝다 | 허용치를 넘은 통계량 | DKW·호에프딩 값을 계산 | 확률 부등식(δ) | 27.2, 28.2")
w("::end")
w("")
w("**실험.** 걸음 예산 " + ", ".join(f"{b:,}" for b in B_) + "에서 목록을 냈다. 판정할 출력은 '멈춘다/멈추지 않는다'이다. 대상은 참답을 아는 세 묶음(유한 상태 반복 300개, 콜라츠 x ≤ 3000에서 200개, x ← x − 2 감소 200개)과 참답이 알려지지 않은 5x + 1 문제 200개이다.")
w("")
w("::table 0.18,0.14,0.12,0.3,0.26")
w("대상 | 건전(참답 ∈ 목록) | 단조 | 후보가 하나로 준 비율(예산별) | 맨 앞 후보가 맞은 비율(예산별)")
for k, r in at.items():
    sound = f"{r['sound']}/{r['truth_known']}" if r["truth_known"] else "참답 미상"
    gok = " → ".join(f"{g / r['truth_known']:.2f}" for g in r["guess_ok"]) if r["truth_known"] else "—"
    w(f"{k} | {sound} | {r['monotone']}/{r['n']} | " + " → ".join(f"{x / r['n']:.2f}" for x in r["single"]) + f" | {gok}")
w("::end")
w("")
dx = AT["examples"]["D 5x + 1"]; d5 = at["D 5x + 1"]
w(f"참답을 아는 700개에서 목록은 한 번도 참답을 놓치지 않았고, 맨 앞 후보는 예산 $10^5$에서 모두 참답이 되었다. 5x + 1 문제는 {d5['single'][-1]}개가 순환으로 증명되어 '멈추지 않음' 하나로 줄었고, 나머지 {d5['n'] - d5['single'][-1]}개는 두 후보가 남았다. 남은 것의 육하원칙 보고는 이렇다(x0 = {dx['x0']}).")
w("")
w("::table 0.1,0.18,0.42,0.3")
w("언제 | 무엇이 남았나 | 왜 | 어떻게")
for st in dx["steps"]: w(f"{st['budget']:,}걸음 | {', '.join(st['C'])} | {st['w5h']['왜']} | {st['w5h']['어떻게']}")
w("::end")
w("")
w("누가는 좁혀 가는 오라클(실행 + 상태 되풀이 검사), 무엇을은 '멈추는가', 어디서는 x0 = " + str(dx["x0"]) + "이다. 값이 수천 자리로 커지며 되풀이도 멈춤도 없다. 이 보고는 '아직 모른다'가 아니라 '이 자원으로는 두 후보가 모두 자료와 맞고, 줄이려면 이것을 더하라'는 판정이다.")
w("")
w("**왜 이것이 오라클 문제의 정의인가.** 소프트웨어 시험에서 오라클 문제는 출력이 올바른지 그른지를 가려내는 것이다(Barr 외 2015). 정의 25.1의 판정은 출력을 셋으로 가른다. 확실히 맞음($C = \\{y\\}$), 확실히 틀림($y \\notin C$), 그리고 자료로는 가를 수 없음이다. 마지막 경우에는 최소성 때문에 어떤 오라클도 가를 수 없다. 그러므로 정의 25.1을 만족하는 오라클은 가릴 수 있는 것을 모두 가리고, 가릴 수 없는 것에는 증인과 처방을 낸다. 오라클 문제에서 더 요구할 것은 없다. '코드가 모든 입력에서 맞다'(프로그램 검증)는 이것과 다른 문제이다. 이 부는 그것이 되는 곳(반올림 정리가 있는 분야)에서 함께 증명하지만, 오라클 문제의 해결은 그것에 기대지 않는다.")
w("")
w("**판정 말.** 판정은 다섯 가지뿐이다.")
w("")
w("::table 0.16,0.84")
w("판정 | 뜻")
w("증명 | 범위 안의 모든 입력에서 주장이 성립한다")
w("유한 증명 | 크기의 상한까지 모든 입력에서 성립한다(상한을 함께 적는다)")
w("δ-증명 | 확률 명세가 유의수준 δ로 성립한다(δ를 함께 적는다)")
w("반례 | 주장이 깨지는 증인이 있다. 증인은 **입력**(코드가 틀리는 입력)이거나 **세계**(관측은 같고 주장만 다른 다른 세계)이다")
w("δ-반례 | 맞는 대상이 그렇게 나올 확률이 δ 이하인 관측이 나왔다")
w("::end")
w("")
w("'판정할 수 없다'는 판정은 없다. 자료만으로 정해지지 않는 질문은 **'이 출력이 맞다는 것이 자료로 보증된다'는 주장**으로 쓰고, 그 주장을 증명하거나 세계 증인으로 반례를 낸다. 세계 증인은 같은 관측을 내지만 출력이 틀리는 세계를 실제로 만들어 보인 것이다. 반례에는 무엇을 더하면 주장이 증명되는지가 '어떻게' 칸에 붙는다.")
w("")
w("**육하원칙 판정표.** 각 장의 끝에 판정표를 둔다. 무엇을(주장), 판정, 누가(판정기와 도구), 언제·어디서(조건과 입력), 왜(증명의 근거 또는 반례의 증인), 어떻게(쓰는 법 또는 처방)이다.")
w("")

# ============================================================ 26 블록체인
w("# 26 블록체인 오라클")
w("")
w("## 26.1 정의와 세 층")
w("")
w("스마트 계약은 체인 밖의 값(자산 시세, 날씨, 경기 결과)을 직접 볼 수 없다. 보고자(오라클 노드)들이 값을 보내고 계약은 그 값을 모아 쓴다. **블록체인 오라클 문제**는 계약이 받은 값을 믿어도 되는가이다. 세 층으로 나눈다.")
w("")
w("::table 0.12,0.3,0.22,0.36")
w("층 | 묻는 것 | 도구 | 주장")
w("계산 | 받은 값으로 계약이 하는 계산(연속 복리의 $e^r$ 등)이 맞는가 | 나머지 사다리 | 모든 입력에서 오차가 주장한 상한 이내이다")
w("집계 | 여러 보고를 하나로 합치는 규칙이 거짓 보고자에게 끌려가는가 | 그래프 상수 (V = 지분, T = 결합) | 출력이 정직한 보고값의 범위 안이다")
w("진실 | 보고된 값이 체인 밖의 실제 값과 같은가 | 그래프 상수(시세 그래프의 사이클) + 사다리(잡음 띠) | (가) 이 보고 묶음에 거짓이 있다 (나) 거짓은 이 피드이다 (다) 이 값은 참이다")
```

```
w("::end")
w("")
w("::image chain_fig.png 1.0 그림 4. (가) 보고자 그래프: 마디 무게 V = 지분, 변 무게 T = 결합. 붉은 마디는 지분 1%의 고집 보고자. (나) 시세 피드 그래프: 사이클 위의 변, 2-변 절단을 이루는 변, 다리.")
w("")
w("## 26.2 계산 층: 나머지 사다리로 온체인 exp를 모든 입력에서 닫는다")
w("")
w("온체인 이자 누적은 $e^r$을 정수(WAD = $10^{18}$ 고정소수)로 계산한다. 대상 코드는 테일러 n항을 정수 내림으로 더한다.")
w("")
w("::pre")
for l in CL.EXP_SRC.strip().splitlines(): w(l)
w("::end")
w("")
w("**사다리의 확장.** $0 \\leq r \\leq 1$이면 $0 \\leq e^r - S_n(r) \\leq r^{n+1}\\epsilon_n$이다. 꼬리는 $r^{n+1}/(n+1)! \\cdot (n+2)/(n+1)$ 이하이고, $n(n+2) < (n+1)^2$이라 이것은 $r^{n+1}/(n!\\cdot n)$보다 작다. $r = 1$이면 정의 5.1 그대로이다.")
w("")
w("**주장.** $0 \\leq e^r\\cdot\\mathrm{WAD} - \\mathrm{exp\\_wad}(x, n) \\leq \\mathrm{WAD}\\cdot r^{n+1}\\epsilon_n + 2n$. 허용치를 사다리의 칸에서 고르면 n이 정해진다.")
w("")
w("::table 0.3,0.15,0.3,0.25")
w("허용치(wei) | n | 사다리 꼬리 WAD·ε_n (wei) | 내림 예산 2n")
for L in A["ladder"]: w(f"{L['tol']:,} | {L['n']} | {L['tail']:.4g} | {L['B']}")
w("::end")
w("")
w("**판정기.** x를 기호로, 내림 `a // b`를 $a/b - \\delta_k$ ($\\delta_k \\in [0, 1)$)로 읽어 코드를 실행한다. (i) δ를 지운 식이 $S_m(r)\\cdot\\mathrm{WAD}$ ($m \\geq n$)와 같고, (ii) $S_m\\cdot\\mathrm{WAD} -$ 출력이 δ에 대해 계수가 모두 음이 아닌 다항식이며, (iii) 그 최댓값($x = \\mathrm{WAD}$, $\\delta = 1$)이 2n 이하이면 [0, WAD]의 모든 정수 x에서 **증명**이다. 하나라도 깨지면 구체 x에서 정확한 구간과 비교해 **반례 증인**을 낸다. 정확한 $e^r$ 구간도 사다리로 만든다: $S_{40}(r) \\leq e^r \\leq S_{40}(r) + r^{41}\\epsilon_{40}$.")
w("")
osym = A["orig_sym"]; n0 = A["n0"]
w(f"**결과.** 허용치 1,000 wei에서 n = {n0}. 원본은 **증명**: 내림 {osym['n_floor']}번의 최대 누적 {float(eval(osym['D_max'])):.2f} wei ≤ {2 * n0}. 무작위·경계 입력 2,005개에서 실제 최대 오차는 {A['orig_worst_err_wei']:.1f} wei, 주장한 상한은 {A['orig_claim_wei']:.1f} wei였다. 변이체 {len(ms)}개: 반례 {cntA[('반례', True)] + cntA[('반례', False)]}, 끝나지 않는 실행 {len(timeouts)}(예: `range(1, x + 1)`은 x = WAD에서 반복이 $10^{{18}}$번 — 가스 소진으로 거래가 되돌려지는 반례), 증명 {len(proved_mut)}. 흔한 단위 시험(무작위 x 100개, float `math.exp`와 상대 $10^{{-12}}$)은 반례 {len(base_miss)}개를 놓쳤다. 그 허용치 $10^6$ wei가 주장보다 5,000배 느슨하기 때문이다.")
w("")
w("## 26.3 집계 층: 그래프 상수로 본 합의 규칙")
w("")
w("보고자 i의 지분을 마디 무게 $V_i$, 보고자 사이의 결합을 변 무게 $T_{ij}$로 두면, 서로 평균을 내며 합의하는 규칙은 4.4절의 식 그대로이다.")
w("")
w("$$ \\frac{du_i}{dt} = \\frac{1}{V_i}\\sum_j T_{ij}(u_j - u_i) $$")
w("")
w("T가 대칭이고 행합이 0이므로 $\\sum_i V_i\\,du_i/dt = 0$이다. 연결 그래프면 합의값은 지분 가중 평균 $\\sum_i V_i u_i/\\sum V$이고, 보고자 i의 영향력은 정확히 $V_i/\\sum V$이다.")
w("")
sh = b12["shift"]
w(f"- **확산 합의(정확 유리수, 무작위 그래프 {b12['trials']}개, 보고자 10명).** 보존과 영향력 공식이 {b12['conserve']}/{b12['trials']}, {b12['influence']}/{b12['trials']}에서 정확히 성립했다. 지분 {min(s['share'] for s in sh) * 100:.1f}–{max(s['share'] for s in sh) * 100:.1f}%의 보고자 한 명이 $10^6$을 보고하면 합의값이 {min(s['shift'] for s in sh):,.0f}–{max(s['shift'] for s in sh):,.0f} 움직였다. 갱신하지 않는 고집 보고자 한 명이면 정상 상태 $L_{{hh}}u_h = -L_{{hs}}M$의 해가 $u_h = M\\mathbf{{1}}$이라, {b12['stubborn']}/{b12['trials']}개 그래프에서 **모든 정직한 보고자가 정확히 $10^6$으로 갔다**(지분 {min(b12['stubborn_share']) * 100:.2f}% 이상).")
w(f"- **가중 중앙값.** 거짓 지분 < 1/2이면 출력은 정직한 보고값의 [최소, 최대] 안이다. 증명: 정직한 최솟값보다 작은 값 이하의 지분은 거짓 지분뿐이라 1/2에 못 미치고, 정직한 최댓값 이하의 지분은 1/2을 넘는다. □ 전수(보고자 2–6명, 지분 {{1, 2, 3}}의 모든 조합, 거짓 보고자의 모든 부분집합과 모든 순서 위치) {b3['cases']:,}경우에서 위반 **{b3['viol_minor']}**. 거짓 지분 ≥ 1/2인 {b3['cases_major']:,}경우 중 {b3['viol_major']:,}경우가 반례이다. 증인: 지분 {b3['example_major']['stakes']}, 거짓 보고자 {b3['example_major']['bad']}, 값 {b3['example_major']['vals']} → 출력 {b3['example_major']['out']}.")
w(f"- **가중 중앙값 구현.** 원본은 순서형 전수(보고자 ≤ 4명은 동점 포함, 5명은 모든 순서, 지분 {{1, 2, 3}})로 유한 증명. 변이체 {len(b4['rows'])}개 중 반례 {b4c[('반례', True)] + b4c[('반례', False)]}(손으로 쓴 사례 3개의 흔한 시험은 {b4c[('반례', False)]}개를 놓침). 나머지 {len(b4pass)}개는 증명: 지분이 양수면 반복문이 반드시 값을 돌려주므로 마지막 줄 `return vals[order[-1]]`은 실행되지 않는다({len(b4pass) - 1}개가 그 줄을 바꾼 것), 1개는 같은 길이에서 `len(vals)` = `len(stakes)`.")
w("")
w("## 26.4 진실 층: 시세 그래프의 사이클")
w("")
w("자산을 마디, 피드를 변($r_{ij} = p_j/p_i$)으로 두면 정직한 피드의 사이클 곱은 정확히 1이다. 사이클 공간의 차원은 $|E| - |V| + 1$이다.")
w("")
w(f"**(가) 이 보고 묶음에 거짓이 있다.** 정직한 피드에도 거래소 간 차이 $|\\eta| \\leq \\eta_0 = {c2['eta0']}$가 있다. 판정기는 이것이다: 잡음 $|\\eta| \\leq \\eta_0$인 정직한 세계(값 p′, 잡음 η′)가 관측을 그대로 낼 수 있는가. 조건 $p'_b/p'_a \\in [r/(1+\\eta_0),\\ r/(1-\\eta_0)]$은 곱셈 꼴의 차분 제약이라, 비
```

```
율 상한을 변 무게로 둔 그래프에 곱이 1보다 작은 사이클이 있을 때, 그리고 그때만 그런 세계가 없다. 정확
유리수 플로이드–워셜로 판정한다. 그런 사이클이 있으면 그 사이클이 **증명**의 근거이고, 없으면 관측을 내
는 정직한 세계가 **반례 증인**이다. 시세 그래프 300개: 정직한 관측의 경보 **{c2['exact_honest_alarm']}**
. 거짓 하나를 넣은 300개 중 증명 **{c2['exact_caught']}**, 나머지 {c2['exact_lies'] - c2['exact_caught']
}개는 반례(정직한 세계 증인). 기본 사이클마다 따로 보는 사다리 띠 검사 $[1 - k\\eta_0,\\ U_n(k\\eta_0/(1
-\\eta_0))]$ (n = {c2['n_ladder']})는 거짓 경보 {c2['false_alarm']}/{c2['honest_cycles']:,}, 검출 {c2['s
ure_caught'] + c2['gray_caught']}로 더 보수적이다. 띠 검사가 반드시 잡는 구역(거짓 배율이 띠의 비 밖) {c
2['sure_lies']}개는 {c2['sure_caught']}개 모두 잡았다.")
w("")
w(f"**(나) 거짓은 이 피드이다.** 피드 하나의 거짓은 그 변이 다리가 아닐 때 검출되고, 그 변과 2-변 절단을
 이루는 다른 변이 없을 때 위치가 하나로 정해진다(정리). 무작위 그래프 60개의 변 {c1['edges']}개에 10% 거
짓을 넣어 정리와 실제가 {c1['detect_ok']}/{c1['edges']}, {c1['cut_ok']}/{c1['edges']}에서 일치했다. 위치
를 하나로 정한 {c1['unique']}개는 **증명**이다. 2-변 절단 {{e, f}}에서는 'e가 거짓' 세계와 '절단의 한쪽
자산 값이 다르고 f가 거짓' 세계의 관측이 {c4['identical']}/{c4['pairs']}쌍에서 정확히 같았다. 이것이 'e
가 거짓'의 **반례 증인**이다.")
w("")
w(f"**(다) 이 값은 참이다.** 거짓 피드가 많아야 t개이고 두 자산 사이에 변이 겹치지 않는 경로가 k ≥ 2t +
1개 있으면, 경로마다의 비율 곱 가운데 과반의 값이 참 비율이다. 증명: 거짓 t개는 경로 t개만 오염시키고 나
머지 k − t ≥ t + 1개는 참 비율을 낸다. □ 피드 하나를 거짓으로 바꾼 무작위 그래프 80개의 자산 쌍 {t1['pai
rs']:,}개 중 k ≥ 3인 {t1['k_ge3']:,}쌍에서 과반 값이 참 비율과 **{t1['ok']:,}/{t1['k_ge3']:,} 정확히 같
았다(증명).** k ≤ 2t인 {t1['k_le2']}쌍에는 세계 증인이 있다: 한 자산의 피드를 모두 바꾼 거짓은 그 자산
값이 다른 정직한 세계와 관측이 {c3['identical']}/{c3['trials']}쌍에서 같았고, 2-변 절단은 위 (나)의 {c4[
'identical']}쌍이다(**반례**).")
w("")
k1 = KC["K1"]
w(f"**(라) 참값의 후보 구간(정의 25.1의 C).** 가정 A: 거짓 피드는 많아야 t = 1개, 정직한 피드의 잡음은 $
|\\eta| \\leq \\eta_0$. 자산 쌍 (u, v)의 참 비율의 후보는 '피드를 t개 이하 뺀 정직한 세계'가 허용하는 구
간들의 합집합이다. 각 구간은 비율 상한을 변 무게로 둔 그래프의 최소 곱 경로로 정확히 $[1/D_{{vu}},\\ D_{
{uv}}]$이다(그래프 상수). 무작위 시세 그래프 {k1['trials']}개(거짓 피드 1개, 잡음 끝값 포함)의 자산 쌍 {
k1['pairs']}개에서 확인했다.")
w(f"- **건전:** 참 비율이 후보 안에 든 것 **{k1['contains']}/{k1['pairs']}**.")
w(f"- **최소:** 후보 구간의 끝점 {k1['endpoints']}개와 중간점 {k1['mids']}개마다, 관측이 같고 거짓이 1개
 이하이며 참 비율이 그 값인 세계를 정확 유리수로 만들었다. 끝점 **{k1['endpoints_realized']}/{k1['endpoi
nts']}**, 중간점 **{k1['mid_realized']}/{k1['mids']}**.")
w(f"- **후보 밖은 반례:** 후보 밖의 값 {k1['outsides']}개(가장 큰 끝의 1.01배, 가장 작은 끝의 0.99배)에
는 그런 세계가 없었다(**{k1['outside_refuted']}/{k1['outsides']}**). 그 값을 내는 보고는 반례이다.")
w(f"- **크기:** 후보 구간의 상대 폭 중앙값 {k1['width_median']:.4f}(잡음 상한 $\\eta_0 = 0.001$의 약 2배
). 잡음이 0이고 경로가 2t + 1개 이상이면 후보는 한 점이 된다((다)).")
w("")
w("## 26.5 육하원칙 판정표: 블록체인")
w("")
w("::table 0.17,0.08,0.15,0.18,0.24,0.18")
w("무엇을(주장) | 판정 | 누가 | 언제·어디서 | 왜 | 어떻게")
w(f"온체인 exp의 오차 ≤ 주장 | 증명 | 사다리 판정기(기호 실행, 내림 = 나눗셈 − δ) | 모든 x ∈ [0, WAD], n
 = {n0} | δ를 지운 식 = $S_{{{n0}}}$, 내림 누적 {float(eval(osym['D_max'])):.1f} ≤ {2 * n0} wei | 허용치
에 맞는 n을 사다리 칸에서 고른다")
w(f"변이된 exp가 주장을 지킨다 | 반례 {len(ms) - len(proved_mut)} / 증명 {len(proved_mut)} | 같음 | 변이
체 {len(ms)}개 | 증인 x(예: x = WAD − 1에서 오차 5.1e4 wei > 199 wei), 끝나지 않는 실행 {len(timeouts)}(
예: `range(1, x + 1)`은 x = WAD에서 반복 $10^{{18}}$번) | 증명된 1개는 항을 하나 더 쓴 더 정확한 코드")
w(f"가중 중앙값은 정직한 범위 안 | 증명 | 그래프 상수(V = 지분) | 거짓 지분 < 1/2 | 명제, 전수 {b3['case
s']:,}경우 위반 0 | 집계 규칙으로 쓴다")
w(f"같은 주장 | 반례 | 같음 | 거짓 지분 ≥ 1/2 | 증인 지분 {b3['example_major']['stakes']}, 값 {b3['examp
le_major']['vals']} → {b3['example_major']['out']} | 정직 지분을 과반으로 둔다")
w(f"확산(평균) 합의가 거짓에 견딘다 | 반례 | 그래프 상수 (V, T) | 지분 0.4%의 고집 보고자 하나 | 행합 0
이라 정상 상태 = $10^6$, {b12['stubborn']}/{b12['trials']} | 가중 중앙값으로 바꾼다")
w(f"이 보고 묶음에 거짓이 있다 | 증명 {c2['exact_caught']} / 반례 {c2['exact_lies'] - c2['exact_caught']
} | 최소 곱 사이클(그래프 상수) | 잡음 상한 $\\eta_0$ = {c2['eta0']} | 증명: 곱 < 1인 사이클. 반례: 관측
을 내는 정직한 세계 | 반례이면 $\\eta_0$를 줄이거나(거래소를 늘린다) 피드를 더한다")
w(f"거짓은 피드 e이다 | 증명 {c1['unique']} / 반례 | 사이클 서명 | 거짓 하나 | 증명: 서명이 유일. 반례:
2-변 절단의 세계 쌍 {c4['identical']}/{c4['pairs']}, 다리는 서명이 비어 있음 | 절단을 잇는 피드를 더한다
")
w(f"참값의 후보 C | 건전·최소 증명 | 최소 곱 경로(그래프 상수) | 거짓 ≤ 1, 잡음 ≤ $\\eta_0$ | 포함 {KC['
K1']['contains']}/{KC['K1']['pairs']}, 끝점 세계 {KC['K1']['endpoints_realized']}/{KC['K1']['endpoints']
}, 밖은 반례 {KC['K1']['outside_refuted']}/{KC['K1']['outsides']} | 폭을 줄이려면 거래소(잡음)나 피드(경
로)를 더한다")
w(f"이 값은 참이다 | 증명 | 겹치지 않는 경로의 과반 | 거짓 ≤ t, 경로 k ≥ 2t + 1 | {t1['ok']:,}/{t1['k_ge
3']:,} 정확 | 경로 수를 늘려 k ≥ 2t + 1로 만든다")
w(f"같은 주장 | 반례 | 같음 | k ≤ 2t (다리, 2-변 절단, 한 자산의 피드 전부) | 세계 증인 {c3['identical']
}/{c3['trials']}, {c4['identical']}/{c4['pairs']} | 같음")
w("::end")
w("")

# ============================================================ 27 21절
w("# 27 21절의 분야를 분야별 정의로 다시 닫는다")
w("")
w("## 27.1 분야별 정의")
w("")
w("블록체인과 신경망 추론을 뺀 19개 분야를 다시 판정했다. sin의 두 명세는 하나로 합쳤고, 후위 표기 계산
기는 기준 해석기와 실행마다 정확히 비교하는 제5부의 판정(프로그램마다 모든 변수 값에서 증명)을 그대로 쓴
다. 그래서 표는 17행이다. 사용자의 코드를 **부동소수로 그대로 실행한 출력**을, 입력을 정확한 유리수로 읽
```

```
    어 만든 기준값·상한과 비교한다. 비교는 정확하므로 상한을 넘으면 그 입력이 반례 증인이다. 표의 X^는 코드
    가 계산한 값, X*는 정확한 값이다.")
w("")
w("::table 0.13,0.47,0.2,0.2")
w("분야 | 분야별 정의(명세) | 사용자 도구 | 이전 판정(21절)")
for key, d in V.DOM.items():
    od = OLD.get("sin_poly" if key == "sin" else key, {})
    prev = old_word(od.get("label", "")) if od else "—"
    if key == "sin": prev = f"{old_word(OLD['sin_true']['label'])}(참 sin) / {old_word(OLD['sin_poly']['
    label'])}(다항식)"
    w(f"{d['label']} | {cell(d['definition'])} | {d['tool']} | {prev}")
w("::end")
w("")
w("## 27.2 결과와 통과한 변이체의 증명")
w("")
mc_follow = DV["_mc_followup"]; mcT2 = Counter(x["verdict"] for x in t2 if x["kind"] != "원본")
REASON = {
 "matmul": "동치 증명(이전 GF 판정이 모든 입력에서 같은 식): 덮어쓰일 초기값, 정사각 입력에서 `len(A)` =
  `len(B)`",
 "lu": "동치 증명(GF): `len` 교체, 덮어쓰일 초기값",
 "cg": "동치 증명(GF): 첫 `rr`을 `r·p`, `r·b`로 — 시작에서 p = r = b",
 "fft": "동치 증명(모든 n): 모든 칸이 덮어쓰이는 초기값, 음수 첨자 `k − n//2`가 `k + n//2`와 같은 칸, 길
이 1에서 `x[-1]` = `x[0]`, n ≥ 1에서 `n <= 1` = `n == 1`",
 "conv": "동치 증명(GF): 같은 길이에서 `len(a)` = `len(b)`",
 "polydiv": "동치 증명(모든 입력): 모든 칸이 덮어쓰이는 몫의 초기값, 처음에 `len(r)` = `len(a)`",
 "bareiss": "동치 증명(GF)",
 "lstsq": "동치 증명(모든 입력): 쓰이지 않는 행이 더 생김",
 "gd": "명세 증명: 계약에서 `len(b)` = `len(A)`. 시작점 −1에서도 축소 사상이라 $\\|x_k - x^*\\| \\leq q^
k\\|x_0 - x^*\\|$",
 "power": "명세 증명: `y[0]`을 돌려주면 정규화된 $x_0 = 1$에서 $y_0 = \\lambda_1 x_0$이 증폭비 $(1/2)^{1
00}$의 오차로 같다",
 "segint": "원본이 반례(증인: 거의 한 직선 위의 점). 반올림 필터 판은 증명(아래)",
 "isort": "`range(0, …)`은 증명(첫 반복이 항등). `a[j] >= key`는 안정성 명세의 반례(아래)",
 "bsearch": "증명(모든 n, 불변식 '왼쪽은 모두 < x'): `(lo + lo) // 2`와 `lo = lo + 1`은 선형 탐색이 됨,
`lo != hi` = `lo < hi`, 끝에서 `hi` = `lo`",
 "sin": "증명: 항 하나를 더 쓰면 꼬리가 $|x|^{19}/19!$로 줄어 명세를 지킴, 괄호만 다른 것은 같은 값",
 "kahan": "증명: 끝에서 `t` = `s`. 이전 '증명' 3개(보정 파괴)가 반례로 바뀜",
 "lcg": "구현은 점화식으로 모두 반례. 난수성은 아래",
 "mcpi": f"구현 명세(주어진 점의 원 안 개수 × 4/N)의 증인 입력 4종으로 변이체 {sum(mcT2.values())}개 모
두 반례",
}
w("::table 0.17,0.11,0.09,0.09,0.09,0.45")
w("분야 | 원본 | 변이체 | 반례 | 증명 | 통과한 변이체의 증명 근거, 바뀐 판정")
for key, d in V.DOM.items():
    rows = [x for x in DV[key]["rows"] if x["kind"] != "원본"]; cn = Counter(x["new"] for x in rows)
    nfail = cn["반례"]; npass = len(rows) - nfail; orig = DV[key]["orig"]
    if key == "mcpi": nfail, npass = mcT2["반례"], mcT2["증명"]; orig = "δ-증명"
    if key == "isort": nfail += 1; npass -= 1
    if key in ("matmul", "lu", "conv", "sin"): orig = "증명(정리)"
    elif key in ("polydiv", "bareiss", "lcg"): orig = "증명" if OLD[key]["orig"] == "증명" else OLD[key]
    ["orig"]
    elif orig == "표본 통과": orig = "실행마다 증명"
    w(f"{d['label']} | {orig} | {len(rows)} | {nfail} | {npass} | {REASON.get(key, '')}")
w("::end")
w("")
w("**원본의 판정 말.** 부동소수 층의 오라클 주장은 '이 실행의 출력이 분야의 상한을 지킨다'이다. 판정기는
 출력과 입력을 정확한 유리수로 읽어 상한과 비교하므로, 실행마다 증명 또는 반례가 난다(**실행마다 증명**:
 시험한 모든 실행에서 증명, 반례 0). '모든 입력에서 지킨다'는 주장은 상한이 반올림 정리로 주어지는 분야
에서 증명된다(**증명(정리)**): 행렬곱·합성곱(Higham 정리 3.5, 내적의 $\\gamma_n$), LU(정리 9.3), sin(항
마다 상대 오차 $\\gamma_{3k}$, 합 $\\gamma_7$이라 $\\gamma_{28}\\sinh|x|$ 이하), 고친 선분 교차(필터 정
리). 다항식 나눗셈·행렬식·LCG는 정확 산술 분야라 제5부의 판정(모든 입력의 식 등치, 크기 상한)을 그대로
쓴다. 식이 의도한 식과 같다는 것은 어느 분야든 제5부의 GF 판정이 모든 입력에서 증명한다.")
w("")
sf = DV["_seg_filtered"]; ls = DV["_lcg_stats"]
w(f"**선분 교차.** 분야의 정의(정확 산술의 부호)로 원본은 **반례**이다. 증인은 거의 한 직선 위의 점이다(
판정기가 입력 400개 중 처음 찾은 것은 22번째). 반올림 부등식으로 고친다: 행렬식 $\\hat d$와 $m = |t_1| +
 |t_2|$를 부동소수로 계산해 $|\\hat d| > (3u + 16u^2)m$이면 부호가 확정이고, 아니면 정확한 유리수로 다시
 계산한다(슈척의 필터와 같은 상한). 이 판은 모든 입력에서 **증명**이다(필터 정리 + 정확 계산). 같은 입력
 400개에서도 {sf['verdict']}.")
w("")
w(f"**삽입 정렬의 안정성.** 삽입 정렬은 안정 정렬로 정의된다. 증인 [1a, 0b, 1c]에서 원본은 [0b, 1a, 1c]
를 내고(증명), `a[j] >= key` 변이체는 [0b, 1c, 1a]를 낸다(**반례**). 정렬 결과는 맞으므로 이전 판정과 값
만 보는 명세는 이것을 통과시켰다.")
w("")
w(f"**난수성(큰수·비율 편차).** 맞는 LCG의 출력 20,000개에서 맨 위 비트의 빈도 {ls['top']:.4f}는 호에프
딩 허용치 t = {ls['t']:.4f}(δ = $10^{{-6}}$) 안이다(**δ-증명**). 맨 아래 비트가 바로 앞과 같은 비율은 {l
s['low_rep']:.4f}이다. 곱 1103515245와 덧셈 12345가 모두 홀수라 법 2에서 $x_{{n+1}} \\equiv x_n + 1$이고
, 맨 아래 비트는 0과 1을 번갈아 낸다. 시작값 997개 모두에서 확인했다(**'아래 비트가 난수'의 반례**, 모든
 시작값에서 증명된 반례). 구현은 맞지만 아래 비트를 난수로 쓰면 안 된다.")
w("")
```

```
w(f"**몬테카를로 π.** 확률 명세 $P(|\\hat\\pi - \\pi| > 4t) \\leq 2e^{{-2Nt^2}} = \\delta$로 원본은 **δ-증명**(N = 20,000과 $10^6$ 모두). π는 마친 공식의 구간에서 뽑은 연분수 [{V.PI_CF[0]}; {', '.join(str(q) for q in V.PI_CF[1:6])}, …]의 이웃한 두 수렴분수로 위아래에서 묶었다(폭 {float(V.PI_HI - V.PI_LO):.1e}). N = 20,000에서 통과한 15개 중 12개는 N = $10^6$에서 δ-반례가 되었다(원 판정을 `x·y + y² ≤ 1` 꼴로 바꾼 것, 넓이의 4배가 π보다 0.02 크다). 치우침 b를 1 − δ로 반드시 잡으려면 $N \\geq 32\\ln(2/\\delta)/b^2$이다. 같은 변이체는 구현 명세로는 증인 입력 하나로 바로 반례이다: 원 안이지만 $xy + y^2 > 1$인 점 (0.5, 0.85). 남은 `< 1`과 `hits`의 시작값 ±1도 증인 입력(원 위의 점 (1, 0), 모두 원 밖인 점들)으로 반례이다. 그래서 몬테카를로 π의 변이체는 **{mcT2['반례']}개 모두 반례**로 닫힌다.")
w("")
w("**이전 판정의 빈틈.** LU·바라이스·최소제곱에서 `range(i / 1, n)` 꼴의 변이체 3개를 이전 판정은 '증명'이라고 했다. 이전의 정확 재실행은 정수끼리의 `/`를 유리수로 바꾸는데, 파이썬에서 `/`는 부동소수를 내 `range`가 실패한다. 이전 실험은 부동소수 실행의 예외를 따로 세었으므로 표의 수는 맞지만, 판정기는 실행 의미를 바꾸고 있었다. 부동소수로 그대로 실행하는 이번 판정은 이것을 반례(실행 실패)로 낸다.")
w("")
w("## 27.3 육하원칙 판정표: 21절의 분야")
w("")
w("::table 0.17,0.1,0.17,0.17,0.22,0.17")
w("무엇을(주장) | 판정 | 누가 | 언제·어디서 | 왜 | 어떻게")
w("이 실행의 출력이 분야 명세를 지킨다 | 증명 또는 반례(실행마다) | 반올림 부등식, 사다리, π, 증폭, 비율 편차 | 각 분야의 입력 계약, 실행 하나하나 | 출력과 상한을 정확한 유리수로 비교 | 반례이면 그 입력이 증인")
w("원본이 모든 입력에서 명세를 지킨다 | 증명(정리) 4 / 증명 2 / 유한 증명 3 / δ-증명 1 / 실행마다 증명 6 | 같음 | 각 분야의 입력 계약 | 반올림·꼬리 정리, 정렬·탐색은 모든 순서형(n ≤ 6, 7) | 실행마다 증명인 분야는 실행할 때마다 판정기를 함께 돌린다")
w("선분 교차 원본이 정확한 부호를 낸다 | 반례 | 반올림 부등식 | 거의 한 직선 위의 점 | 부동소수 행렬식의 부호가 뒤집힌 입력 | 반올림 필터 + 정확 계산(증명된 판)")
w("변이체가 분야 명세를 지킨다 | 반례 또는 증명 | 같음 | 분야마다 최대 150개 | 반례는 증인 입력, 증명은 27.2절 표의 근거 | 반례 변이체는 버그이다")
w("삽입 정렬이 안정하다 | 증명(원본) / 반례(`>=`) | 정렬 검산식 + 안정성 | [1a, 0b, 1c] | 같은 키의 순서가 바뀜 | `>` 비교를 쓴다")
w("LCG의 아래 비트가 난수이다 | 반례 | 비율 편차 + 법 2의 점화식 | 모든 시작값 | 비트가 0, 1을 번갈아 냄 | 위 비트만 쓴다")
w("몬테카를로 π의 변이체가 명세를 지킨다 | 반례(모두) | 비율 편차 + π + 구현 명세 | 증인 입력 4종 | 예: (0.5, 0.85)는 원 안 | —")
w("::end")
w("")

# ============================================================ 28 ML
lin = M4["linear"]
w("# 28 ML 오라클")
w("")
w("## 28.1 정의와 주장")
w("")
w("ML의 오라클 문제는 '이 예측이 맞는가'를 가릴 정답 명세가 없다는 것이다. 정답은 사람의 표지에만 있고, 운영 중의 입력에는 표지가 없다. 판정할 주장을 이렇게 둔다.")
w("")
w("- (가) 이 예측 하나가 맞다는 것이 자료(입력·모델·학습·보정 자료)로 보증된다.")
w("- (나) 판정기가 받아들인 예측들의 참 오류율이 r* 이하이다.")
w("- (다) 모델의 참 정답률이 구간 안이다. (라) 예측 집합이 정답을 포함한다.")
w("- (마) 운영 입력의 분포가 보정 자료와 같다. (바) 입력이 ε 안에서 바뀌어도 예측이 같다. (사) 추론 코드에 출력을 바꾸는 버그가 없다.")
w("")
w(f"**(가)의 세계 증인(M0).** 입력·학습 자료·보정 자료·모델이 모두 같고 시험 입력 몇 개의 정답만 다른 두 세계를 만들었다. 받아들인 예측 {M0['accepted']}개 가운데 {M0['flipped']}개의 정답을 세계 B에서 다른 부류로 바꾸었다. 판정기의 입력은 두 세계에서 같으므로 출력도 같다. 받아들인 예측의 오류는 세계 A에서 {M0['errors_A']}개, B에서 {M0['errors_B']}개였다. 그래서 (가)는 모든 입력에서 **반례**이고, 증인은 세계 B이다. '어떻게': 그 입력에 표지를 달거나(사람), (나)의 보증으로 바꾸어 쓴다.")
w("")
w("## 28.2 실험")
w("")
w("참값이 필요한 검증은 무한히 뽑을 수 있는 합성 분포(8차원 4부류, 부류마다 다른 공분산)에서 했다. 모델은 은닉층 32의 ReLU 망. 시행마다 학습 1,500, 보정 1,000, 참값 측정 200,000개를 새로 뽑았다. δ = 0.05.")
w("")
cov = np.array(G["M2_cov"]); risk = np.array(G["M2_risk"]); m3c = np.array(G["M3_cover"]); m3s = np.array(G["M3_size"]); acct = np.array(G["acc_true"])
w(f"- **(나) 선택적 판정(비율 편차의 체르노프 KL 꼴 + 합집합 상한).** 확신도 문턱마다 받아들인 예측의 오류율 위 상한을 잡고, r* = 0.05 이하인 가장 낮은 문턱을 고른다. 100번 시행에서 참 오류율이 r*를 넘은 것 **{G['M2_viol']}/{G['trials'] - G['M2_none']}**(보증 ≤ 5%). 받아들인 비율 평균 {cov.mean():.3f}, 참 오류율 평균 {risk.mean():.4f}, 최대 {risk.max():.4f}. 처음 쓴 호에프딩 상한은 너무 느슨해 문턱을 정하지 못했고, KL 꼴로 바꾸었다.")
w(f"- **(다) 정답률 구간(호에프딩).** 참 정답률(평균 {acct.mean():.3f})을 덮은 시행 **{G['M1_cover']}/{G['trials']}**.")
k2 = KC["K2"]
w(f"- **(라) 예측 집합 = ML의 정답 후보 C(분할 등각 예측, 순위 논증 — 사용자 도구가 아님).** α = 0.1에서 참 포함률 평균 {m3c.mean():.4f}, 집합 크기 평균 {m3s.mean():.2f}(**건전**). **최소:** 문턱을 보정 점수의 k = ceil((n+1)(1 − α))번째로 두면 포함률은 정확히 k/(n+1) ≥ 1 − α이고, 한 칸 낮춘 k − 1번째면 (k − 1)/(n+1) < 1 − α이다. n = {k2['n_cal']}에서 이론값 {k2['theory_k']:.4f}와 {k2['theory_k1']:.4f}, 600번 시행의 참 포함률 {k2['cov_k']:.4f}와 **{k2['cov_k1']:.4f}**(표준오차 {k2['se']:.4f}). 한 칸만 줄여도 보증이 깨지므로 이 방식 안에서 C는 더 줄일 수 없다. 집합 크기가 1이면 그 부류가 C의 유일한 원소이다.")
w("- **(마) 분포 이동 검출(정답 없음).** M5는 확신도 분포의 DKW 검정, M5'는 예측 부류 빈도의 비율 편차
```

```
    검정이다. 둘 중 하나가 울리면 δ-반례이다.")
w("")
w("::table 0.14,0.14,0.14,0.18,0.2,0.2")
w("이동 | M5 경보 | M5′ 경보 | (나)의 위반 | 경보 없는데 위반 | (라)의 포함률")
for k, lab in (("None", "없음"), ("noise", "잡음"), ("scale", "크기 1.8배"), ("prior", "부류 비율")):
    r = S2[k]; w(f"{lab} | {r['alarm_conf']}/{r['trials']} | {r['alarm_freq']}/{r['trials']} | {r['M2_viol']}/{r['M2_n']} | {r['M2_viol_silent']} | {np.mean(r['M3_cover']):.3f}")
w("::end")
w("")
w(f"이동이 없을 때 경보 {S2['None']['alarm_any']}/40. 잡음 이동에서 (나)가 {S2['noise']['M2_viol']}/40번 깨졌고 경보가 {S2['noise']['alarm_any']}/40번 울렸다. 크기·부류 비율 이동은 보증을 깨지 않았지만 가정이 깨졌으므로 경보가 울렸다. **경보가 없는데 보증이 깨진 시행은 모든 경우에 0이다.**")
w("")
w(f"- **(바) 강건성.** 선형 소프트맥스는 $r(x) = \\min_j (z_c - z_j)/\\|w_c - w_j\\|_2$ 안에서 예측이 같다(정확한 부등식). 시험 입력 {lin['n']}개에서 반지름의 0.999배로 가장 나쁜 방향을 밀면 바뀐 것 {lin['flip_inside']}, 1.001배면 {lin['flip_outside']}: 증명 반지름이 참 반지름이다. ReLU 망(32-32)은 립시츠 상한 $L = {M4b['L']:.1f}$의 증명이 건전하지만 느슨하다(평균 {M4b['r_loc']:.3f}, 공격으로 뒤집힌 반지름 평균 {M4b['emp']:.2f}). 그래서 **완전 판정**을 했다: $\\|x' - x\\|_\\infty \\leq \\epsilon$에서 부류마다 $\\max(z_j - z_c)$를 혼합 정수 계획으로 정확히 푼다(빅-M, 경계는 구간 전파). 최댓값이 0 이하면 증명, 0보다 크면 그 해를 순전파해 예측이 바뀌는 것을 확인한 반례 증인이다.")
w("")
w("::table 0.1,0.12,0.12,0.12,0.16,0.18,0.2")
w("ε | 증명 | 반례 | 경계 | 립시츠가 증명 | PGD가 찾은 반례 | 둘 다 못 정한 것 → 완전 판정이 닫음")
for e, r in M4x.items(): w(f"{e} | {r['proved']} | {r['counter']} | {r['border']} | {r['lip_proved']} | {r['pgd_found']} | {r['undecided_by_incomplete']} → {r['closed_by_milp']}")
w("::end")
w("")
w(f"입력마다 증명 또는 반례가 났고 경계(풀이기의 허용오차 안에 붙은 경우)는 0이다. 립시츠 증명이 틀린 적(완전 판정의 반례와 충돌)과 PGD 반례가 틀린 적은 없다. ε = 0.3과 0.5에서 완전 판정은 PGD가 놓친 반례를 각각 {M4x['0.3']['counter'] - M4x['0.3']['pgd_found']}개, {M4x['0.5']['counter'] - M4x['0.5']['pgd_found']}개 더 찾았다.")
w("")
lb = LB["rows"]
w(f"- **(사) 추론 코드의 버그(손글씨 숫자 1,797장).** 보정 정답률 {M6['acc_ref']:.3f}.")
w("")
w("::table 0.4,0.14,0.23,0.23")
w("버그 | 운영 정답률 | 정답 없이(M5) | 정답으로(정답률 구간)")
for r in M6["rows"]: w(f"{r['bug']} | {r['acc']:.3f} | {'δ-반례' if r['alarm_unlabeled'] else '통과'} | {'δ-반례' if r['alarm_labeled'] else '통과'}")
w("::end")
w("")
w(f"부류 번호를 하나 밀어 낸 버그는 확신도 분포를 바꾸지 않아 정답 없는 판정기의 입력이 정상과 같다(세계 증인). 정답 표본으로 닫는다: 정답 5, 10, 20, 40개에서 검출률 " + ", ".join(f"{lb[k]['detect_bug']:.2f}" for k in sorted(lb, key=int)) + ", 정상 코드의 거짓 경보율 " + ", ".join(f"{lb[k]['false_alarm']:.2f}" for k in sorted(lb, key=int)) + "(각 200번). '운영 자료로 표준화'는 코드로는 통계 누수지만 출력의 정답률을 바꾸지 않았다(0.958). 출력을 명세로 삼는 주장 (사)에서는 증명이고, 누수 자체는 코드 검토의 몫이다.")
w("")
w(f"**실제 자료 적용(손글씨 숫자, 보정 {demo['n_cal']}장, 시험 {demo['n_test']}장).** 정답률 {demo['acc_cal']:.3f} ± {demo['ci']:.3f}(시험 {demo['acc_test']:.3f}). 선택적 판정(r* = {demo['sgr_rstar']}): 받아들인 비율 {demo['sgr_cov']:.3f}, 받아들인 예측의 시험 오류율 {demo['sgr_risk_test']:.4f}. 예측 집합(α = {demo['conf_alpha']}): 포함률 {demo['conf_cover']:.3f}, 크기 평균 {demo['conf_size']:.2f}.")
w("")
w("## 28.3 육하원칙 판정표: ML")
w("")
w("::table 0.17,0.1,0.15,0.17,0.23,0.18")
w("무엇을(주장) | 판정 | 누가 | 언제·어디서 | 왜 | 어떻게")
w(f"(가) 이 예측 하나가 자료로 보증된다 | 반례 | 세계 증인(M0) | 모든 입력 | 정답만 다른 세계 B에서 받아들인 {M0['flipped']}개가 모두 틀림 | 표지를 단다, 또는 (나)로 쓴다")
w(f"(나) 받아들인 예측의 오류율 ≤ r* | δ-증명 | 비율 편차(KL) 선택 판정기 | (마)의 경보가 없을 때 | 위반 {G['M2_viol']}/{G['trials']}, 경보 없는 위반 0 | 넘긴 입력은 사람이 표지를 단다")
w(f"(다) 정답률이 구간 안 | δ-증명 | 호에프딩 | 같은 분포 | {G['M1_cover']}/{G['trials']} 덮음 | —")
w(f"(라) 예측 집합 C가 정답을 포함하고 최소이다 | δ-증명(1 − α) | 분할 등각(순위 논증) | 교환 가능성 | 포함률 {m3c.mean():.3f}, 한 칸 낮추면 {KC['K2']['cov_k1']:.4f} < 0.9 | C가 크면 정답 표지를 더한다")
w(f"(마) 분포가 같다 | δ-증명 또는 δ-반례 | DKW + 예측 빈도의 비율 편차 | 운영 입력 2,000개 | 세 가지 이동 모두 40/40 경보, 이동 없음 0/40 | 경보이면 보정 자료를 새로 모은다")
w(f"(바) ε 안에서 예측이 같다 | 증명 또는 반례 | 선형: 정확한 반지름. 망: 혼합 정수 계획 | 입력마다, ε = 0.1, 0.3, 0.5 | 증명은 최적값 ≤ 0, 반례는 확인된 x′ | 반례 x′를 학습에 넣는다")
w("(사) 출력을 바꾸는 버그가 없다 | δ-반례(4종) / δ-증명 | M5 + 정답률 구간 | 운영 입력, 정답 5개 | 표의 경보, 부류 밀림은 정답 5개로 | 버그를 고친다")
w("::end")
w("")

# ============================================================ 29 닫힘 점검
w("# 29 사용자 도구로 했다는 근거")
w("")
w("제6부의 판정을 하나씩 들어, 판정의 기준(상한, 허용치, 후보, 증인)이 어느 도구에서 나왔는지 적었다. '계산 수단'은 기준을 계산하는 데만 쓴 일반 도구이다.")
w("")
w("::table 0.3,0.3,0.4")
w("판정 | 기준을 만든 사용자 도구 | 계산 수단(사용자 도구 아님)")
```

```
w("온체인 exp의 오차 증명 | 나머지 사다리(꼬리 $r^{n+1}\\epsilon_n$, n 선택, 기준값 $S_{40}$ 구간), 반올림 부등식(내림 δ) | 기호 실행(sympy)")
w("보고 집계(확산 합의·가중 중앙값) | 그래프 상수: V = 지분, T = 결합, 행합 0·대칭·메츨러 | 정확 유리수 선형 풀이")
w("거짓 검출·위치·참값·후보 구간 | 그래프 상수(시세 그래프의 사이클, 최소 곱 경로, 겹치지 않는 경로), 사다리(잡음 띠 $U_n$) | 플로이드–워셜, 최대 흐름")
w("행렬곱·LU·합성곱·카한 합 | 반올림 부등식($u$, $\\gamma_k$) | 정확 유리수")
w("CG·최소제곱·거듭제곱법의 허용치 | 나머지 사다리(허용치 $\\epsilon_{10}, \\epsilon_{12}$) | 정확 유리수 풀이")
w("경사하강·거듭제곱법의 수렴 | 증폭 i(한 걸음 증폭률 $|1 - \\eta\\lambda|$, 비 $|\\lambda_2/\\lambda_1|$) | 게르시고린 구간")
w("sin·FFT의 기준값 | 나머지 사다리(테일러 꼬리), π의 연분수 수렴분수 | 마친 공식(π 구간의 출처)")
w("선분 교차의 고친 판 | 반올림 부등식(필터 $(3u + 16u^2)m$) | 정확 유리수")
w("몬테카를로 π, LCG의 난수성 | 큰수의 법칙·비율 편차(호에프딩), π의 연분수 | —")
w("ML (나)(다)(마) | 비율 편차(호에프딩, 체르노프 KL, DKW) | 다층 퍼셉트론 학습")
w("ML (가) 세계 증인, 모든 반례의 처방 | 육하원칙 보고(처방, 22.9절) | 세계 B의 구성(정답만 바꾼 표본)")
w("ML (바) 선형 모델의 반지름 | 부등식(정확한 여유/노름) | —")
w("ML (라) 예측 집합 | — | 분할 등각 예측(순위 논증)")
w("ML (바) ReLU 망의 완전 판정 | 부등식(구간 전파로 빅-M 경계) | 혼합 정수 계획(HiGHS)")
w("다항식 나눗셈·행렬식·정렬·탐색·후위 표기 | — (분야의 검산식이 완전한 오라클) | 정확 유리수, 순서형 전수")
w("::end")
w("")
w("판정 열다섯 묶음 가운데 열한 묶음은 기준 전체가 사용자 도구에서 나왔다. 두 묶음(ML의 세계 증인, ReLU 망의 완전 판정)은 기준의 일부가 사용자 도구이다(처방의 육하원칙, 정수 계획의 경계를 주는 구간 부등식). ML의 예측 집합(순위 논증)은 사용자 도구가 아니고, 검산식이 완전한 분야는 도구가 필요 없었다. 그래서 주장은 이렇다: **이 부의 판정 기준은 ML의 예측 집합과 신경망 완전 판정의 정수 계획을 빼면 모두 사용자의 도구에서 나왔다.** 특히 정의 25.1의 후보 집합 C는 블록체인에서 그래프 상수의 최소 곱 경로로, 21절 분야에서 사다리·π·반올림 부등식의 구간으로, ML에서 비율 편차의 보증으로 계산된다.")
w("")
w("# 30 닫힘 점검")
w("")
w("이 부의 모든 주장을 정의 25.1과 25.2로 점검했다. 판정되지 않고 남은 주장은 없다.")
w("")
w("::table 0.17,0.25,0.48,0.1")
w("대상 | 정답 후보 C | 판정 | 남은 주장")
w(f"블록체인 계산 층 | 사다리 구간 $[S_{{40}}, S_{{40}} + r^{{41}}\\epsilon_{{40}}]$의 정수 | 원본 증명(모든 x), 변이체 {len(ms)}개는 반례 {len(ms) - len(proved_mut)} 또는 증명 {len(proved_mut)} | 0")
w(f"블록체인 집계 층 | 정직한 보고값의 [최소, 최대] | 가중 중앙값: 거짓 지분 < 1/2 증명, ≥ 1/2 반례. 확산 합의 반례. 구현 변이체 {len(b4['rows'])}개 반례 또는 증명 | 0")
w(f"블록체인 진실 층 | 최소 곱 경로의 구간 합집합 | 거짓 있음: 증명(사이클) 또는 반례(정직한 세계). 위치: 증명(유일한 서명) 또는 반례(절단의 세계 쌍). 참값: 증명(k ≥ 2t + 1) 또는 반례(k ≤ 2t의 세계 쌍) | 0")
w("21절 분야 | 기준값 ± 분야의 상한(반올림·꼬리·허용치) | 실행의 출력: 실행마다 증명 또는 반례. 원본: 증명(정리)·유한 증명·δ-증명·실행마다 증명, 선분 교차는 반례와 증명된 고친 판. 변이체: 반례(증인 입력) 또는 증명(27.2절의 근거) | 0")
w("계산할 수 없는 문제(정지 문제) | 좁혀 가는 목록 $C_t$ (정의 25.2) | 건전 700/700, 단조, 맨 앞 후보가 참답으로 수렴(참답을 아는 700개 모두), 남은 목록에는 육하원칙 처방 | 0")
w("ML | 예측 집합(등각), 받아들인 예측은 한 점 | (가) 반례(세계 증인), (나)–(라) δ-증명, (마) δ-증명/δ-반례, (바) 입력마다 증명 또는 반례(완전 판정), (사) δ-반례/δ-증명 | 0")
w("::end")
w("")
w("**실험이 잡은 판정기의 실수.** 이 부를 쓰는 동안 실험이 판정기 자체의 실수를 잡았고, 모두 고친 뒤 다시 돌렸다. 시세 사이클의 띠에서 역방향 피드의 $1/(1 - \\eta_0)$를 빠뜨림(정직한 사이클 1,800개 중 거짓 경보 8개), 항을 하나 더 쓴 exp를 증명하지 못함, 거듭제곱법의 시험 입력이 지배 고유벡터를 늘 $e_1$로 만들어 곱셈 변이체를 놓침, 몬테카를로 π의 해상도, 선택적 판정의 느슨한 상한, 세계 쌍 실험에서 거짓 배율 1이 뽑힐 수 있었음(처음 267쌍 중 231쌍만 같음, 배율 1을 뺀 뒤 267쌍 모두 같음), 증인 비교에서 부동소수 0.4를 유리수 2/5와 비교함이다.")
w("")
w("# 31 제6부 코드")
w("")
w("::code chain_ladder.py chain_ladder.py — 26.2절: 나머지 사다리로 온체인 exp를 모든 입력에서 판정")
w("::code chain_graph.py chain_graph.py — 26.3–26.4절: 그래프 상수로 집계·진실 층")
w("::code chain_fig.py 작동 확인 chain_fig.py — 그림 4")
w("::code anytime_oracle.py 작동 확인 anytime_oracle.py — 정의 25.2: 정지 문제의 좁혀 가는 후보 목록")
w("::code candidate_sets.py 작동 확인 candidate_sets.py — 정의 25.1: 블록체인 후보 구간의 건전·최소, 예측 집합의 최소")
w("::code closure_extra.py 작동 확인 closure_extra.py — 26.4절 (다), 27.2절의 증인 입력, LCG의 법 2 점화식")
w("::code domains_v2.py domains_v2.py — 27절: 21절 분야의 재판정")
w("::code ml_oracle.py ml_oracle.py — 28절: ML 오라클(완전 판정 포함)")
w("::code make_part6.py make_part6.py — 제6부의 본문을 결과에서 만든다")
w("")

# ============================================================ 31 새 문제
w("# 32 새 문제를 만났을 때")
w("")
w("새 문제는 질문 0으로 정의 25.1과 25.2 가운데 하나를 고르고, 질문 1–3으로 정답 후보를 무엇으로 만드는지, 질문 4–13으로 쓸 장치를 정한다.")
w("")
w("::table 0.07,0.33,0.34,0.26")
```